

Python 程序设计实践教程

张建 龚兰兰 主编
李嘉伟 王丽平 伍桐 副主编

清华大学出版社
北 京

内容简介

本书为 Python 程序设计实践课程教材。教材在结构设计上以实践案例为驱动，旨在通过实践教学，巩固、提高理论知识点的学习。教材每章对所用的理论知识进行概括，以计算思维培养为主线，通过范例程序、改错程序、完善程序、独立开发程序等循序渐进、逐步加深难度、举一反三的方式为学者提供了实践操作指导。

此外，教材针对爬虫、数据分析、数据可视化等模块，融入了综合三个模块的数据分析案例，从数据的获取、清洗到数据可视化的呈现，全方位覆盖数据分析相关知识，切实提升学习者综合动手开发的能力，助力其掌握 Python 在数据分析领域的实用技能。

教材坚持“项目引领、案例驱动”，以 OBE 理念开展各个章节、模块的开发，力图培养学生“学以致用”的实践动手能力，为后续课程学习提供扎实的基础。

前言

为了适应当今信息社会飞速发展的需求，培养学生的计算思维，使学生具备利用计算机分析问题、解决问题的基本技能，并能够胜任专业研究与应用的需求，大多数高等院校开设了程序设计课程。由于 Python 语言简单易学、第三方库众多，在应用开发、数据可视化分析、机器学习、人工智能等领域有着广泛的应用，较多的院校将该语言作为计算机程序设计与其他专业进行交叉融合开展计算思维的培养。因此一本有针对性的提升学生实践动手开发能力培养的入门指导书显得十分重要，是获得良好教学效果的保障。

本书为 Python 程序设计实践课程教材。内容包括对每个章节的学习目标和相关要求进行明确，对应的理论知识进行了总结，并采用循序渐进、逐步加深难度，为学者提供了实践操作的指导。

本书由苏州城市学院张建、龚兰兰、王丽平、伍桐以及苏州瑞立思科技有限公司李嘉伟等人编著，在编写过程中得到了学院的多位同仁支持，在此表示感谢！

本书为以下项目的研究成果：

1. 2025 年江苏省高等教育教改研究课题：基于专业群融合方式构建应用型本科高校 AI 通识课“1+2+X”模块化内容建设与实践（2025JGYB585）

2. 2025 年江苏本科高校“人工智能通识课程、基础课程教学改革研究”专项课题：分层分类的“1+2+X”人工智能通识课程教材建设（2025ZNT-37）

3. 2024 年苏州城市学院高等教育改革研究课题：AI 时代下计算机专项基础课程的改革研究与实践（24JGJ06）

4. 2024 年江苏省高校人工智能通识教育教学改革研究专项课题：面向应用型本科高校的人工智能通识教育与专业融合方案研究与实践（2024AIGE73）

5. 苏州城市学院第二批校级基层教学组织：公共计算机虚拟教研室

6. 2021 年全国高等院校计算机基础教育研究会计算机基础教育教学研究项目：构建“课证融通 以证促学”的计算机基础课程教学资源建设研究（2021-AFCEC-080）

由于作者水平有限，本书难免存在错漏之处，敬请使用本教程的广大师生及其他读者批评指正。本书提供上机实践的参考答案，供教师参考，有需要的教师可与作者联系。

作者

2025 年 12 月

目 录

第 1 章 绪论	1
1.1 学习目标与要求	1
1.2 知识要点	1
1.2.1 Python 语言优点	1
1.2.2 Python 语言应用领域	1
1.2.3 Python 开发环境安装	1
1.2.4 IDLE 两种模式使用	4
1.2.5 IDLE 交互模式快捷键	6
1.3 实践与指导	6
实践 1 计算圆柱体的体积	6
实践 2 计算矩形周长和面积	7
实践 3 程序分析及完善	8
实践 4 编写程序输出 HelloWorld	8
实践 5 编写程序输出*号组成的三角形	9
实践 6 编写程序输出一个字符串	9
第 2 章 Python 基础	11
2.1 学习目标与要求	11
2.2 知识要点	11
2.2.1 基本数据类型	11
2.2.2 内置函数	12
2.2.3 常见的第三方库	14
2.2.4 编码规范	15
2.2.5 运算符	16
2.3 实践与指导	16
实践 1 常量变量	16
实践 2 计算一元二次方程的根	17
实践 3 程序分析及完善	18
实践 4 四则运算	19
实践 5 数位数字和	20
实践 6 年龄计算	20
第 3 章 选择结构	22
3.1 学习目标与要求	22
3.2 知识要点	22
3.2.1 单分支结构	22
3.2.2 双分支结构	22
3.2.3 多分支结构	23
3.3 实践与指导	24
实践 1 发放购物券	24
实践 2 一日游收费	25
实践 3 BMI	26
实践 4 游戏人物血量	27

实践 5 归一化	28
实践 6 天数	28
第 4 章 循环结构	30
4.1 学习目标与要求	30
4.2 知识要点	30
4.2.1 while 循环	30
4.2.2 扩展 while 循环	30
4.2.3 for 循环	31
4.2.4 break 和 continue	31
4.2.5 循环嵌套	31
4.3 实践与指导	31
实践 1 阅读推演	31
实践 2 字符类别统计	32
实践 3 偶数拆分为两个素数之和	33
实践 4 四位升序数	34
实践 5 Fibonacci 数列	34
实践 6 e^x 计算	35
第 5 章 列表与元组	37
5.1 学习目标与要求	37
5.2 知识要点	37
5.2.1 列表	37
5.2.2 元组	38
5.3 实践与指导	39
实践 1 阅读推演	39
实践 2 随机数个数统计	40
实践 3 生成不重复的随机数	41
实践 4 评委评分	42
实践 5 成绩数据统计	43
实践 6 选择排序	44
第 6 章 字典与集合	46
6.1 学习目标与要求	46
6.2 知识要点	46
6.2.1 字典	46
6.2.2 集合	47
6.3 实践与指导	48
实践 1 字典集合常见操作	48
实践 2 天数	49
实践 3 生成不重复的随机数	50
实践 4 公司合并	51
实践 5 歌手海选	51
实践 6 字符统计	52
第 7 章 函数和模块	54
7.1 学习目标与要求	54
7.2 知识要点	54
7.2.1 函数定义	54

7.2.2 函数参数	54
7.2.3 变量作用域	54
7.2.4 匿名函数	54
7.2.5 递归函数	54
7.2.6 内置函数	55
7.2.7 模块	55
7.3 实践与指导	55
实践 1 函数的调用过程	55
实践 2 组合数	57
实践 3 有序序列插入数据	58
实践 4 素数	59
实践 5 猴子吃桃	59
实践 6 自幂数	60
第 8 章 类和对象	63
8.1 学习目标与要求	63
8.2 知识要点	63
8.2.1 类	63
8.2.2 对象	64
8.2.3 属性和方法	64
8.2.4 继承	66
8.3 实践与指导	67
实践 1 动物类的创建	67
实践 2 计算学生年龄	68
实践 3 购物车结算	69
实践 4 教师、学生类的继承	71
实践 5 图形计算系统	74
实践 6 薪资管理系统	75
第 9 章 文件处理	77
9.1 学习目标与要求	77
9.2 知识要点	77
9.2.1 文件	77
9.2.2 文件的访问	77
9.3 实践与指导	78
实践 1 程序阅读	78
实践 2 求均值	79
实践 3 最高得分名单	80
实践 4 字符统计	81
实践 5 村长选举	82
实践 6 成绩统分析	83
第 10 章 中文分词和词云	86
10.1 学习目标与要求	86
10.2 知识要点	86
10.2.1 分词	86
10.2.2 jieba	86
10.2.3 关键词提取	86

10.2.4 词云图	87
10.3 实践与指导	88
实践 1 中文分词三种模式	88
实践 2 词典的维护	89
实践 3 文件关键词提取	90
实践 4 英文小说词云图制作	91
实践 5 制作《给青年的十二封信》中文词云图	92
实践 6 制作《苏东坡传》关键词词云图	93
第 11 章 图形界面开发	95
11.1 学习目标与要求	95
11.2 知识要点	95
11.2.1 Tkinter 库	95
11.2.2 Tkinter 组件	95
11.2.3 布局管理器	96
11.2.4 事件处理	98
11.3 实践与指导	99
实践 1 猜数字游戏	99
实践 2 背单词游戏	101
实践 3 单位转换器	102
实践 4 用户登录界面	105
第 12 章 网络爬虫	107
12.1 学习目标与要求	107
12.2 知识要点	107
12.2.1 爬虫介绍	107
12.2.2 爬虫伦理	107
12.2.3 常见的爬虫	107
12.2.4 selenium	107
12.3 实践与指导	109
实践 1 爬取豆瓣 Top250 电影数据	109
实践 2 爬取苏州空气质量数据	111
实践 3 爬取成绩数据	112
第 13 章 数据分析与可视化	115
13.1 学习目标与要求	115
13.2 知识要点	115
13.2.1 数据分析与可视化介绍	115
13.2.2 数据分析	115
13.3 实践与指导	120
实践 1 苏州空气质量数据分析	120
实践 2 成绩数据分析——阶段性成绩对比柱状图	122
实践 3 成绩数据分析——时长和卷面分关系的散点图绘制	123
实践 4 证券指数数据分析	125
第 14 章 综合案例	128
14.1 学习目标与要求	128
14.2 知识点	130
14.2.1 数据获取	130

14.2.2 数据清洗	131
14.2.3 绘制图	132
14.3 实践与指导	133
实践 1 G7+1 国家 GDP 变化分析	133
实践 2 主要国家人口变化分析	137
实践 3 主要国家人口出生率变化分析	139
第 15 章 AI+编码	142
15.1 学习目标与要求	142
15.2 知识点	142
15.2.1 集成开发环境	142
15.2.2 Trae AI IDE 的使用	147
15.2.3 通用大模型赋能编码	148
参考文献	158

第 1 章 绪论

1.1 学习目标与要求

- (1) 了解 Python 语言优点及应用领域
- (2) 掌握 Python 开发环境的安装
- (3) 掌握使用 Python 交互模式与文件模式编写程序的方法及其区别

1.2 知识要点

本实验主要涉及 Python 开发环境的安装以及它的两种运行方式——Shell 交互方式和文件执行方式的使用。

1.2.1 Python 语言优点

- (1) 语法优美、简单易学
- (2) 可扩展性好、开发效率高
- (3) 开源和跨平台等。

1.2.2 Python 语言应用领域

- (1) Web 开发
- (2) 网络编程
- (3) 科学计算
- (4) 数据库应用
- (5) 多媒体开发
- (6) 游戏开发等。

1.2.3 Python 开发环境安装

Python 集成开发环境（IDE：Integrated Development Environment）较多，如：PyCharm、VSCode、Jupyter Notebook 等。其中，Python IDLE（Integrated Development and Learning Environment，集成开发与学习环境）是官方内置的一个简单、小巧的 Python IDE，安装好 Python 环境后，即可通过 IDLE 进行 Python 的学习与程序开发。

步骤 1：下载 Python。官方下载网址为：<https://www.python.org>。图 1-1、1-2 为 Python 安装包的下载路径图。本教材选用的是 3.10.x，64 位的版本。

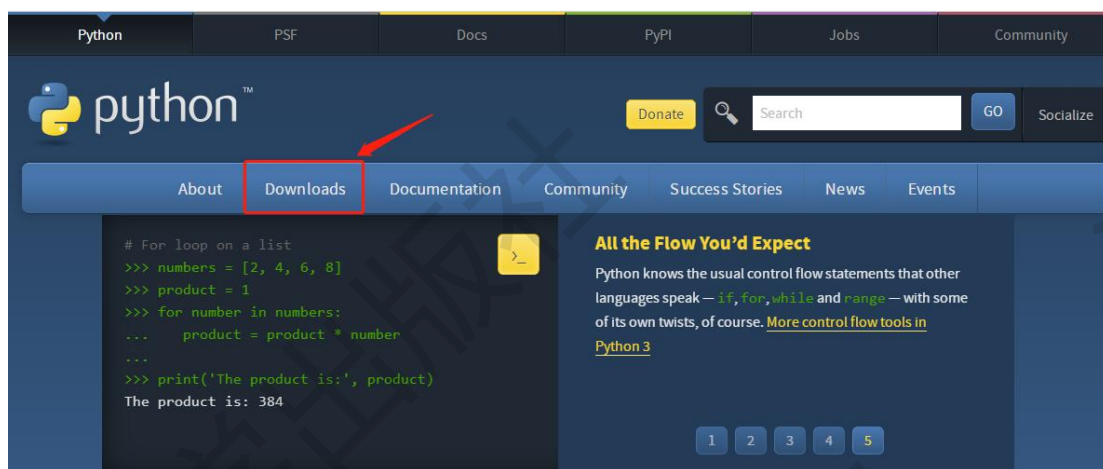


图 1-1 Python 官网首页

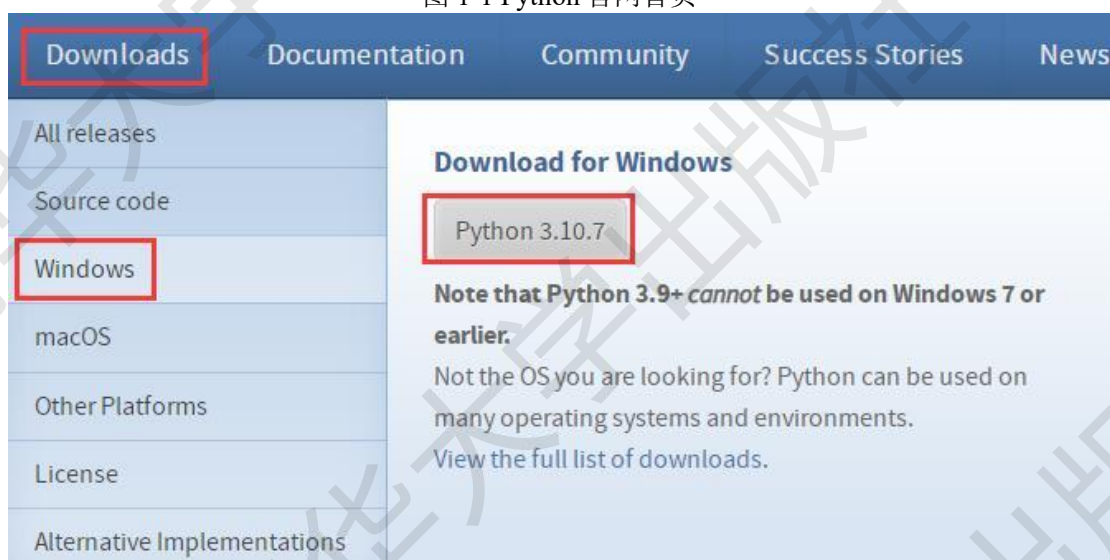


图 1-2 Download for Windows 界面

步骤 2: Python 安装。双击下载的文件 python-3.10.7-amd64.exe, 如图 1-3 所示。选择自定义安装模式 Customize installation。



图 1-3 Python 安装向导界面

图 1-4 为自定义安装的界面图，建议选择“Add Python to environment variables”选项，其他选项用户可酌情选择。

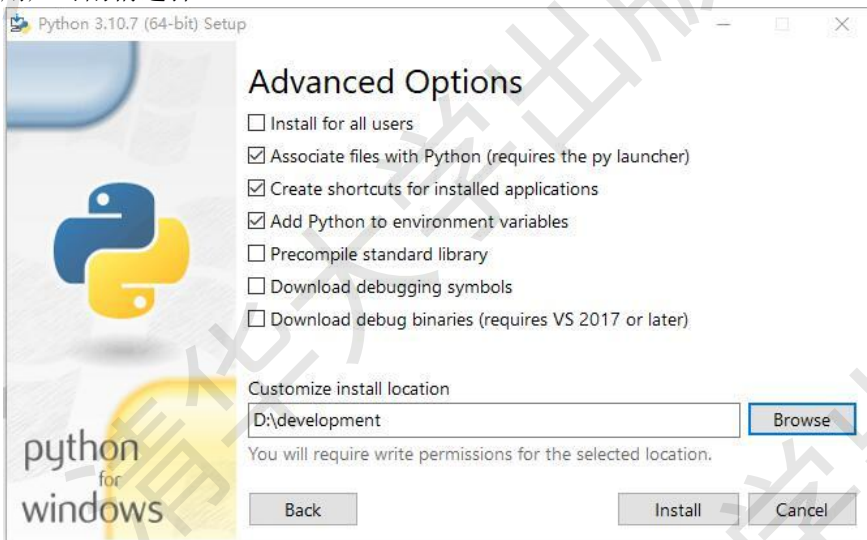


图 1-4 Python 自定义安装选项界面

步骤 3：检测 Python 是否安装成功。在开始菜单输入 cmd，打开“命令提示符”程序窗口。在当前命令提示符后输入 Python，按下 Enter 键，出现如图 1-5 所示的版本信息，则说明 Python 安装成功，同时系统进入交互式 Python 解释器中。当出现命令提示符“>>>”时可以输入 Python 命令与系统进行交互了。

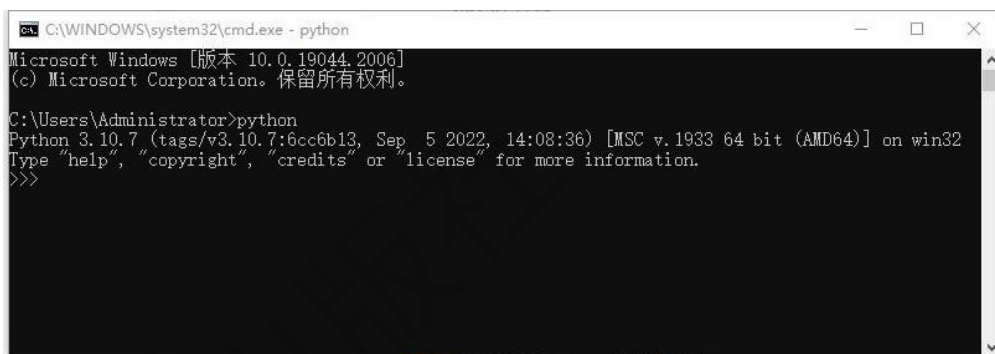


图 1-5 Windows 系统命令行窗口

1.2.4 IDLE 两种模式使用

IDLE 提供交互模式和文件执行模式两种。交互模式实时将计算结果呈现出来，与用户之间进行“一问一答”，优点在于实时性，缺点为不能够批量处理程序，导致代码运行效率低。文件执行模式需先编写代码，然后一次性执行编写好的多条代码，将最终的结果呈现给用户，该模式优点是效率高，缺点是代码错误只有在运行过程中才能体现。

下面以圆的周长和面积求解来讲解 IDLE 两种模式的使用。

输入圆的半径，分别用交互模式和文件执行模式计算并输出圆的的周长和面积。

Windows 系统下，在开始菜单中打开 IDLE，打开的 IDLE 窗口，默认进入了交互模式。

图 1-6 为默认进入的交互模块。

(1) 交互模式



图 1-6 交互模式下计算圆周长和面积

(2) 文件执行模式

在图 1-7 中，通过 File 菜单，New File 菜单项，进行创建新的文件。在其中输入表 1-1 中的代码，其中第一列以#开头的为代码行号，用于辅助代码讲解使用，输入代码时不需要将其输入。

表1-1 圆柱体体积算法设计及实现

代码	代码功能
#01 r = input('请输入圆的半径:')	## input函数从键盘获取数据，保存到变量r中
#02 r = float(r)	## 将字符串格式转换为float格式，更新变量r
#03 import math	## 导入math模块，将使用其中的圆周率
#04 length = 2*math.pi*r	## 计算圆周长，保存到变量length中
#05 area = math.pi*r*r	## 计算圆面积，保存到变量area中
#06 print(f"圆的周长={length}，圆的面积={area}")	## 调用print函数，将变量中的数据进行输出

保存文件。执行 File 菜单中的 Save 菜单项，将代码保存到文件。

执行代码。执行 Run 菜单中的 Run Module 菜单项，IDLE 将输出结果显示在输出窗口。



图 1-7 文件执行模式下计算圆周长和面积

文件执行模式，若程序有错误，执行窗口中会显示这个错误出现的位置，以及错误信息。

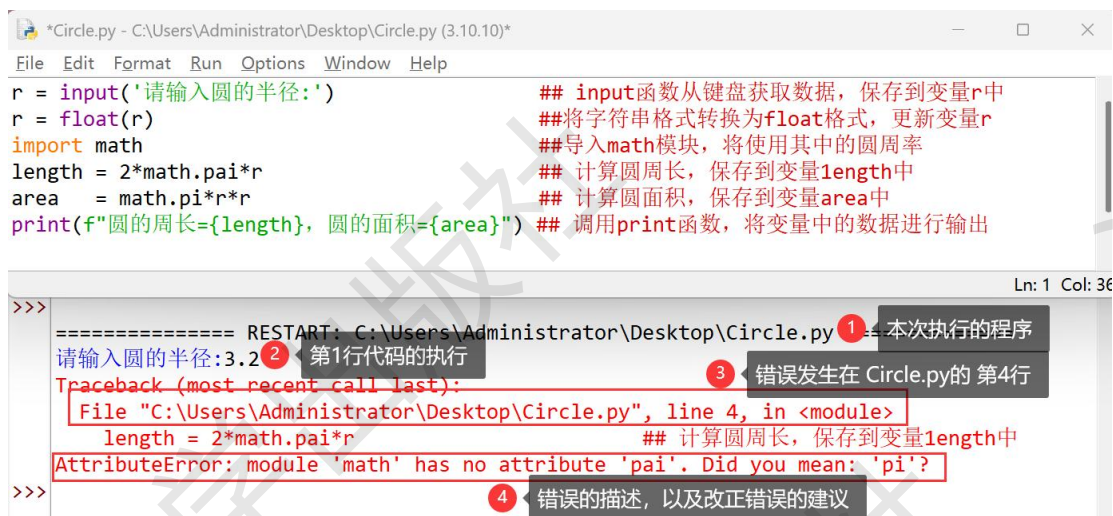


图 1-8 错误窗口

1.2.5 IDLE 交互模式快捷键

快捷键的灵活使用，有助于快速编写代码，表 1-2 为 IDLE 交互模式的快捷键。

表1-2 交互模式下快捷键

快捷键	用途	快捷键	用途
Tab	键入代码，按Tab可显示提示	Alt+P	调出上一条代码
Alt+/	自动补全	Alt+N	调出下一条代码
Ctrl+[	减少代码缩进	Ctrl+]	增加代码缩进
Alt+3	增加注释	Alt+4	去掉注释
Ctrl+Z	撤销	Ctrl+Shift+Z	重做

1.3 实践与指导

实践 1 计算圆柱体的体积

实践目的

- (1) 熟悉 IDLE 开发环境
- (2) 掌握 IDLE 两种模式的程序开发

实践要求

- (1) 参照 1.2.4 部分，完成该题。

输入圆柱体底面半径和圆柱体的高，分别用交互模式和文件执行模式计算并输出圆柱体的体积。

(2) 在表 1-3 中，先写思路，然后将思路转换为代码。并通过 IDLE 的两种模式进行验证所写代码的准确性。

表1-3 圆柱体体积算法设计及实现

算法思路	代码

实践 2 计算矩形周长和面积

实践目的

- (1) 熟悉 IDLE 开发环境
- (2) 掌握 IDLE 两种模式的程序开发

实践要求

(1) 参照 1.2.4 部分，完成该题。
输入矩形的长和宽，分别用交互模式和文件执行模式计算并输出矩形的周长和面积。

(2) 参照表 1-4 中的算法思路，将该思路转换为代码，并通过 IDLE 的两种模式进行验证所写代码的准确性。

表1-4 圆柱体体积算法设计及实现

算法思路	代码
#01 ## 从键盘上获取一个数，作为矩形的长，保存到变量a中	
#02 ## 将变量a中保存的字符串转换为float格式，更新变量a	
#03 ## 从键盘上获取一个数，作为矩形的宽，保存到变量b中	
#04 ## 将变量b中保存的字符串转换为float格式，更新变量b	
#05 ## 计算矩形的周长，将结果保存到变量length中	
#06 ## 计算矩形的面积，将结果保存到变量area中	
#07 ## 输出矩形的周长和面积	

实践 3 程序分析及完善

实践目的

- (1) 阅读代码能力
- (2) 完善程序的能力

实践要求

分析表 1-5 中代码实现的功能，补充填写每行代码的功能，代码中有一处错误，在验证程序过程中，将其改正过来，并补充缺失的代码。

注意：#04 行代码和#03 行代码有一个缩进，即#04 行代码的 n 字母与#03 行代码字母 u 在同一列（第 5 列）。#03 行代码的末尾有个英文冒号。

表1-5 程序分析及完善

代码	代码功能
#01 number = input('请输入一个数:')	## input函数从键盘获取数据，保存到变量number中
#02 number = eval(number)	## 将number字符串界定符去掉后保存到number中
#03 if number<0:	## _____
#04 number = -num	## _____
#05 print(_____)	## 输出结果

错误的代码行号是_____，IDLE 执行窗口中，这个错误的提示信息是_____，分析代码，产生这个错误的原因是_____，修改后的代码_____。

#05 行，空白处应填写的代码是_____。
程序实现的功能是_____。

实践 4 编写程序输出 HelloWorld

实践目的

- (1) 熟练掌握 IDLE 文件执行模式进行程序的开发
- (2) 掌握简单的 print 语句的使用

实践要求

(1) 编写程序，输出 HelloWorld 两个单词，两个单词各占一行。运行结果为：
Hello
World

(2) 在表 1-6 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表1-6 HelloWorld输出

算法思路	代码
------	----

实践 5 编写程序输出*号组成的三角形

实践目的

- (1) 熟练掌握 IDLE 文件执行模式进行程序的开发
- (2) 掌握简单的 print 语句的使用

实践要求

(1) 编写程序，输出一个直角三角形，三角形由*号组成，第一行包含 1 个*号，第 2 行包含 3 个*号，最后一行包含 9 个*号，构成三角形的总行数为 5。运行结果为：

```
*
***
*****
*****
*****
```

(2) 在表 1-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表1-7 打印输出三角形

算法思路	代码

实践 6 编写程序输出一个字符串

实践目的

- (1) 熟练掌握 IDLE 两种模式进行程序的开发
- (2) 了解字符串界定符的使用
- (3) 掌握简单的 print 语句的使用

实践要求

(1) 编写一个 Python 语言程序，输出 He said:"I'm a boy."，运行结果为：He said:"I'm a boy."

提示：字符串的界定符包括：一对单引号、一对双引号、一对三单引号、一对三双引

号，使用时需要成对交叉使用，且不可有歧义性。

(2) 在表 1-8 中，先写思路，然后将思路转换为代码。并通过 IDLE 的两种交互模式分别进行验证所写代码的准确性。

表1-8 打印输出一个字符串

算法思路	代码

第2章 Python 基础

2.1 学习目标与要求

- (1) 掌握 input 函数的用法
- (2) 掌握 print 函数的用法
- (3) 掌握 eval 函数的用法
- (4) 了解 Python 的基本数据类型
- (5) 掌握常见的内置函数
- (6) 了解常见的库

2.2 知识要点

本实验主要涉及 Python 内置函数的使用，旨在教授 Python 基础知识、顺序结构程序的设计、常见第三方库及内置的函数的使用，进一步提高 IDLE 两种运行方式的使用。

2.2.1 基本数据类型

Python 中有 2 个基本数据类型和 4 个复合数据类型，共 6 个标准的数据类型，主要为：Number（数值）、String（字符串）、List（列表）、Tuple（元组）、Dictionary（字典）、Set（集合）、NoneType（空类型）。其中 Number、String、Tuple 为不可变数据类型，也可称为只读数据类型；List、Dictionary、Set 为可变数据类型，也可称为可读写数据类型；NoneType 是一种特殊的数据类型，用于表示对象的缺失或空值，当一个变量未被赋予任何值时，其默认值会被设置为 None，通常用于表示无返回值的函数、未初始化的变量或空对象。

(1) Number（数值）类型。

包括四种子类型：整数、布尔型、浮点数和复数。

Python 中的整数 int，对应为长整数，其内置函数为 int()。

Python 中的 bool 类型为 bool，其值为 False 或 True，所有数据类型都可转换为 bool 型，其内置函数为 bool()。

Python 中的实数 float，其内置函数为 float()。

Python 提供复数数据类型，虚部的单位用 j 表示，如：3+4j，其内置函数为 complex()。

(2) String（字符串）类型。

用字符串界定符括起来的数据。Python 中字符串界定符包括：一对单引号、一对双引号、一对三单引号、一对三双引号。

字符串内部可有转义字符，以“\”开头的字符表示转义字符，如：“\n”表示换行、“\t”表示制表符，也可以在字符串界定符前用字符“r”来表示字符串内部的“\”符号不表示转义。字符串可以用“+”运算符进行拼接，用“*”运算符进行重复。值得注意的是：字符串是不可变数据类型。

字符串是有序的只读序列，可通过“[]”对字符串进行切片，选中其中部分的字符，如：

```
s = "1234567890"
s[0:6:2] #结果为 135
```

切片操作同样适用于元组和列表，它们都是有序序列。

常见处理字符串的内置函数：

- len()函数：返回字符串中的字符个数。
- max()函数：返回字符串中的最大字符。
- min()函数：返回字符串中的最小字符。

常见字符串对象处理的函数：

- endswith(s1:str):bool: 如果字符串是以子串 s1 结尾则返回 True
 - startswith(s1:str):bool: 如果字符串是以子串 s1 开始则返回 True
 - find(s1,[start,[end]]):int: 返回 s1 在这个字符串的最低下标，如果字符串中不存在 s1 则返回 -1，可以指定搜索区域[start,end]
 - count(substring):int: 返回子串在字符串中出现的无覆盖次数
 - index(sub[, start[, end]]):int: 返回字符串在另一个字符串指定返回内首次出现的位置
 - split([sep,[maxsplit]]:list: 以 sep 为分隔符，拆分字符串成一个列表，默认分隔符为空白符，maxsplit 为拆分次数，默认为 -1，表示无限拆分
 - lower():str: 返回复制字符串并将所有字母转换为小写的
 - upper():str: 返回复制字符串并将所有字母转换为大写的
 - replace(old,new,[count]):str: 返回一个新的字符串，用一个新字符串替换旧字符串所有出现的地方，count 指定替换次数，省略 count 表示全部替换
 - strip([chars]):str: 返回去掉两端 chars 中字符的字符串，默认去掉空白字符
 - isalpha():bool: 如果字符串中的字符是字母且至少有一个字符则返回 True
 - isdigit():bool: 如果字符串中只含有数字字符则返回 True
 - islower():bool: 如果字符串中的所有字符全是小写且至少有一个字符则返回 True
 - isupper():bool: 如果字符串中所有字符全是大写且至少有一个字符则返回 True
 - zfill(width):str: 返回一个给定宽度的字符串，右对齐，左边不够补 0
 - format(items):str: 格式化一个字符串
- (3) 列表、元组、字典、集合数据类型
后续章节会详细介绍。

2.2.2 内置函数

本小节只讲解即将用到的部分内置函数，后续章节部分会讲解余下的常见内置函数。

(1) 输入输出

1) input

input 函数用于输入信息，接收键盘输入，并返回一个字符串，语法格式为：

```
input(prompt=None, /)
```

注意：

prompt 是提示字符串，可以省略

input 返回的是一个字符串，其他数据类型需要进一步处理。

常见的多个对象输入案例：

案例 1：以英文逗号分隔

```
a, b, c = eval(input("请输入三个数:")) ## 输入 1, 2, 3 最终变量 a,b,c 分别得到 1,2,3
```

案例 2：以空格分隔

```
a, b, c = map(int, input("请输入三个数:").split()) ## 输入 1 2 3 最终变量 a,b,c 分别得到 1,2,3
```

比较案例 1、2，可得出案例 1 较 2 更简洁，案例 1 使用的较多。

2) print

print 函数用于输出信息，语法格式为：

```
print( [obj1,...] [,sep=' '] [,end='\n'] [,file=sys.stdout] )
```

注意:

所有参数均可省略, 无参数时输出一个空行。

同时输出多个对象时, 多个对象之间用 `sep` 参数指定的分隔符分隔。

`sep` 参数为指定分隔符, 默认值为 1 个空格。

`end` 参数为指定输完所有对象后, 输出的字符, 默认值为 1 个回车符。

`file` 参数为指定输出的位置, 默认为 `sys.stdout`, 该值代表显示器。

3) eval

`eval` 函数用来执行一个字符串表达式, 并返回表达式的值。字符串表达式可以包含变量、函数调用、运算符和其他 Python 语法元素。语法格式为:

`eval(expression)`

`eval()` 函数将字符串 `expression` 解析为 Python 表达式, 并在当前环境中执行它。图 2-1 为 `eval` 函数使用的示例。

```
>>> eval("3*2+1") #字符串表达式, 去除字符串界定符后, 剩下3*2+1, 这个表达式经计算后为7
7
>>> eval(3*2+1) #表达式非字符串, 没有字符串界定符, 报错
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    eval(3*2+1) #表达式非字符串, 没有字符串界定符, 报错
TypeError: eval() arg 1 must be a string, bytes or code object
>>> eval("'hello,mike.'") #两层字符串界定符, 去除最外层的一对后, 还留下一对, 经计算后为字符串
'hello,mike.'
>>> a, b = 2, 3
>>> eval('2*(a+b)') #去除字符串界定符后, 剩下 2*(a+b), 在当前执行空间内变量a,b均有值, 计算后得到10
10
```

图 2-1 eval 函数使用示例

`eval` 函数通常会结合 `input` 函数, 来去除 `input` 返回的字符串界定符作用, 使其转换为普通数据类型或者元组, 达到完成变量赋值的目的。

(2) 数学处理函数

1) sum

对数值型对象进行求和, 通常用于序列对象。

2) max/min

对数值型对象进行求最大/小值, 通常用于序列对象; 也适用于字符串, 来求字符串中的码值最大/小的字符。

3) abs

对单个数值求绝对值; 也可用于复数, 来求复数的模。

4) round

对单个数值求四舍五入。

(3) 数据类型

这类函数包括: `int()`、`float()`、`complex()`、`str()`、`bool()`、`tuple()`、`list()`、`dict()`、`set()` 等, 在使用时, 需要明确被转换的对象能否被转换为目标类型, 如果不能, 则会抛出异常。如: `s = "12.1ac2"`, 将其转换为 `int` 数据 `n = int(s)`, 会报错: “invalid literal for int() with base 10: '12.1ac2'”。

(4) 序列

1) len

返回序列中元素的个数。

2) sorted

返回排序后的序列, 原序列不变。

3) reversed

返回逆序后的序列，原序列不变。

4) range

语法格式为：range([start,] stop[,step])

返回一个 range 对象，该对象为一个可遍历的序列，该序列从[start stop)，以 step 为补偿产生的序列。start 省略表示用默认值 0，step 省略表示默认步长为 1。注意：range 对象为不可见对象，可通过 next()函数按照索引逐个增大方式进行访问。

5) map

语法格式为：

map(func, *iterables)

map 函数实现将 func 函数对 iterables 中的每个对象进行映射。返回一个 map 对象，该对象为一个不可见、可遍历的序列。序列中的每个元素均为被 func 函数作用后的对象。如：s="12345" 然后执行 map(int, s) 后，map 函数返回一个含有 5 个整数的可迭代对象，5 个整数分别为 1、2、3、4、5。

(5) 对象

1) id

id(obj)函数返回对象占用内存的地址。地址相同的对象可认为是同一个对象。

2) type

type(obj)函数返回对象的数据类型。

3) isinstance

isinstance(obj, class_or_tuple)函数返回 obj 对象的类型是否在 class_or_tuple 里面，如果在返回 True，否则返回 False。

2.2.3 常见的第三方库

Python 开发生态较为完备，第三方库可满足各个行业应用的开发。内置函数所在库的名称为 builtins，该模块是 Python 的一个特殊模块，除了包含内置函数，还包括异常和类型等，在 Python 启动时自动加载到全局命名空间中。第三方库在使用时，需由程序员通过 import 关键字进行导入方可使用。表 2-1 为常用的第三方库。

表2-1 常见第三方库

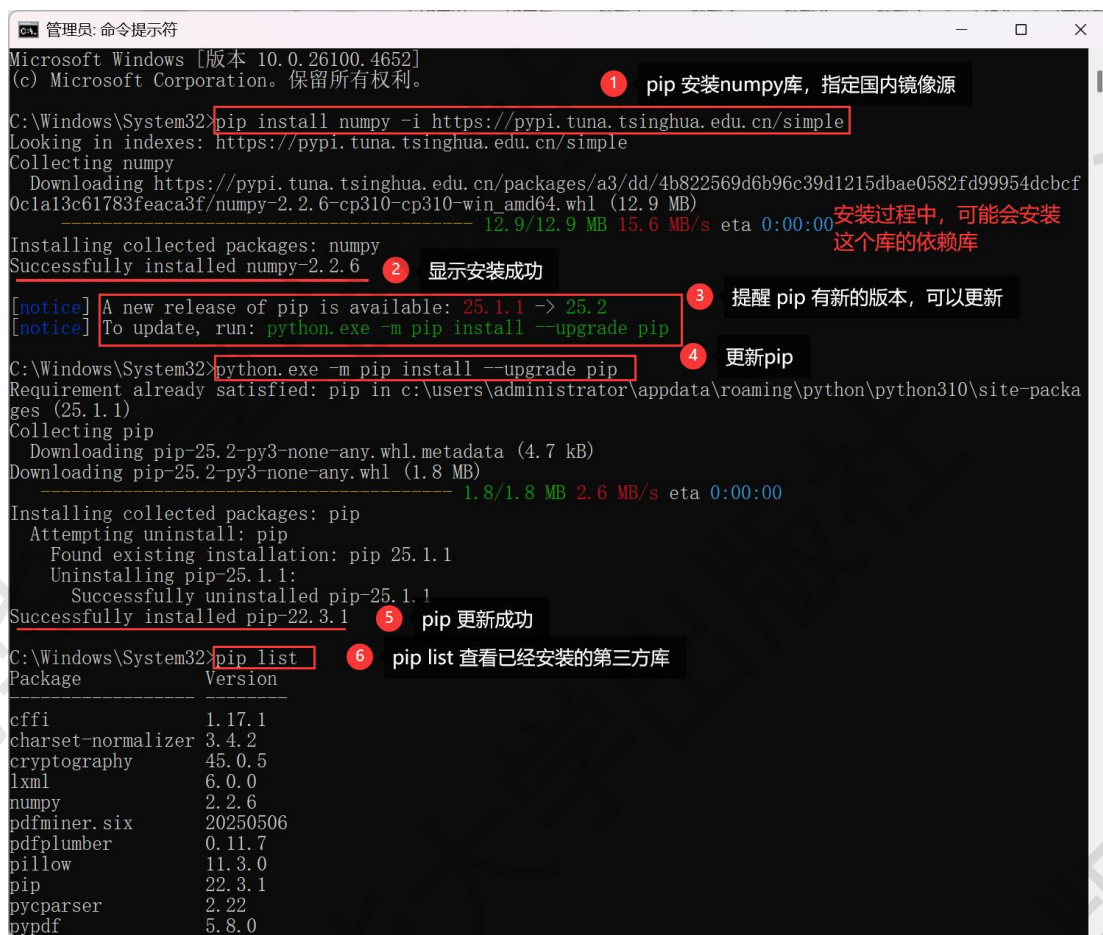
库名	用途	库名	用途
math	数学中常用的常量、函数	time	时间
random	用于产生随机并运用随机数	numpy	数值计算
turtle	海龟绘图	pandas	数据分析
os	操作系统交互	matplotlib	数据可视化（图表）
jieba	中文分词		

Python 在安装时，会自动安装一些常用的库，用户使用时不需要额外安装，这些库称为标准库，包括：os、sys、datetime、math、re、json、random、turtle 等。未安装的库，需要用户自行安装，安装方法：

打开命令提示符 cmd 窗口，在其中输入：pip install 库名 [-i 镜像源的 url]；指定国内镜像源会提高安装的速度。图 2-2 为 pip 安装 i 三方库，选用国内清华大学的镜像源。

查看已安装的第三方库的命令：pip list

卸载已安装的第三方库的命令：`pip uninstall 库名`



```
管理员: 命令提示符
Microsoft Windows [版本 10.0.26100.4652]
(c) Microsoft Corporation. 保留所有权利。

C:\Windows\System32>pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
Looking in indexes: https://pypi.tuna.tsinghua.edu.cn/simple
Collecting numpy
  Downloading https://pypi.tuna.tsinghua.edu.cn/packages/a3/dd/4b822569d6b96c39d1215dbae0582fd99954dcbcf0c1a13c61783feaca3f/numpy-2.2.6-cp310-cp310-win_amd64.whl (12.9 MB)
    12.9/12.9 MB 15.6 MB/s eta 0:00:00
Installing collected packages: numpy
Successfully installed numpy-2.2.6

[notice] A new release of pip is available: 25.1.1 -> 25.2
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Windows\System32>python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\administrator\appdata\roaming\python\python310\site-packages (25.1.1)
Collecting pip
  Downloading pip-25.2-py3-none-any.whl.metadata (4.7 kB)
  Downloading pip-25.2-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 2.6 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.1.1
    Uninstalling pip-25.1.1:
      Successfully uninstalled pip-25.1.1
Successfully installed pip-22.3.1

C:\Windows\System32>pip list
Package            Version
-----
cffi                1.17.1
charset-normalizer  3.4.2
cryptography        45.0.5
lxml                6.0.0
numpy               2.2.6
pdfminer.six        20250506
pdfplumber          0.11.7
pillow              11.3.0
pip                 22.3.1
pyparser            2.22
pypdf               5.8.0
```

1 pip 安装numpy库, 指定国内镜像源

2 显示安装成功

3 提醒 pip 有新的版本, 可以更新

4 更新pip

5 pip 更新成功

6 pip list 查看已经安装的第三方库

安装过程中, 可能会安装这个库的依赖库

图 2-2 pip 安装第三方库

2.2.4 编码规范

(1) 标识符命名规则

- 标识符由字母、数字、中文字符和下划线组成
- 第一个字符不能是数字
- 大小写敏感。

(2) 保留字

保留字也称关键字, 指被程序语言内部定义并保留使用的标识符。指代特定的含义, 不能用于用户的标识符。Python 中的保留字有:

and/or	import/ from	False/True/None	if/elif/else
while/for	break/continue	in/not/is	try/except/ finally/raise/ assert
def/return	class	pass	as
del	global	lambda	nonlocal/with/ yield

(3) 书写规范

- 缩进来表示代码块或者代码的层次
- 缩进相同的一组语句构成代码组, 如 `if...elif...else`、`while...else` 等
- 续行可用反斜杠 “\” 来表示, 避免一行过长

- 多条语句写在一行，可用“;”进行分隔

(4) 注释

注释不仅可以帮助他人阅读代码，还有助于厘清代码的思路，提高编码能力。Python中有两种常见的注释：

块注释：以一对三单引号或一对三双引号括起来的多行注释。实质上是定义了一个字符串，只是该字符串没有和变量关联起来，导致后续代码中无法对其进行使用。

行注释：以“#”开头，同一行的后续字符。行注释通常用于注释某行代码的作用、某个变量、常量的解释。从书写美观角度，也可以用多个连续的“#”开发来美化注释。

2.2.5 运算符

(1) 运算符。表 2-2 为常用的运算符，其中包括指数/乘方运算符、算术运算符、位、比较、身份、成员、逻辑等运算符。

表2-2 常见运算符

运算符	描述	运算符	描述
**	指数（级别最高）	& ^	位操作：与、异或、或
~ + -	按位翻转、正号、负号	is not is	身份运算符
* / % //	乘、除、模、整除	in not in	成员运算符
+ -	加、减	< <= > >= == !=	小于、小于等于、大于、大于等于、等于、不等于
>> <<	位操作：右移、左移	not and or	逻辑：非、与、或

在表达式中，通常会同时出现多个运算符，运算符的计算顺序是先计算括号内的，然后按运算符的优先级高低进行，先进行优先级高的运算，最后优先级相同的运算符从左到右依次计算。在书写表达式时，不能明确运算符优先级的，可用括号来提升，以避免混淆。

(2) 表达式

表达式是用运算符将变量连接起来进行运算的式子。

2.3 实践与指导

实践 1 常量变量

实践目的

- (1) 熟悉 IDLE 开发环境
- (2) 掌握常见的运算符使用
- (3) 掌握内置函数的使用
- (4) 了解 Python 对于小整数和短小的字符所执行的缓存机制。缓存机制是指将这些对象进行缓存，不会为相同的对象分配多个内存空间。

小整数：范围在-5~256 之间。

短小字符串：长度通常不超过 20，由纯 ASCII 字母、数字和下划线组成的字符串。

实践要求

- (1) IDLE 交互模式完成。

(2) 逐条输入，在表 2-3 中，记录 IDLE 输出的结果，并对结果进行分析。

表2-3 常量变量及运算符

行号	代码	IDLE输出的结果	结果分析
#01	3+4, 3+4.0, type(3+4), type(3+4.0)		结果的数据类型是： 3+4和3+4.0结果区别是：
#02	13/4, 13.0/4, 13%4, 13.0%4, 13//4, 13.0//4		
#03	-13%4, 13%-4, -13//4, 13//-4		
#04	-13.2//4, -13.2%4		
#05	2*3 > 4, 3<1+2		
#06	a = b = 12.34	-	-
#07	a is b		
#08	a=3; b=3; c=257	-	-
#09	id(a), id(b), id(3), id(c), id(257)		
#10	abs(-3), abs(-3-4j)		
#11	digits = "1234567890"		
#12	digits = "1234567890" max(digits), min(digits), len(digits)		
#13	sum(digits)		
#14	m = map(int, digits)	-	-
#15	sum(m)		
#16	print("my", "name", "is", "jact")		
#17	print("my", "name", "is", "jact", sep="_")		
#18	a, b, c = eval(input()) 输入： 3, 4, 5 a*b+c		

实践 2 计算一元二次方程的根

实践目的

- (1) 熟悉 IDLE 开发环境
- (2) 掌握+、-、*、/、**运算符的使用
- (3) 掌握 input、eval、print 函数的使用。

实践要求

(1) IDLE 文件执行模式完成。

(2) 输入一元二次方程的系数，打印输出该方程的两个实根，假定输入的系数能够求解出实根，即满足 $\Delta \geq 0$ 。

输入样例为：1,2,1

输出样例为：x1=-1, x2=-1

(3) 根据表 2-1 的算法思路，试着将其转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表2-4 一元二次方程根的计算

算法思路	代码
#01 ## 定义三个变量a, b, c 用于接收input的输入，input输入得到的字符串，交由eval函数处理，并讲处理结果分别赋值给变量a, b, c	
#02 ##通过求根公式 $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ ，分别计算出2个根，分别赋值给变量x1、x2	
#03 ## 打印输出x1、x2的值，输出格式要符合样例格式	

实践 3 程序分析及完善

实践目的

- (1) 阅读代码能力
- (2) 完善程序的能力
- (3) turtle 库的使用

实践要求

分析表 2-5 中代码实现的功能，补充填写每行代码的功能，代码中有两处错误，在推演程序过程中，将其改正过来，并补充缺失的代码。然后用 IDLE 验证自己的推演是否正确。

注意：#04 行代码和#03 行代码有一个缩进，即#04 行代码的 n 字母与#03 行代码字母 u 在同一列（第 5 列）。#03 行代码的末尾有个英文冒号。

表2-5 程序分析及完善

代码	代码功能
#01 import turtle	## 导入海龟库
#02 a,b = eval(input("请输入矩形的长度和宽度:"))	##
#03 turtle.penup()	## 抬笔
#04 turtle.setposition(-a/2, -b/2)	## 设置海龟初始位置
#05 _____	## 落笔

#06	turtle.fd(a)	## 前进
#07	turtle.left(90)	## 调整行进方向 逆时针旋转90°
#08	turtle.fd(b)	
#09	turtle.left(_____)	
#10	turtle.fd(b)	
#11	turtle.left(_____)	
#12	turtle.fd(a)	

代码逻辑错误有两处，错误 1 的行号是_____，修改后的代码_____。

错误 2 的行号是_____，修改后的代码_____。

#05 行，空白处应填写的代码是_____。

#09 行，空白处应填写的代码是_____。

#11 行，空白处应填写的代码是_____。

程序实现的功能是_____。

实践 4 四则运算

实践目的

- (1) 熟练掌握 IDLE 文件执行模式进行程序的开发
- (2) 掌握简单的 print 语句的使用
- (3) 掌握算术运算符的使用

实践要求

(1) 输入两个不为 0 的整数，计算两个数的和、差、积、商、整除、取余的值，并输出计算的结果。

输入：-37, 5

输出：-32 -42 -185 -7.4 -8 3

(2) 在表 2-6 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表2-6 四则运算

算法思路	代码

实践 5 数位数字和

实践目的

- (1) 熟练掌握 IDLE 文件执行模式进行程序的开发
- (2) 掌握简单的 `print` 语句的使用
- (3) 掌握算术运算符的使用

实践要求

(1) 输入 1 个三位正整数，计算该数各个数位上数字之和，并将计算的结果输出。

输入：351

输出：9

(2) 在表 2-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表2-7 数位数字之和

算法思路	代码

实践 6 年龄计算

实践目的

- (1) 熟练掌握 IDLE 文件执行模式进行程序的开发
- (2) 掌握简单的 `print` 语句的使用
- (3) 掌握算术运算符的使用

实践要求

(1) 输入 1 个人出生后的天数，计算这个人的年龄和月份，并将计算的结果输出。为简化计算，假设每个月 30 天，每年 360 天；未满 360 天，不算 1 年；未满 30 天，不算 1 月。

输入：5982

输出：16 年 7 月

(2) 在表 2-8 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表2-8 年龄计算

算法思路	代码

第 3 章 选择结构

3.1 学习目标与要求

- (1) 深入理解分支结构的原理
- (2) 掌握分支结构程序的编写
- (3) 掌握分支结构程序执行流程
- (4) 掌握代码缩进对执行流程的影响

3.2 知识要点

选择结构主要有三种形式：单分支结构、双分支结构和多分支结构。

3.2.1 单分支结构

单分支结构是选择结构中最简单的形式，它只包含一个条件判断语句，即 if 语句。if 语句根据条件的真假来决定是否执行特定的代码块。

图 3-1(a)为单分支结构执行流程图，语法格式为：

```
if 表达式:  
    语句块  
#后续语句
```

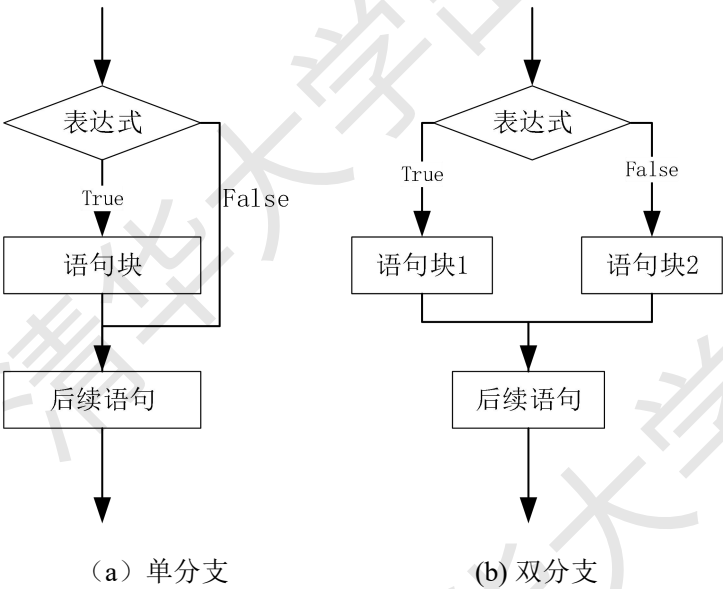


图 3-1 单双分支结构流程图

3.2.2 双分支结构

双分支结构是选择结构中扩展了一种情况处理的形式，它除了包含一个条件判断语句（if 语句），还包含一个 else 语句。通过双分支结构，程序可以根据条件的真假执行不同的代码块。

在双分支结构中，if 语句后面的条件表达式会被求值，如果条件表达式的值为真

(True) 则执行 if 代码块中的语句；如果条件表达式的值为假 (False)，则执行 else 代码块中的语句。

图 3-1(b)为双分支结构执行流程图，语法格式为：

```
if 表达式:
    语句块 1
else:
    语句块 2
#后续语句
```

3.2.3 多分支结构

多分支结构是选择结构中最灵活的形式，它是通过使用多个条件判断语句 (if-elif-else) 来执行不同的代码块。在多分支结构中，程序会逐个检查条件，并执行与第一个满足条件相关联的代码块。

在多分支结构中，if 语句后面的条件表达式会被求值，如果条件表达式的值为真 (True)，则执行 if 代码块中的语句；如果条件表达式的值为假 (False)，则继续检查下一个 elif 语句的条件，如果条件为真，则执行对应的代码块；如果所有的条件都为假，则执行 else 代码块中的语句。

图 3-2 为多分支结构执行流程图，语法格式为：

```
if 表达式 1:
    语句块 1
elif 表达式 2:
    语句块 2
...
elif 表达式 n:
    语句块 n
else:
    语句块 n+1
#后续语句
```

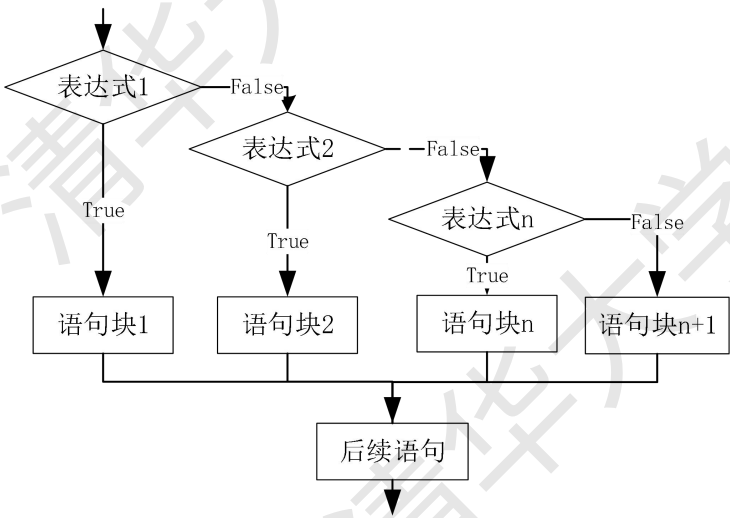


图 3-2 多双分支结构流程图

3.3 实践与指导

实践 1 发放购物券

实践目的

- (1) 掌握 if 单分支结构执行流程
- (2) 掌握 if 多分支结构执行流程
- (3) 掌握条件表达式的计算

实践要求

(1) 阅读下面程序，先推演代码，填写表 3-1，然后将代码输入到文件中，使用文件模式运行代码，将运行的结果和推演结果进行对比，如不一致，试着分析推演错误之处。

(2) 为了提升消费能力，激活市场，政府决定给民众发放购物券，购物券以电子形式发放，发放的基数为 800 元/人，根据个人可支配收入金额，给予不同的发放系数，可支配收入金额越低发放系数越大。具体标准为：

可支配收入金额不超过 500 元，系数为 1.8

可支配收入金额不超过 800 元，系数为 1.7

可支配收入金额不超过 1000 元，系数为 1.6

可支配收入金额不超过 1200 元，系数为 1.5

可支配收入金额不超过 1500 元，系数为 1.4

可支配收入金额不超过 1800 元，系数为 1.3

可支配收入金额不超过 2000 元，系数为 1.2

可支配收入金额不超过 2200 元，系数为 1.1

可支配收入金额超过 2200 元，系数为 1.0

输入可支配收入金额，输出发放购物券的金额。程序代码如下：

表3-1 购物券发放

行号	代码	代码分析
#01	<code>disposable_income = \</code> <code>eval(input('请输入可支配收入的金额:'))</code>	可支配收入的金额，以输入750为案例进行分析
#02	<code>money_base = 800</code> <code>##购物券发放基数</code>	-
#03	<code>if disposable_income <= 500: ##可支配收入不超过500</code>	<code>disposable_income <= 500</code> 的值为____
#04	<code>money = money_base * 1.8</code>	#04行代码执行不? ____ 若执行 <code>money</code> 的值为____
#05	<code>if disposable_income <= 800:</code>	<code>disposable_income <= 800</code> 的值为____
#06	<code>money = money_base * 1.7</code>	#06行代码执行不? ____ 若执行 <code>money</code> 的值为____
#07	<code>if disposable_income <= 1000:</code>	<code>disposable_income <= 1000</code> 的值为____
#08	<code>money = money_base * 1.6</code>	#06行代码执行不? ____ 若执行 <code>money</code> 的值为____
#09	<code>if disposable_income <= 1200:</code>	<code>disposable_income <= 1200</code> 的值为____
#10	<code>money = money_base * 1.5</code>	#06行代码执行不? ____ 若执行 <code>money</code> 的值为____
#11	<code>if disposable_income <= 1500:</code>	<code>disposable_income <= 1500</code> 的值为____
#12	<code>money = money_base * 1.4</code>	#06行代码执行不? ____

		若执行 money 的值为_____
#13	if disposable_income<=1800:	disposable_income<=1800 的值为___
#14	money = money_base*1.3	#06 行代码执行不? _____ 若执行 money 的值为_____
#15	if disposable_income<=2000:	disposable_income<=2000 的值为___
#16	money = money_base*1.2	#06 行代码执行不? _____ 若执行 money 的值为_____
#17	if disposable_income<=2200:	disposable_income<=2200 的值为___
#18	money = money_base*1.1	#06 行代码执行不? _____ 若执行 money 的值为_____
#19	if disposable_income>2200:	disposable_income>2200 的值为___
#20	money = money_base*1.0	#06 行代码执行不? _____ 若执行 money 的值为_____
#21	print(f'发放购物券金额:{money:.2f} 元')	输出: _____

将程序录入到计算机，用 IDLE 的文件执行模式运行，结果为_____

试着比较推演的结果与计算机执行的结果是否一致，若不一致，请分析下推演出错的地方在哪里？

本程序能否实现购物券正确发放？如果不能，请修改上述程序，并以正确的程序来修订原程序，并记录在表 3-1 中。

实践 2 一日游收费

实践目的

- (1) 掌握 if...else ... 双分支结构执行流程
- (2) 掌握条件表达式的计算

实践要求

(1) 市区“一日游”收费标准为：5 人以内（含 5 人）按散客标准，每人 160 元；超过 5 人，按团体标准，每人 140 元。编写程序输入人数，输出旅游总费用。

(2) 根据表 3-2 的算法思路，试着将其转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。验证方式可参照：3 人费用为 480 元、5 人费用为 800 元、8 人费用 1120 元。运行程序三次，分别输入 3、5、8，查看程序输出的结果是否和预期值相同。

表3-2 一日游费用

	算法思路	代码
#01	## 定义人数变量person_count用于接收用户输入，并转换为数值型	
#02	## 如果人数≤5，总费用=人数×160	
#03	## 否则，总费用=人数×140	
#04	## 输出总费用	

实践 3 BMI

实践目的

- (1) 掌握 if 多分支结构执行流程
- (2) 掌握条件表达式的计算

实践要求

(1) 身体质量指数 BMI (Body Mass Index) 是国际上常用的衡量人体胖瘦程度及健康状况的标准, 其计算公式为:

$$\text{BMI} = \frac{\text{体重}}{\text{身高}^2}$$

体重单位为: kg、身高单位为: m, BMI 的单位为 千克/平方米

表 3-3 为中国的健康参考标准。

表3-3 中国BMI标准

BMI标准 (单位: Kg/m ²)	BMI分类	相关疾病发病的危险性
<18.5	偏瘦	低 (但其他疾病危险性增加)
18.5~24.0(不含)	正常	平均水平
24.0~27.0(不含)	偏胖	增加
27.0~30.0(不含)	肥胖	中度增加
≥30.0	重度肥胖	严重增加

请编写程序, 输入体重和身高, 计算出 BMI 值, 根据表 3-3, 给出 BMI 分类和相关疾病发病危险性的提示信息。

表 3-4 中的程序代码为实现该功能的部分程序, 请根据已有的程序, 将下划线部分补充完整, 另程序中有 2 处错误, 请找出并改正。将代码功能补充完整。完善并改正后的程序, 请输入到计算机中, 采用 IDLE 文件执行模式验证是否正确。

表3-4 BMI

代码	代码功能
#01 weight, height = eval(input("请输入体重[单位:kg], 身高[单位: cm], 两个数据之间用英文逗号分隔"))	-
#02 height //= 100	##转换成米
#03 BMI = weight/height*height	##计算BMI
#04 if BMI<18.5:	##BMI低于18.5的情况
#05 print("偏瘦: 低 (但其他疾病危险性增加)")	
#06 elif 18.5<=BMI<24.0:	## _____
#07 print("_____")	
#08 elif 24<=BMI<27.0:	## _____
#09 print("偏胖: 增加")	

#10	elif 27<=BMI<30.0:	## _____
#11	print("肥胖：中度增加")	
#12	_____:	## _____
#13	print("重度肥胖：严重增加")	

代码逻辑错误有两处，错误 1 的行号是_____，修改后的代码_____。

错误 2 的行号是_____，修改后的代码_____。

#07 行，空白处应填写的代码是_____。

#12 行，空白处应填写的代码是_____。

实践 4 游戏人物血量

实践目的

- (1) 熟练单分支结构
- (2) 掌握条件表达式的运算

实践要求

(1) 某款简单游戏的人物在行进路上，捡到金币，血量加 10；捡到银币，血量加 5；捡到铜币，血量加 2；遇到怪兽，血量减少 50；初始化血量为 1 到 100 之间随机数，当血量低于 0 时，游戏结束。请设计程序，完成单次血量变化。

(2) 在表 3-5 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表3-5 游戏人物血量

算法思路	代码
## 导入_____模块	
## 定义血量变量blood，并初始化	
## 定义变量npc表示{金币、银币、铜币、怪兽} ## 约定 输入的数为1表示金币，2为银币，3为铜币，4为怪兽；npc初值为1到4之间的随机数	
## 打印输出血量和本次遇到的npc	
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	

实践 5 归一化

实践目的

- (1) 掌握双分支结构执行的流程
- (2) 掌握条件表达式的运算

实践要求

- (1) 在强化学习中，动作空间可能被限制在 $[-1, 1]$ 。若输入的值大于 1，则值取 1；若输入的值小于 -1，则值取 -1，请编写程序实现这个功能。
- (2) 在表 3-6 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表3-6 归一化

算法思路	代码
## _____	
## _____	
## _____	
## _____	
## _____	

实践 6 天数

实践目的

- (1) 掌握多分支结构执行的流程
- (2) 掌握条件表达式的运算

实践要求

- (1) 输入年份和月份，输出该月的天数。需要考虑当前年份是否为闰年。闰年可分为世纪闰年和非世纪闰年。世纪闰年指的是年份能被 400 整除；非世纪闰年指的是年份能被 4 整除且不能被 100 整除。
- 算法提示：可优先考虑月份，如果输入的月份是 1、3、5、7、8、10、12，则天数为 31；若月份是 4、6、9、11，则天数为 30；若月份为 2，需要验证是否为闰年，如是，则天数为 29，否则为 28。
- (2) 在表 3-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表3-7 天数

算法思路	代码
## _____	
## _____	

## _____	
## _____	
## _____	
## _____	
## _____	
## _____	

第 4 章 循环结构

4.1 学习目标与要求

- (1) 深入理解循环结构的原理
- (2) 深入理解 while 循环结构程序的执行流程
- (3) 深入理解 for 循环结构程序的执行流程
- (4) 掌握 while 循环结构
- (5) 掌握 for 循环结构
- (6) 掌握 range 函数的用法

4.2 知识要点

循环结构是对同一个代码块进行多次重复执行，根据指定的条件来确定是否继续执行循环体内的代码，从而实现重复执行的功能。常见的循环结构有两种：while 循环和 for 循环。

4.2.1 while 循环

由一个循环条件表达式和一个循环体组成，语法格式：

```
while 条件表达式:  
    语句块
```

图 4-1 为 while 循环执行的流程，从中可看出，循环体在条件表达式为 True 时，一直执行下去，直到为 False 才跳出循环。

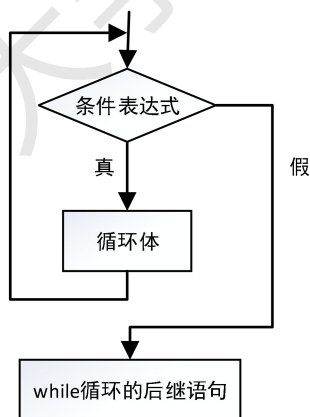


图 4-1 while 循环

4.2.2 扩展 while 循环

while 循环后面可跟随一个 else 子句。else 子句在 while 循环条件正常结束时执行。如果 while 循环是通过 break 语句提前退出的，那么 else 子句将不会被执行。语法格式为：

```
while 条件表达式:  
    语句块 1  
else:  
    语句块 2
```

4.2.3 for 循环

for 循环是一种计数循环，通过指定循环的次数来执行循环体内的代码。通常在以下情况使用 for 循环：

- (1) 循环次数已知
- (2) 需要遍历处理序列结构或可迭代对象中的每个元素。

语法格式为：

for 循环变量 in 遍历结构：

语句块 1

[else:

语句块 2]

for 循环的执行过程：从遍历结构中逐一提取元素（**按索引逐个增大的方式**），放入循环变量，直至遍历结构中的元素取完。

可选的 else 部分执行方式和 while 语句类似。自然结束执行循环体，则执行 else 后的语句块 2；若因在语句块 1 中执行了 break 语句而结束循环时，不会执行 else 后的语句块 2。

注意：

- (1) for 循环的循环次数等于序列结构或可迭代对象的元素个数。
- (2) 若需要按指定次数循环，可以使用 range() 函数产生的 range 对象来配合控制循环。

4.2.4 break 和 continue

break 语句是用于提前终止本层循环。当某个条件满足时，我们可以使用 break 语句立即结束整个循环，不再执行循环体内剩余的代码，直接跳出循环。

continue 语句用于提前结束本次循环，本次循环后续的语句不执行，而转去执行下一次循环。

4.2.5 循环嵌套

循环的嵌套也是一种循环结构，它允许在一个循环内部包含另一个循环。通过嵌套循环，可以在外层循环的每次迭代中执行内层循环，从而实现更加复杂的循环逻辑和嵌套迭代操作。

嵌套循环的工作原理是，外层循环的每次迭代都会触发内层循环的完整执行。在内层循环执行完毕后，外层循环再进行下一次迭代，直到外层循环结束。这种层层嵌套的循环结构可以实现多维度的数据遍历、复杂的条件判断和逻辑处理。

4.3 实践与指导

实践 1 阅读推演

实践目的

- (1) 掌握 while...else... 循环结构执行流程
- (2) 掌握 break、continue 循环控制语句
- (3) 掌握条件表达式的计算

实践要求

- (1) 阅读下面程序，先推演代码，填写表 4-1，然后将代码输入到文件中，使用文件

模式运行代码，将运行的结果和推演结果进行对比，如不一致，试着分析推演错误之处。

表4-1 阅读推演

行号	代码	代码分析
#01	x = 10	-
#02	while x>0:	x = _____ 表达式 x>0 的值_____ 执行#0__到#0__行代码
#03	if x == 8:	表达式 x==8的值_____ 执行#04行____ (√×)
#04	break	下一条语句行号#_____
#05	if x%3 != 1:	表达式 x%3!=1的值_____ 执行#06行____ (√×)
#06	continue	下一条语句行号#_____
#07	x -= 2	更新变量x的值为_____ 下一条语句行号#_____
#08	else:	#08行执行的条件是 变量x的值为_____ #02行代码中的条件不成立时 变量x的值为_____, 所以#08-09行_____ 会执行
#09	print(f'x={x}')	
#10	print(f'x={x}')	输出: _____

实践 2 字符类别统计

实践目的

- (1) 掌握循环结构执行流程
- (2) 掌握条件表达式的计算

实践要求

- (1) 输入一段英文，统计其中出现字母数量、数字数量和其他字符的数量，并输出。
- (2) 根据表 4-2 的算法思路，试着将其转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

提示：判断一个字符变量 a 中存储是否为数字可用 '0'<=a<'9'来判断。

表4-2 字符统计

算法思路	代码
#01 ## 定义变量s用于接收用户输入的一段英文	
#02 ## 定义alpha_count、digit_count、other_count变量 ## 表示字母数量、数字数量、其他字符数量，并初始化为0	
#03 ## 循环遍历变量s中的每个字符，放到变量ch中	
#04 ## 如果ch的值为字母，则alpha_count加1	
#05 ## 否则如果ch的值为数字，则digit_count加1	
#06 ## 否则other_count加1	
#07 ## 输出三个变量的值	

实践 3 偶数拆分为两个素数之和

实践目的

- (1) 掌握 if 单分支结构执行流程
- (2) 掌握循环结构程序的编写
- (3) 掌握条件表达式的计算

实践要求

(1) 哥德巴赫猜想：“任一不小于 4 的偶数，均可表示为两个素数之和”。编写程序实现输入一个不小于 4 的偶数，将其拆分为两个素数之和。

表 4-3 中的程序代码为实现该功能的部分程序，请根据已有的程序，将下划线部分补充完整，另程序中有 2 处错误，请找出并改正。将代码功能补充完整。完善并改正后的程序，请输入到计算机中，采用 IDLE 文件执行模式验证是否正确。

推演程序分析程序时，可假设输入的值为 8 或者 10，可以有效控制循环次数。

表4-3 哥德巴赫猜想——偶数

代码	代码功能
#01 num = int(input())	## _____
#02 for i in range(2, num):	## _____
#03 for m in range(2, i%2):	## _____
#04 if i%m == 0:	## _____
#05 break	## _____
#06 else:	## _____
#07 j = _____	## _____
#08 for n in range(2, j//2):	## _____
#09 if j%m== 0:	## _____
#10 _____	## _____
#11 else:	## _____
#12 print(f"{num} = _____")	## _____
#13 break	## _____

代码逻辑错误有两处，错误 1 的行号是_____，修改后的代码_____。
错误 2 的行号是_____，修改后的代码_____。

- #07 行，空白处应填写的代码是_____。
- #10 行，空白处应填写的代码是_____。
- #12 行，空白处应填写的代码是_____。

实践 4 四位升序数

实践目的

- (1) 熟练掌握循环执行流程
- (2) 掌握条件表达式的运算

实践要求

- (1) 输出 4 位数中的所有升序数，所谓升序数是指个位数比十位数大，十位数比百位数大，百位数比千位数大。如：2347、3789 等
- (2) 在表 4-4 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表4-4 四位升序数

算法思路	代码
## 采用 for...in 结构，遍历所有4位数放到变量n中	
## 如果千位<百位<十位<个位，则输出这个数	

针对实践 4：①尝试用 while 循环来实现 ②4 位数的范围能否优化到 6790？

实践 5 Fibonacci 数列

实践目的

- (1) 灵活运用循环结构处理问题
- (2) 掌握 while 和 for 循环

实践要求

- (1) 斐波那契数列又称兔子繁衍问题。初始的两项为 1，从第 3 项开始，值为前面两项之和。1，1，2，3，5，8，13，21...，编写程序，实现输入项数 n（n 的值≥3），输出前 n 项的值。
- (2) 在表 4-5 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表4-5 Fibonacci数列

算法思路	代码
## _____	

## _____	
## _____	
## _____	
## _____	
## _____	

实践 6 e^x 计算

实践目的

- (1) 灵活运用循环结构处理问题
- (2) 掌握 while 和 for 循环

实践要求

- (1) e^x 可用下面的公式进行求解：

$$e^x = \sum_{i=0}^n \frac{x^i}{i!} = \frac{x^0}{0!} + \frac{x^1}{1!} + \frac{x^2}{2!} + \dots + \frac{x^n}{n!}$$

当 x 的值输入 1 时，计算出来的就是自然对数 e 的值，请编写程序计算 e^x，并验证 x=1 时输出的值是否正确。当累加项 $\frac{x^i}{i!} < 10^{-10}$ 时，结束累加。

- (2) 在表 4-6 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表4-6 e^x

算法思路	代码
## 定义变量x，接收用户的输入	
## 定义变量cx，用于存储e ^x 值 ## 定义变量i，用于做循环变量 ## 定义变量fi，用于存储 i! ## 定义变量xi，用于存储x ⁱ 值 ## 同时完成各个变量的初始化	
## _____	
## _____	
## _____	
## _____	

## _____	
## _____	

第5章 列表与元组

5.1 学习目标与要求

- (1) 掌握列表常见的操作
- (2) 深入理解 for...in 遍历列表元素的原理
- (3) 掌握元组常见的操作

5.2 知识要点

列表是一种有序的集合，可以包含任意数量和类型的元素。元组与列表非常相似，区别在于列表可读可写，元组为只读、不能修改。

列表(list)是 Python 中最重要的内置对象之一。形式上，列表的所有元素放在一对方括号[]中，相邻元素之间使用逗号分隔。同一个列表中的元素的数据类型可以各不相同，可以同时包含整数、实数、字符串等基本类型的元素，也可以包含列表、元组、字典、集合、函数以及其他任意对象。

5.2.1 列表

(1) 创建

格式 1：用内置函数创建空列表，并将其与列表变量关联起来。格式为：列表名 = list()

格式 2：将已有的列表与列表变量关联起来。格式为：列表名 = 列表

(2) 删除

del 列表名

(3) 清空

列表名.clear()

(4) 列表元素访问

列表名[index]

index 的正向索引值为[0 len(列表名))，逆向索引值为[-len(列表名) -1]

(5) 表 5-1 为列表的基本操作。

表5-1 列表的常用操作

方法/函数	描述	方法/函数	描述
append(obj)	将obj追加到列表尾部	clear()	清空列表
extend(obj:list)	将列表obj中的所有元素追加到列表尾部 实现将列表obj拼接到列表尾部	index(obj)	返回obj第一次出现的index值
insert(index, obj)	将obj插入到列表的index位置，index及之后的元素依次后移一个位置	count(obj)	返回obj在列表中出现的数量
remove(obj)	删除列表中第一个obj元素	reversed()	原地逆序，返回None
pop([index])	删除index位置处的元素，并返回该元素；index缺省值为-1	sort(key=None, reverse=False)	原地排序，key指定排序规则，reverse默认升序，值True表示降序

（6）运算符

拼接操作

+: 非原地操作，速度慢。

+=: 原地操作，速度快；与 `append()` 方法一样高效。

重复操作

*: 非原地操作，速度慢。

*=: 原地操作，速度快。

成员测试

`in`: `obj in L` 测试 `obj` 对象是否为列表 `L` 的元素

`not in`: `obj not in L` 测试 `obj` 对象是否不是列表 `L` 的元素

比较运算符

等于 (`==`)、不等于 (`!=`)、大于 (`>`)、小于 (`<`)、大于等于 (`>=`) 和小于等于 (`<=`) 等，在使用时对参与比较的两个列表对应位置的元素进行比较。

（7）列表推导式

格式为：

```
[expression for expr1 in sequence1 if condition1
 for expr2 in sequence2 if condition2
 for expr3 in sequence3 if condition3
 ...
 for exprN in sequenceN if conditionN]
```

`expression` 是由 `expr1`、`expr2`、`expr3`、...、`exprN` 构成的表达式

（8）列表切片

`[start:stop:step]`

`start`: 开始的索引

`stop`: 结束的索引

`step`: 步长，正数表示正向，负数表示逆向

值得注意的是：列表每个元素的索引值都有正向索引值和逆向两个索引值。从 `start` 能否到 `stop` 要结合 `step` 的方向来看。

切片除了适合列表之外，还适合元组和字符串，它们共同点都是有序序列。

（9）对列表操作的内置函数

`len()`、`max()`、`min()`、`sum()`、`zip()`、`map()`、`sorted()`、`reversed()` 等函数，其中 `max`、`min`、`sum` 函数要求列表对象要能比较大小或者求和，否则会报错。

5.2.2 元组

（1）创建

格式 1：用内置函数创建空元组，并将其与变量关联起来。格式为：元组名 = `tuple()`

格式 2：将已有的元组与元组变量关联起来。格式为：元组名 = 元组

值得注意的是：空元组的表示为 `(,)`

（2）删除

`del` 元组名

（3）元组元素访问

元组名[index]

`index` 的正向索引值为 `[0 len(列表名))`，逆向索引值为 `[-len(列表名) -1]`

（5）元组与列表的异同点

相同点：有序序列，支持双向索引，支持切片

不同点：元组为只读型数据，不可改变，所以没有 `append`、`extend`、`insert`、`pop`、`remove` 等改变元组的方法。元组的访问速度要远大于列表，但不如列表灵活。

5.3 实践与指导

实践 1 阅读推演

实践目的

- (1) 掌握列表元素遍历方法
- (2) 掌握列表子元素遍历方法
- (3) 掌握 `for...in` 结构的使用方法

实践要求

(1) 阅读下面程序，先给下面的代码加上注释，填写表 5-2；然后对代码进行推演，填写表 5-3；最后将代码输入到文件中，使用文件模式运行代码，将运行的结果和推演结果进行对比，如不一致，试着分析推演错误之处。

表5-2 阅读推演

行号	代码	代码分析
#01	<code>ls = [[3, 4, 5], [8, 2, 9], [1, 6, 7]]</code>	## _____
#02	<code>result = 0</code>	## _____
#03	<code>for tmp in ls:</code>	## _____
#04	<code> for num in tmp:</code>	## _____
#05	<code> result += num</code>	## _____
#06	<code>print(f'result={result}')</code>	## _____

将推演过程中的各个变量值的变化记录在表 5-3 中，值没有变化的，可不填写。

表5-3 变量记录

执行的代码	ls	result	tmp	num
#01	[[3, 4, 5], [8, 2, 9], [1, 6, 7]]	-	-	-
#02		0	-	-
#03			[3, 4, 5]	-
#04				3
#05		3		
#04				4
#05		7		
#04				5
#05		12		5

#03	## 定义列表变量ls2，初值为空，用于保存不重复的ls1中的数	
#04	## 遍历ls1中的每个数，保存到num	
#05	## 如果num不在ls2中，则添加到ls2中	
#06	## 对ls2进行排序	
#07	## 遍历ls2中的每个元素，保存到num	
#08	## 输出num和num在ls1中出现的个数	

实践 3 生成不重复的随机数

实践目的

- (1) 掌握列表的使用方法
- (2) 掌握列表 count 函数的使用
- (3) 掌握 in/not in 的使用

实践要求

(1) 表 5-5 中程序代码实现的功能是：随机生成 40 个 40 到 100 之间的数，生成的 40 个数字没有重复。代码中三处错误，请将其修改；代码中有三处缺失，请在相应的下划线处填写正确的代码。填写代码功能，完善并改正后的程序，请输入到计算机中，采用 IDLE 文件执行模式验证是否正确。

表5-5 生成不重复的随机数

代码	代码功能
#01 import random	## _____
#02 ls = list()	## _____
#03 while _____:	## _____
#04 num = random.randint(_____)	## _____
#05 if num in ls:	## _____
#06 ls.extend(num)	## _____
#07 if len(ls)=40:	## _____
#08 _____	## _____
#09 print(ls)	## _____

代码逻辑错误有三处，错误 1 的行号是_____，修改后的代码_____。
错误 2 的行号是_____，修改后的代码_____。
错误 3 的行号是_____，修改后的代码_____。

#03 行，空白处应填写的代码是_____。
#04 行，空白处应填写的代码是_____。

#08 行，空白处应填写的代码是_____。

实践 4 评委评分

实践目的

- (1) 掌握列表、元组的使用方法
- (2) 掌握内置函数对列表、元组的作用

实践要求

(1) 在一次歌舞比赛大会上，邀请了 8 个评委对 5 名选手进行评分，选手的得分为去掉一个最高分、一个最低分后，取余下 6 个评分的平均分。请编写程序完成选手分数的录入，并计算出各个选手的均分，将选手均分打印输出。

选手得分输入样式如下（每行为一个选手的得分）：

9.30, 9.00, 9.00, 9.00, 10.00, 9.30, 9.60, 9.40
9.90, 9.60, 9.40, 9.00, 9.70, 9.20, 9.40, 9.20
9.30, 10.00, 9.90, 9.50, 10.00, 10.00, 9.80, 9.40
9.00, 9.10, 9.80, 9.30, 9.30, 9.10, 9.90, 9.60
9.90, 9.00, 9.10, 9.80, 10.00, 9.10, 9.50, 9.20

输出格式：

9.27
9.42
9.77
9.37
9.43

(2) 在表 5-6 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表5-6 评委评分

算法思路	代码

提示：输入时去除字符串界定符后就是个元组，将元组直接 append 到列表上，然后计算该元组的分数总和，减去最高分、减去最低分后求均分，保存到一个均分的列表中。最后将均分列表中的均分逐个打印输出。

实践 5 成绩数据统计

实践目的

- (1) 灵活运用循环结构处理问题
- (2) 掌握 while 和 for 循环

实践要求

(1) 对一组百分制成绩，按照优秀、良好、中等、及格、不及格进行统计分析，要求输出各个等地的人数，占比。一组成绩采用英文逗号分隔，一次性输入。成绩输入样式如下：

65, 92, 78, 70, 94, 48, 73, 81, 93, 97, 95, 50, 46, 71, 98, 63, 85, 100, 61, 56, 73, 98, 68, 76, 71, 55, 91, 87, 53, 46, 49, 58, 62, 76, 98, 48, 63, 78, 70, 80

结果输出格式：

优秀：10 人，占比：25.00%

良好：4 人，占比：10.00%

中等：10 人，占比：25.00%

及格：6 人，占比：15.00%

不及格：10 人，占比：25.00%

(2) 优秀的分数范围：[90 100]，良好的分数范围：[80 89]，中等的分数范围：[70 79]，及格的分数范围：[60 69]，不及格的分数范围：[0 59]。

(3) 在表 5-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表5-7 成绩统计

算法思路	代码
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	

实践 6 选择排序

实践目的

- (1) 列表元素遍历
- (2) 深入理解选择排序算法的原理
- (3) 掌握选择排序算法的原理

实践要求

- (1) 在不使用现有排序功能，编写代码，实现对列表进行降序排序。

选择排序算法：找 n 个数中最大值并记录该数的下标，然后通过下标实现和第 1 个元素进行交换；接着找剩下（去除第 1 个数） $n-1$ 个数中最大值并记录该数的下标，然后通过下标实现和第 2 个元素进行交换；直到最后剩下一个数为止。总共找最大值或者交换的次数应该是 $n-1$ 次。例如：对“3、9、7、2、5”五个数采用选择排序的过程为：

步骤 1：[3, 9, 7, 2, 5]，最大值 9 的下标为 1，将 0、1 号位置的元素进行交换，列表更新为：[9, 3, 7, 2, 5]

步骤 2：[9, 3, 7, 2, 5]中对 1~4 号元素进行找最大值，将 1、2 号位置的元素进行交换。列表更新为：[9, 7, 3, 2, 5]

步骤 3：[9, 7, 3, 2, 5]中对 2~4 号元素进行找最大值，将 2、4 号位置的元素进行交换。列表更新为：[9, 7, 5, 2, 3]

步骤 4：[9, 7, 5, 2, 3] 中对 3~4 号元素进行找最大值，将 3、4 号位置的元素进行交换。列表更新为：[9, 7, 5, 3, 2]

至此，已完成 5 个元素的排序

- (2) 在表 5-8 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表5-8 选择排序

算法思路	代码
## 定义变量ls，存储待排序的数	
## 定义变量_____，存储未排序元素中，最大值的下标	
## 循环 数组元素个数-1次（举例中的步骤数量）	

第 6 章 字典与集合

6.1 学习目标与要求

- (1) 掌握字典常见的操作
- (2) 掌握集合常见的操作

6.2 知识要点

6.2.1 字典

字典 (Dictionary) 在 Python 中是一种无序、可变、可迭代的数据结构，它用于存储键值对 (key-value pairs)。字典中的每个元素都由一个键和一个值组成，键用于唯一标识元素，而值是与键关联的实际数据。每个键值对用冒号 ":" 分割，每个键值对之间用逗号 "," 分割，整个字典包括在花括号 "{}" 中。

键可以是数字、字符串或元组等不可变的类型；值可以是任何类型的数据，包括数字、字符串、列表、甚至可以是另一个字典。

字典具有无序性、键唯一性、值多样性、可变性、动态性、高效性、可扩展性等特点。

(1) 创建

格式 1：用内置函数创建空字典，并将其与变量关联起来。格式为：字典名 = dict()

格式 2：将已有的字典与变量关联起来。格式为：字典名 = 字典

(2) 删除

del 字典名

(3) 清空

字典名.clear()

(4) 字典元素访问

读取键 key 对应的值：字典名[key] 或者 字典名.get(key)

改写键 key 对应的值：字典名[key] = new_value

(5) 字典元素删除

删除指定字典中键 key 的元素：del 字典名[key] 或者 字典名.pop(key)

(6) 表 6-1 为字典的基本操作

表6-1 字典的常用操作

方法/函数	描述	方法/函数	描述
keys()	返回字典所有key构成的可迭代对象	values()	返回字典所有value构成的可迭代对象
items()	返回字典所有(key,value)二元组构成的可迭代对象	get(key, defaultvalue)	获取key对应的值，若key不存在，则以defaultvalue为返回值
update(dict1)	用dict1去更新字典，对不重复的key，直接加入；重复的key，用dict1的值去更新	pop(key)	若key存在，返回对应value，同时删除该元素；否则抛异常
clear()	清空所有元素	popitem()	随机弹出一个(key,value)二元组，同时删除该元素

(7) 字典的遍历

字典的操作默认是对 key 进行操作。

通过 key 进行遍历：for key in d: 或者写成 for key in d.keys():
通过 key、value 进行遍历：for key,value in d.items():

6.2.2 集合

集合存储不重复的元素，和字典一样，集合使用一对花括号 {} 作为定界符，元素之间使用逗号进行分隔。集合中的元素只能是不可变类型的数据，不能包含像列表、字典等可变类型的数据。

(1) 创建

格式 1：用内置函数创建空集合，并将其与变量关联起来。格式为：

集合名 = set([迭代对象])

格式 2：将已有的非空集合与变量关联起来。格式为：集合名 = 集合

(2) 删除

del 集合名

(3) 清空

集合名.clear()

(4) 集合元素访问

可用 for...in 结构实现对集合元素遍历

(5) 集合元素删除

删除指定元素有两种方法。

方法 1：remove(obj)，若 obj 不在集合中，则抛异常

方法 2：discard(obj)，若 obj 不在集合中，不抛异常

(6) 表 6-2 为集合的基本操作

表6-2 集合的常用操作

方法/函数	描述	方法/函数	描述
intersection()	返回多个集合的交集	union()	返回多个集合的并集
difference()	返回集合间的差集	add()	添加一个元素
pop()	随机删除一个元素，函数返回值为该元素	update()	更新集合

(7) 运算符

|：并集

-：差集

&：交集

^：对称差，两个集合中相异元素，等价于 并集-交集

>、<：真子集的判断

>=、<=：子集判断

==、!=：是否相等

(8) 内置函数

len()、max()、min()、sum()、sorted()、reversed()等函数也适用于集合。

列表、元组、字典、集合为 Python 常用的复合数据类型，在程序开发中，使用非常频繁，表 6-3 为四种复合数据类型的对比，通过对比，在后续编写程序中，可选择相应的数据类型来进行问题的求解。

表6-3 复合数据类型对比

对比项	列表	元组	字典	集合
类型名	list	tuple	dict	set
定界符	[]	()	{}	{}
是否可变	是	否	是	是
是否有序	是	是	否	否
是否支持下标	索引下标	索引下标	键下标	否
元素形式要求	无	无	键:值	必须可哈希
元素值的要求	无	无	“键”必须可哈希	必须可哈希
元素是否可重复	是	是	键不可重复	否
元素查找速度	非常慢	很慢	非常快	非常快
增加、删除元素的速度	尾部操作快 其他位置慢	不允许	快	快

可哈希（hashable）：指的是一个对象是否具有哈希值，且该哈希值在对象的生命周期内保持不变。哈希值是一个固定长度的整数，用于快速比较对象的相等性。

6.3 实践与指导

实践 1 字典集合常见操作

实践目的

- （1）掌握字典、集合的基本操作
- （2）能够灵活运用字典、集合进行处理问题
- （3）掌握内置函数对字典、集合的使用

实践要求

- （1）IDLE 交互模式完成。
- （2）逐条输入，在表 6-4 中，记录 IDLE 输出的结果，并对结果进行分析。

表6-4 字典集合常见操作

行号	代码	推演结果	IDLE输出结果
#01	d = {1:1, 2:8, 3:28, 4:64}	-	-
#02	d[3]		
#03	d[3] = 3**3	-	-
#04	d[3]		

#05	d1 = {4:16, 5:5**3}		
#06	d.update(d1)	-	-
#07	d		
#08	d.get(6, 0)	-	-
#09	sum(d.keys())		
#10	s1, s2 = {1,2,3,4}, {4,3,6,7}		
#11	s1 s2		
#12	s1		
#13	s1-s2		
#14	s1&s2	-	-
#15	s2		
#16	s1.intersection(s2)		
#17	s1		
#18	s1.intersection_update(s2)		
#19	s1		
#20	s2		

实践 2 天数

实践目的

(1) 掌握字典的使用方法

实践要求

(1) 输入年份和月份，输出该月的天数。需要考虑当前年份是否为闰年。

算法提示：将天数和月份对应存储到字典中，2 月份天数的值为 28，以输入的月份为键到字典中将其值取出。如果是 2 月份，再判断是否为闰年，为闰年将取出的天数+1。

(2) 根据表 6-5 的算法思路，试着将其转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表6-5 天数

	算法思路	代码
#01	## 定义字典变量____，用于存储月份与天数关系。以月份为键，以天数为值。如：1:31, 2:28	

#02	## 定义变量____、____，分别接收用户输入的年份和月份	
#03	## 从字典中获取该月对应的天数	
#04	## 如果是2月，判断是否为闰年，闰年的话，将天数+1	
#05	## 输出天数	

实践 3 生成不重复的随机数

实践目的

- (1) 掌握集合的使用方法
- (2) 掌握 eval 函数的使用
- (3) 掌握 in/not in 的使用

实践要求

表 6-6 中程序代码实现的功能是：随机生成 n 个 40 到 100 之间的数，生成的 n 个数字没有重复。代码中两处错误，请将其修改；代码中有两处缺失，请在相应的下划线处填写正确的代码。填写代码功能，完善并改正后的程序，请输入到计算机中，采用 IDLE 文件执行模式验证是否正确。

表6-6 生成不重复的随机数

代码	代码功能
#01 import random	## _____
#02 nums_set = set()	
#03 total_count = input()	## _____
#03 while True:	## _____
#04 num = random.randint(40, 100)	## _____
#05 if num in nums_set:	
#06 nums_set. _____	## _____
#07 if len(nums_set) >= total_count::	
#08 _____	## _____
#09 print(nums_set)	## _____

代码逻辑错误有两处，错误 1 的行号是_____，修改后的代码_____。

错误 2 的行号是_____，修改后的代码_____。

#06 行，空白处应填写的代码是_____。

#08 行，空白处应填写的代码是_____。

实践 4 公司合并

实践目的

- (1) 掌握字典的常用方法
- (2) 掌握多字典合并、更新元素的方法

实践要求

(1) 两个公司进行合并重组，涉及到公司部门合并与更新，原则是：针对不重叠部门，直接保留；针对重叠部门，则将两公司重叠部门的值进行相加。

A 公司的财务部有 3 人，福利部有 1 人，人事部有 2 人；

B 公司的财务部有 1 人，人事部有 1 人，后勤部有 2 人；

新成立的 C 公司的财务部人数为 4 人，后勤部人数为 2 人，人事部人数为 3 人，福利部人数为 1 人。

假定用户输入的数据格式为：

```
{'财务部':3, '福利部':1, '人事部':2}
```

```
{'财务部':1, '人事部':1, '后勤':2}
```

IDLE 输出格式为：

```
{'后勤': 2, '人事部': 3, '财务部': 4, '福利部': 1}
```

(2) 在表 6-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表6-7 公司重组

算法思路	代码

提示：新公司的部门，可通过原来两个公司部门去重后得到；新公司部门的人数可通过原公司部门的人数相加得到，避免公司没有这个部门而产生异常，可通过 get 函数进行操作。

实践 5 歌手海选

实践目的

- (1) 灵活运用循环结构处理问题
- (2) 掌握 while 和 for 循环

实践要求

(1) 歌唱比赛进行海选活动，歌手分两组，按序号进行投票，第一组歌手编号为 1、2、3、4、5，第二组歌手编号为 6、7、8、9、10，投票编号为 4、7、9、1、2、2、6、2、2、1、6、9、7、4、5、5、7、9、5、5、4。编程找出获得选票的全部歌手序号、第一组获得选票的歌手序号、第二组获得选票歌手的序号。输出的三个数据，均需要按照歌手编号升序排列。

输出结果为：
1, 2, 4, 5, 6, 7, 9
1, 2, 4, 5
6, 7, 9

(2) 在表 6-8 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表6-8 歌手海选

算法思路	代码
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	

实践 6 字符统计

实践目的

- (1) 掌握字典的常用方法
- (2) 掌握集合的使用方法
- (3) 掌握列表的使用方法
- (4) 掌握复合数据类型解决实际的方法

实践要求

(1) 输入一段英文，统计其中每个字符出现的数量，并按照字符数量降序输出。

输入：Technology continues to shape our world in profound ways, connecting people across distances and improving access to information.

输出：
18 <这个是空格>
o 15
n 13
...

(2) 在表 6-9 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

第7章 函数和模块

7.1 学习目标与要求

- (1) 掌握函数的定义与调用
- (2) 掌握函数参数的传递方式
- (3) 深入理解函数参数值传递与地址传递方式
- (4) 深入理解函数的递归调用
- (5) 掌握常见的内置函数使用方法

7.2 知识要点

7.2.1 函数定义

函数是组织好的，可重复使用的，用来实现单一或相关联功能的代码段。函数能提高应用的模块性和代码的复用性，有效降低代码之间的耦合性，同时降低开发成本、提高开发效率。函数可接受输入参数，并返回函数运算后的结果，从而实现模块化和封装性编程的目的。

函数定义的语法格式：

```
def function_name([parameters]) [->datatype]:  
    function_body  
    [return expression]
```

7.2.2 函数参数

函数的参数主要分为形参和实参两种。形参是定义函数时的参数变量，实参是调用函数时使用的值（通常取自变量）。调用函数时可使用的参数类型有：位置参数、关键字参数、默认值参数、可变长参数等。

- (1) 位置参数：通过位置对应关系，实现实参值到形参之间的传递。
- (2) 关键字参数：调用函数时，按照参数名称进行传递值的形式。
- (3) 默认值参数：调用时，没有为设置默认值的形参传递值，那么函数执行中将使用默认值。

7.2.3 变量作用域

变量作用域分为全局变量和局部变量。

- (1) 局部变量：定义在函数内部，其作用域仅限于函数内部，在函数外不能使用。
- (2) 全局变量：全局变量的作用范围从其定义开始，到本文件结束。可以定在函数体外部，也可以在函数体内部通过 `global` 关键字来声明该变量为全局变量。

7.2.4 匿名函数

语句少而简单的函数，可通过创建匿名函数来实现，使用 `lambda` 表达式来创建匿名函数。格式为：

```
lambda [arg1, [args,...]]:expression
```

7.2.5 递归函数

一个函数在内部直接或间接调用自己本身，这个函数就是递归函数。

7.2.6 内置函数

(1) help

帮助函数，用来查看函数或模块的详细信息。语法格式为：help(obj)

obj 可以是函数名、数据类型、模块名称

(2) divmod

计算两个数的商和余数，返回商和余数组成的二元组。

(3) pow

计算 x 的 y 次方。

(4) filter

将序列中不符合条件的元素过滤掉，返回符合条件元素组成的迭代对象，可通过 list、tuple 等函数将其转为具体的列表、元组等数据类型对象。

(5) all/any

判断迭代对象中所有元素是否全为 True/False，如果是，返回 True/False，否则返回 False/True。

(6) iter

创建一个迭代器对象。语法格式为：iter(obj)

(7) next

迭代器的下一个元素。语法格式为：next(iterable)

(8) enumerate

返回一个迭代器对象，其中每个元素是一个二元组，二元组由序列中的元素和其下标组成。

(9) zip

返回一个迭代器对象，其中每个元素是一个 N 元组，N 元组由多个迭代器对象中的元素组成。

7.2.7 模块

模块指独立的、可识别的、作为一个整体来处理的程序代码。模块中可包括变量、常量、函数、类等。模块可提高代码的可重用性，避免变量名和函数名冲突。Python 中模块是按照文件来划分的，即一个 Python 源代码文件为一个模块，模块名称为该文件名。

模块在调用执行时，通过 `__name__ == "__main__"` 语句来判断当前运行的模块是否为自身，从而决定是否执行模块中可执行的语句。

7.3 实践与指导

实践 1 函数的调用过程

实践目的

- (1) 掌握函数的定义与调用
- (2) 深入理解函数的调用过程
- (3) 掌握函数的返回值

实践要求

- (1) 下面的 max2 函数实现了比较两个数中最大值，并返回最大值。max3 函数实现

了比较三个数中最大值，并返回最大值，通过调用 `max3`，来启动 `max2` 的函数执行，进而达到了简化代码的目的。

(2) 推演下面的代码，分析代码功能，并填表 7-1。

表7-1 最大值求解过程

行号	代码	代码功能
#01	def max2(a:int, b:int) -> int:	
#02	if a>b:	
#03	ret = a	
#04	else:	
#05	ret = b	
#06	return ret	
#07	def max3(x:int, y:int, z:int) -> int:	
#08	ret = max2(x, y)	
#09	ret = max2(ret, z)	
#10	return ret	
#11	if __name__ == "__main__":	
#12	a, b, c = eval(input())	
#13	m = max3(a,b,c)	
#14	print(f'max({a},{b},{c}) = {m}')	

分析上面代码执行的流程，参照部分执行流程，填写表 7-2 中未填写完毕的执行流程，然后将代码录入到计算机并运行，查看结果是否与推演的结果一致，如不一致，请分析推演出错在哪里。假设本次执行输入的数据为：67, 85, 59

表7-2 程序执行流程

执行代码行号	变量值	执行代码行号	变量值
#11	-	#12	a = 67 b = 85 c = 59
#13	m = ?	#07	max3: x = 67 y = 85 z = 59
#08	ret = ?	#01	max2: a = 67 b = 85
#02	max2: 67>85 False	#05	max2: ret = 85
#06	return 85	#08	max3: ret = 85
#09			

实践 2 组合数

实践目的

- (1) 掌握函数编写方法
- (2) 掌握函数调用的方法
- (3) 深入理解函数返回值的用途

实践要求

(1) 组合是数学中的重要概念之一。从 n 个不同元素中每次取出 m 个不同元素，不管其顺序合成一组，称为从 n 个元素中不重复地选取 m 个元素的一个组合。所有这样的组合的种数称为组合数。

$$C_n^m = \frac{n!}{m!(n-m)!}$$

算法提示：编写一个计算阶乘的函数，通过三次调用函数，实现计算组合数。

(2) 根据表 7-3 的算法思路，试着将其转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表7-3 组合数

算法思路	代码
#01 ## 编写计算 $n!$ 的函数 <code>factorial</code> ，参数为 <code>int</code> 型，返回值为 <code>int</code> 型	
#02 ## 定义一个用于存放 $n!$ 的变量 <code>ret</code> ，初值为_____	
#03 ## 循环计算出 <code>ret</code> 的值	
#04 ## 返回 <code>ret</code>	
#05 ## 输入两个数 <code>n, m</code>	
#06 ## 分别调用 <code>factorial</code> ，计算出 $n!$ 、 $m!$ 、 $(n-m)!$ ，保存到对应变量中	
#07 ## 计算并输出组合数	

实践 3 有序序列插入数据

实践目的

- (1) 掌握函数的开发方法
- (2) 掌握函数的调用

实践要求

- (1) 下面程序代码完成在有序的列表中插入数据，插入后的列表依旧有序。
- (2) 请分析程序，填写表 7-4 中的代码功能。
- (3) 在空白处填写代码；程序中有两处错误，请改正。最后将正确的程序输入到计算机中，采用 IDLE 文件执行模式验证是否正确。

表7-4 生成不重复的随机数

代码	代码功能
#01 def insert_position(ls:list, val:int) -> int:	##在有序列表ls中，查找val插入的位置，返回位置
#02 for i in range(0, len(ls)):	##
#03 if ls[i]>=val:	##
#04 ret = i	##
#05 break	##
#06 else:	##
#07 ret = len(ls)	##
#08 return ret	##
#09 def insert_value(ls:list, val:int) -> None:	##在有序列表ls中，查找val插入的位置
#10 pos =	##
#11 if pos>=len(ls):	##
#12 ls.append(val)	##
#13 return None	##
#14	## 扩容
#15 for i in range(len(ls), pos, -1):	##
ls[i] = ls[i-1]	##
#16 ls[pos] = val	##
#17 if __name__ == "__main__":	## 启动
#18 numbers = [6, 14, 27, 33, 46, 62, 87]	
#19 number = input()	
#20 insert_value(numbers, number)	
#21 print(numbers)	

代码逻辑错误有两处，错误 1 的行号是_____，修改后的代码_____。

错误 2 的行号是_____，修改后的代码_____。

#10 行，空白处应填写的代码是_____。

#14 行，空白处应填写的代码是_____。

实践 4 素数

实践目的

- (1) 掌握函数的开发方法
- (2) 掌握函数的调用

实践要求

(1) 编写开发一个素数判断的函数，返回 bool 值。测试该函数：输出三位素数，根据输出的结果判定开发的函数是否正确

(2) 在表 7-5 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表7-5 素数

算法思路	代码

实践 5 猴子吃桃

实践目的

- (1) 掌握函数的开发方法
- (2) 掌握函数的递归调用

实践要求

(1) 猴子每天吃的桃子数是当天总桃子数的一半再加一个，第 10 天吃完，求总共多少个桃子。

算法分析：第 10 天吃完，说明第 10 天吃之前桃子数量为 1

第 9 天的桃子数量为 $2 \times (1+1) = 4$

第 8 天的桃子数量为 $2 \times (4+1) = 10$

....

用 `monkey_peach(n)` 表示第 n 天桃子的数量，其值为

$\text{monkey_peach}(n) = 2 \times (\text{monkey_peach}(n-1) + 1)$

当天数为 10 时，`monkey_peach(n)` 值为 1

试用递归方式求解出第 1 天桃子的数量。

(2) 在表 7-6 中，先写思路，然后将思路转换为代码。并写代码，启动该函数，测试

验证该函数的功能是否符合要求。

表7-6 猴子吃桃

算法思路	代码
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	
## _____	

实践 6 自幂数

实践目的

- (1) 掌握复杂程序处理方法
- (2) 掌握函数开发的方法

实践要求

(1) 自幂数 (Narcissistic number) 在数学中定义为：一个 n 位数，其每个位上的数字的 n 次幂之和等于该数本身。例如： $153 = 1^3 + 5^3 + 3^3$ ，以下为 1~10 位自幂数。

- 1 位：独身数 (0-9 均为自幂数)
- 3 位：水仙花数 (153、370、371、407)
- 4 位：四叶玫瑰数 (1634、8208、9474)
- 5 位：五角星数 (54748、92727、93084)
- 6 位：六合数 (548834)
- 7 位：北斗七星数 (1741725、4210818 等)
- 8 位：八仙数 (24678050、24678051 等)
- 9 位：九九重阳数 (146511208、472335975 等)
- 10 位：十全十美数 (4679307774)

开发不少于 3 个函数，实现自幂数。

(2) 在表 7-7~7-12 中，可先规划开发的函数，然后在后续表中完成对应函数的功能代码。下面表的行数不够，可自行增加。

表7-7 自幂数程序函数规划表

函数名	函数参数	函数功能

表7-8 函数

函数代码	代码功能

表7-9 函数

函数代码	代码功能

表7-10 函数

函数代码	代码功能

表7-11 函数

函数代码	代码功能

表7-12 函数

函数代码	代码功能

启动函数调用程序：


```

if __name__ == "__main__":
    for i in range(1, 100000000000): #循环 1~10 位数
        if narcissistic_number(i):    #如果 i 是自幂数，打印输出
            print(i)

```

第 8 章 类和对象

8.1 学习目标与要求

- (1) 理解面向对象编程的特点，类和对象的概念
- (2) 掌握类的编写方法，能够熟练编写类
- (3) 掌握对象的实例化及相关属性的访问、方法的调用，能够熟练创建和使用对象
- (4) 理解面向对象编程的继承和多态的概念
- (5) 掌握父类、子类的编写和使用方式
- (6) 能够熟练使用面向对象编程解决应用问题

8.2 知识要点

Python 是面向对象编程语言。面向对象编程 (Object Oriented Programming)，简称 OOP。面向对象编程就是通过面向对象分析和设计，建立模型 (类或对象) 并完成最终程序的过程。在面向对象编程中，编程的主体就是用类或对象构建模型，并使得他们可以相互通信，从而实现解决实际问题。面向对象语言编程的三大基本特征是封装、继承、多态。

8.2.1 类

- (1) 类的定义

类(Class)是抽象的，它定义了同一类对象有的属性和方法，是属性和方法的封装，其定义的语法格式：

```
class ClassName( superclass1 [, superclass2,...] ): ##类头
    class_body ##类体
```

class: 定义类的关键字

ClassName: 类名用于和其他标识符进行区分，其命名要符合 Python 标识符命名规则。

superclass: 超类也叫基类、父类，通过指定超类，可实现继承。

class_body: 类体，主要包括属性和方法。

- (2) 类体

一个类的三个主要组成部分：

属性 (Attributes): 就是“它有什么”，是描述对象状态的数据 (变量)。比如，“人”这个类可以有 名字、年龄、身高 等属性。

方法 (Methods): 就是“它能做什么”，是对象可以执行的操作 (函数)。方法定义在类的内部，用于描述对象的行为。比如，“人”这个类可以有 说话()、走路()、吃饭() 等方法。

初始化方法 __init__: 这是一个非常特殊且重要的方法，可以把它看作是对象的“构造函数”。当你根据类创建一个新实例时，__init__ 方法会自动被调用，用于初始化对象的初始属性。你通常在这里设置属性的初始值。

类的核心思想是【封装】和【复用】。它将数据 (属性) 和操作数据的函数 (方法) 捆绑在一起，使得代码更加模块化、组织更清晰、更容易重复使用。

```
class Dog:
    # 初始化方法 (构造函数)：在创建对象时自动调用，用于设置初始属性
    def __init__(self, name, breed):
        # 属性: self.attribute = value
```

```

self.name = name # 狗狗的名字
self.breed = breed # 狗狗的品种
self.is_sitting = False # 狗狗的状态（是否坐着）

# 方法：定义行为
def sit(self):
    self.is_sitting = True
    print(f'{self.name} is now sitting.')

def bark(self):
    print(f'{self.name} says: Woof!')

def introduce(self):
    print(f'Hi, I'm {self.name}, a {self.breed}.')
    if self.is_sitting:
        print("I'm currently sitting.")

```

8.2.2 对象

对象 (Object / Instance): 是根据类创建出来的一个具体实例，拥有类中定义的属性和方法，但每个对象的属性值可以不同。

用类定义对象，叫实例化。创建对象后，要访问实例对象的属性和方法，可以通过“.”运算符来连接对象名和属性或方法。

在 Python 中，“对象”就是根据“类”这个蓝图创建出来的一个具体实例。它包含了类中定义的数据（属性）和功能（方法），并且拥有自己独立的数据状态。

（1）实例化：根据类创建对象的过程叫做实例化。使用 `ClassName()` 语法来实例化一个对象。

根据 Dog 类创建两个对象

`my_dog = Dog("Buddy", "Golden Retriever")` # `my_dog` 是一个对象

`your_dog = Dog("Lucy", "Poodle")` # `your_dog` 是另一个对象

（2）身份唯一：每个对象在内存中都有唯一的地址（`id(object)`），就像世界上没有两片完全相同的树叶。`my_dog` 和 `your_dog` 是两个完全独立的对象。

（3）状态独立：每个对象都独立存储自己的属性值。修改 `my_dog` 的 `is_sitting` 不会影响 `your_dog` 的 `is_sitting`。

（4）行为共通：所有由同一个类创建的对象共享相同的方法（函数），但这些方法操作的是各自对象内部的数据。

8.2.3 属性和方法

在一个类中，属性是变量，它用于存储与类或对象相关的数据（描述状态）。方法是函数，它用于定义类或对象的行为（能执行什么操作）。它们共同作用，将数据和对数据的操作封装在一起，形成一个完整的、可复用的单元。

（1）属性

属性是绑定到类或对象的变量。

实例属性

定义：属于特定对象实例的属性。每个对象实例都有自己独立的一份副本，修改一个对象的实例属性不会影响其他对象。

如何定义：通常在类的方法内部（最常用的是 `__init__` 构造方法）使用 `self.attribute_name = value` 来定义和初始化。

访问：通过对象实例访问，`object_name.attribute_name`。

```
class Dog:
    def __init__(self, name): # self 代表当前创建的实例对象本身
        self.name = name    # 定义实例属性 'name'
        self.is_sitting = False # 定义实例属性 'is_sitting', 并赋予初始值

my_dog = Dog("Buddy") # 创建对象时, __init__ 被调用, 属性被初始化
your_dog = Dog("Lucy")
print(my_dog.name)    # 输出: Buddy
print(your_dog.name)  # 输出: Lucy
my_dog.is_sitting = True # 修改 my_dog 的实例属性
```

类属性

定义：属于类本身的属性。所有由这个类创建的对象实例共享同一份类属性。改变类属性，所有实例访问到的值都会改变。

如何定义：在类内部、所有方法之外直接定义。

访问：既可以通过类名访问 `Class_Name.attribute_name`，也可以通过对象实例访问 `object_name.attribute_name`。

```
class Dog:
    species = "Canis familiaris" # 类属性，所有狗狗都是这个物种

    def __init__(self, name):
        self.name = name        # 实例属性

my_dog = Dog("Buddy")
your_dog = Dog("Lucy")
# 通过类和实例访问类属性
print(Dog.species)             # 输出: Canis familiaris
print(my_dog.species)          # 输出: Canis familiaris
print(your_dog.species)        # 输出: Canis familiaris
# 修改类属性（通过类名修改）
Dog.species = "Canis lupus"
```

私有属性

定义：一种约定俗成的、表示“仅供内部使用”的属性。Python 中没有真正的私有化，而是通过命名规则来实现：以双下划线开头 `__` 的属性名，Python 解释器会对其进行名称改写，使其在类外不易被直接访问。

目的：实现封装，防止外部代码意外修改内部敏感数据，同时也可避免命名冲突。

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder # 公共属性
        self.__balance = balance            # 私有属性

    def get_balance(self): # 提供公共方法来安全地访问私有属性
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

account = BankAccount("Alice", 1000)
print(account.account_holder) # 可以正常访问
# print(account.__balance) # 报错: AttributeError: 'BankAccount' object has no attribute '__balance'
```

Python 实际上将其改名了，但仍然可以强制访问（但不建议这样做）

```
# print(account._BankAccount__balance) # 输出: 1000
print(account.get_balance()) # 正确的方式: 通过公共方法访问, 输出: 1000
```

(2) 方法

方法是定义在类中的函数，用于描述对象的行为。根据其第一个参数和功能，可分为以下几种：

实例方法

定义：这是类中最常见、最普通的方法。它的第一个参数必须是 `self`，代表调用该方法的对象实例本身。通过 `self`，实例方法可以自由地访问和修改对象的实例属性、其他实例方法以及类属性。

调用：通过对象实例调用。

类方法

定义：用于操作类属性或进行与类本身相关的工厂设计模式（例如创建类的特定实例）。它的第一个参数必须是 `cls`，代表类本身（而不是对象实例）。

如何定义：需要使用装饰器 `@classmethod` 来标记。

调用：可以通过类名直接调用，也可以通过对象实例调用。

静态方法

定义：与类相关，但不需要访问类属性或实例属性的函数。它就像一个放在类命名空间里的普通函数，不会自动传入 `self` 或 `cls`。

目的：主要用于代码组织，表明这个函数在逻辑上属于这个类。

如何定义：需要使用装饰器 `@staticmethod` 来标记。

调用：可以通过类名直接调用，也可以通过对象实例调用。

表 8-1 为属性和方法的对比，从中可看出几种不同的修饰后的不同。

表 8-1 属性和方法对比

类型	定义关键字/装饰器	首参数	主要用途	调用方式
实例属性	<code>self.attr_name</code> (在 <code>__init__</code> 中)	-	存储每个对象独有的数据	<code>obj.attr_name</code>
类属性	在类内部直接定义	-	存储所有对象共享的数据	<code>Class.attr_name</code> 或 <code>obj.attr_name</code>
实例方法	(无)	<code>self</code>	定义对象的行为，操作实例数据	<code>obj.method_name()</code>
类方法	<code>@classmethod</code>	<code>cls</code>	操作类属性或作为工厂方法	<code>Class.method_name()</code> 或 <code>obj.method_name()</code>
静态方法	<code>@staticmethod</code>	无	逻辑上属于类的工具函数	<code>Class.method_name()</code> 或 <code>obj.method_name()</code>

8.2.4 继承

(1) 继承的定义

继承是一种允许一个类（称为子类或派生类）获取另一个类（称为父类、基类或超类）的属性和方法的机制。

(2) 继承的语法

在 Python 中，实现继承的语法非常简单。定义子类时，在类名后的括号中指定其父类即可。

```
class ParentClass:
    # 父类的属性和方法
```

```
pass
```

```
class ChildClass(ParentClass): # ChildClass 继承自 ParentClass
    # 子类可以添加自己特有的属性和方法
    # 也可以重写父类的方法
    pass
```

(3) 继承中的关键概念

super() 函数

用于调用父类（超类）的方法。最常用的场景就是在子类的 `__init__` 方法中调用父类的 `__init__` 来初始化继承来的属性。

方法重写

如果子类的方法与父类方法同名，子类的方法会覆盖父类的方法。这被称为重写。

当我们需要子类对同一个行为有不同于父类的实现时，就需要重写方法。

检查继承关系

`isinstance(object, Class)`: 检查一个对象是否是某个类或其派生类的实例。

`issubclass(ChildClass, ParentClass)`: 检查一个类是否是另一个类的子类。

(4) 继承的类型

单继承

一个子类只继承自一个父类。这是最简单、最常用的继承形式。

多继承

一个子类可以同时继承自多个父类。

```
class A:
    def method(self):
        print("Method from A")
class B:
    def method(self):
        print("Method from B")
class C(A, B): # C 同时继承自 A 和 B
    pass
```

```
obj = C()
obj.method() # 输出: Method from A
```

方法解析顺序：当多个父类有同名方法时（如上面的 `method`），Python 会按照一个特定的顺序来查找该方法，这个顺序称为 MRO。查找顺序是 `C -> A -> B -> object`，所以 `obj.method()` 找到了 A 中的方法。

8.3 实践与指导

实践 1 动物类的创建

实践目的

- (1) 掌握类的定义方法
- (2) 能够熟练编写类，对事物进行属性和方法的封装
- (3) 理解对象的创建和调用

实践要求

(1) 阅读下面程序，填写表 8-2，给代码加上注释，并写出程序输出结果；最后将代码输入到文件中，使用文件模式运行代码，将运行的结果和推演结果进行对比，如不一致，

试着分析推演错误之处。

表8-2 程序阅读

行号	代码	代码分析
#01	class Animal:	## _____
#02	def __init__(self, name):	## _____
#03	self.name = name	## _____
#04	def eat(self):	## _____
#05	print(f'{self.name} is eating.')	
#06	def make_sound(self):	## _____
#07	print(f'{self.name} makes a sound.')	
#08	bird = Animal("Poly")	## _____
#09	fish =Animal("Nemo")	## _____
#10	bird.make_sound()	##输出 _____
#11	fish.eat()	##输出 _____

实践 2 计算学生年龄

实践目的

- (1) 能够熟练编写类，对属性和方法进行封装
- (2) 能够掌握对象的实例化及相关属性的访问、方法的调用

实践要求

定义一个学生类，包括学号、姓名和出生日期三个属性；一个用于给定数据成员初始值的构造函数；一个可计算学生年龄的方法。编写该类并创建 2 个学生对象对其进行测试。根据表 8-3 的算法思路，将其转换为代码，并在开发环境中运行验证代码。

表8-3 计算学生年龄

算法思路	代码
#01 ## 导入日期时间模块 (datetime, date)	
#02 ## 定义类名Student	
#03 ## 定义构造函数，接收3个参数，学号，姓名，出生日期' YYYY-MM-DD'	

#04	## 初始化学号	
#05	## 初始化姓名	
#06	## 将字符串格式的日期转换为日期对象	
#07		
#08	## 进行日期格式的异常捕获	
#09	##提醒：出生日期格式不正确，应为 'YYYY-MM-DD'	
#10	## 定义计算年龄函数calculate_age	
#11	## 获取当前日期	
#12	## 计算年龄，为当前年份与出生年份的差	
#13	## 如果今年生日还没过	
#14	## 年龄减1	
#15	## 返回年龄	
#16	## 创建学生对象，学号"2023001", 姓名"张三",出生日期"2005-03-15"	
#17	## 创建学生对象，学号"2023002", 姓名"李四",出生日期"2004-11-22"	
#18	## 计算并打印张三年龄	
#19	## 计算并打印李四年龄	
#20	## 测试异常情况，进行日期格式异常捕获	
#21	##创建学生对象，学号"2023001", 姓名"王五",出生日期"2005/03/15"	
#22	## 捕获异常	
#23	## 输出异常信息	

实践 3 购物车结算

实践目的

- (1) 熟练掌握类的定义，将属性和方法进行封装

(2) 熟练掌握对象的创建和使用，通过对象访问属性并调用方法实现相应的功能

实践要求

(1) 实现一个购物车管理系统，该系统能够管理用户添加到购物车中的商品，并能够计算购物车中所有商品的总价。

(2) 创建一个 `ShoppingCart` 类，该类应包含以下功能：

一个初始化方法，用于初始化购物车（开始时购物车应为空）；

一个 `add_item` 方法，用于向购物车中添加商品，参数包括商品名称、价格和数量（数量默认为 1）；

一个 `__str__` 方法，用于返回购物车的字符串表示，包括所有商品信息和总价。

(3) 购物车的字符串表示格式要求：

如果购物车为空，返回“购物车是空的”；

如果购物车不为空，按照以下格式返回：

购物车内容：

- 商品 1 名称 × 数量: ¥ 商品 1 总价

- 商品 2 名称 × 数量: ¥ 商品 2 总价

...

总计: ¥ 购物车总价

(4) 编写测试代码：

创建一个购物车实例

向购物车中添加几种商品（包括不同数量的商品）

打印购物车内容，验证输出是否符合预期格式

(5) 根据要求，及相关注释，将表 8-4 中的代码补充完整，并在开发环境中运行、调试代码。

表8-4 购物车结算

算法思路	代码
#01 ##定义购物车类	class ShoppingCart:
#02 ##构造方法	def __init__(self):
#03 ##初始化购物车列表	self.items = []
#04	
#05 ##定义添加商品方法add_item	def add_item(self, name, price, quantity=1):
#06 ##向购物车列表中增加一个商品，字典类型，包含名称、价格、数量	_____
#07	
#08 ##定义__str__方法	def __str__(self):
#09	if not self.items: return "购物车是空的"
#10	result = "购物车内容:\n" total = 0

#11 ##遍历购物车	for item in self.items:
#12 ##计算每件商品的金额	item_total = _____
#13 ##将当前商品信息编辑成字符串“商品名称 × 数量: ¥ 商品1总价”，添加到结果字符串result中	result += _____
#14 ##计算总金额	_____
#15	
#16 ## 结果中增加总金额信息	result += f"总计: ¥ {total:.2f}"
#17 ## 返回result	_____
#18	
#19 ##实例化一个购物车对象cart	_____
#20 ##往购物车增加商品: Python书籍, 价格49.9, 数量2本	_____
#21 ##往购物车增加商品: 鼠标, 价格52, 数量1	_____
#22 ##输出购物车内容	_____

实践 4 教师、学生类的继承

实践目的

- (1) 理解继承的概念。
- (2) 掌握父类、子类的编写和使用方式。

实践要求

(1) 基类设计 (Person)

属性: 姓名, 年龄

方法:

构造方法, 初始化姓名、年龄

introduce 方法, 返回自己的姓名和年龄信息

(2) 子类 (Student), 继承自 Person

属性: 添加子类特有的属性学号和专业

方法:

构造方法: 调用父类的构造函数初始化共有属性, 初始化学号和专业

重写方法 introduce: 返回学生的姓名, 学号, 专业。

增加子类特有的方法 study(self, subject): 返回“该学生正在学习的课程”。

(3) 子类 (Teacher), 继承自 Person

属性: 添加子类特有的工号、部门和课程列表。

方法:

构造方法: 调用父类的构造函数初始化共有属性, 初始化工号、部门和教授的课程

列表。

重写方法 `introduce`: 返回姓名、部门和工号信息。

增加子类特有的方法 `add_course(self, course)`:将课程 `course` 添加到课程列表，返回课程信息。

增加方法 `teach()`, 返回老师目前正在教授的课程。

(4) 在表 8-5 中，将思路转换为代码，填写在表 8-5 中对应的位置。并通过在开发环境中运行，验证所写代码的准确性。

表8-5 教师、学生类的继承

算法思路	代码
#01 ## 定义基类Person	
#02 ##构造方法，初始化姓名、年龄	
#03	
#04	
#05 ## introduce方法，返回自己的姓名和年龄信息 "大家好，我叫xx，今年xx岁"	
#06	
#07	
#08 ## 定义子类Student	
#09 ## 定义构造方法	
#10 ## 调用父类的构造函数初始化共有属性	
#11	
#12 ## 添加子类特有的属性学号和专业	
#13	
#14	
#15 ## 重写父类的 introduce方法，返回字符串 "大家好，我是学生xx，学号xx，专业是xx。"	
#16	
#17	
#18 ## 增加子类特有的方法study(self, subject): 返回字符串"xx正在学习subject。"	
#19	
#20	

#21	## 定义子类Teacher，继承自Person	
#22	## 构造方法	
#23	## 调用父类的构造函数初始化共有属性	
#24	## 初始化工号、部门和课程列表	
#25		
#26		
#27		
#28	##增加子类特有的方法 add_course(self, course):将课程course添加到课程列表，并输出"xx老师开始教授xx课程。"	
#29		
#30		
#31	##增加方法teach()	
#32	## 如该教师授课课程不为空，	
#33	## 选取课程列表中的最后一门课程输出"xx老师目前正在教授xx课程"	
#34	## 否则	
#35	## 输出"xx教师目前没有安排课程。"	
#36		
#37		
#38	## 创建Person对象，"张三", 30	
#39	## 输出张三自我介绍的信息	
#40		
#41	## 创建Student对象："李四", 20, "2023001", "计算机科学"	
#42	## 调用重写的introduce方法，输出李四的自我介绍信息	
#43	## 调用学生类的study方法，输出学习Python课程的信息	
#44		

#45	##创建Teacher对象, "王教授", 45, "T1001", "计算机学院"	
#46	## 调用重写的introduce方法, 输出王教授的自我介绍信息	
#47	## 调用add_course()方法, 让王教授增加一门Python编程的课程	
#48	## 输出王教授正在教授的课程信息	
#49		
#50		
#51		
#52		

实践 5 图形计算系统

实践目的

- (1) 熟练掌握父类、子类的编写和使用, 利用继承实现代码的复用、子类的扩展。
- (2) 能够熟练使用面向对象编程解决应用问题。
- (3) 利用面向对象继承创建结构清晰、易于维护和扩展的代码。

实践要求

(1) 设计一个图形计算系统, 该系统能够计算不同几何图形的面积和周长。系统需要支持至少三种图形: 圆形、矩形和三角形。

(2) 创建一个基类 Shape, 包含以下内容:

一个构造函数, 接收图形名称作为参数;

一个 area() 方法 (抛出 NotImplementedError 异常);

一个 perimeter() 方法 (抛出 NotImplementedError 异常);

一个 __str__() 方法, 返回图形的名称、面积和周长。

(3) 创建三个派生类, 分别表示不同的几何图形:

Circle 类: 继承自 Shape, 需要接收半径作为参数, 实现 area() 和 perimeter() 方法;

Rectangle 类: 继承自 Shape, 需要接收长和宽作为参数, 实现 area() 和 perimeter() 方法;

Triangle 类: 继承自 Shape, 需要接收三边长作为参数, 实现 area() 和 perimeter() 方法。

(4) 编写测试代码:

创建每种图形的实例;

将所有图形放入一个列表中;

遍历列表, 打印每个图形的信息;

计算并打印所有图形的总面积和总周长。

提示:

圆形面积公式: $\pi \times r^2$

圆形周长公式: $2 \times \pi \times r$

矩形面积公式：长 × 宽

矩形周长公式：2 × (长 + 宽)

三角形周长公式：三边之和

三角形面积公式：使用海伦公式 $\sqrt{s(s-a)(s-b)(s-c)}$ ，其中 $s = \frac{a+b+c}{2}$

(5) 预期输出示例：

圆形 - 面积: 78.54, 周长: 31.42

矩形 - 面积: 24.00, 周长: 20.00

三角形 - 面积: 6.00, 周长: 12.00

所有图形的总面积: 108.54

所有图形的总周长: 63.42

设计程序思路，再转换为代码，并在开发环境中运行、调试代码，验证代码的准确性。

实践 6 薪资管理系统

实践目的

- (1) 熟练掌握面向对象编程。
- (2) 利用继承实现代码的复用、子类的扩展。
- (3) 能够熟练使用面向对象编程解决应用问题。

实践要求

(1) 设计一个员工薪资管理系统，该系统能够计算不同类型员工的薪资。系统需要支持至少三种员工类型：小时工、月薪工和销售员。

(2) 创建一个基类 `Employee`，包含以下内容：

一个构造函数，接收员工姓名和工号作为参数，并自动记录入职日期（默认为当前日期）

一个 `calculate_salary()` 方法（抛出 `NotImplementedError` 异常）

一个 `get_employment_duration()` 方法，返回员工入职至今的天数

一个 `__str__()` 方法，返回员工的基本信息

(3) 创建三个派生类，分别表示不同类型的员工：

`HourlyEmployee` 类：继承自 `Employee`，需要接收时薪和工作小时数作为参数；薪资计算规则：正常工作 40 小时以内按正常时薪计算，超过 40 小时的部分按 1.5 倍时薪计算。

`SalariedEmployee` 类：继承自 `Employee`，需要接收月薪和奖金（可选）作为参数；薪资计算规则：月薪 + 奖金。

`SalesEmployee` 类：继承自 `Employee`，需要接收底薪、销售额和佣金率（可选）作为参数；薪资计算规则：底薪 + 销售额 × 佣金率。

(4) 创建一个 `PayrollSystem` 类，用于管理员工和生成薪资报表：

包含一个员工列表

包含 `add_employee()` 方法用于添加员工

包含 `calculate_total_payroll()` 方法用于计算总薪资支出

包含 `print_payroll()` 方法用于打印薪资报表

(5) 编写测试代码：

创建每种员工类型的实例

将员工添加到薪资管理系统中

调用 `print_payroll()` 方法生成薪资报表

预期输出示例

== 员工薪资报表 ==

张三 (ID: E001), 雇佣时长: 0 天, 时薪: 20.0, 工作小时: 45, 薪资: 950.00

李四 (ID: E002), 雇佣时长: 0 天, 月薪: 5000, 奖金: 1000, 总薪资: 6000.00

王五 (ID: E003), 雇佣时长: 0 天, 底薪: 3000, 销售额: 50000, 佣金率: 15.0%, 总薪资:
10500.00

总薪资支出: 17450.00

(6) 设计程序思路, 再转换为代码, 并在开发环境中运行、调试代码, 验证代码的准确性。

第 9 章 文件处理

9.1 学习目标与要求

- (1) 了解文件的分类
- (2) 了解 Python 对文件读写的模式
- (3) 掌握读写 txt 文件的方法

9.2 知识要点

9.2.1 文件

文件是以计算机辅助存储设备为载体存储在计算机上的信息集合，可以是文本文档、图片、程序等。Windows 系统的数据文件按照编码方式分为两大类，文本文件和二进制文件。

(1) 文本文件。数据一般以 ASCII 或其他字符集来存放，可以处理各种语言所需的字符，只包含基本文本字符，不包括诸如字体、字号、颜色等信息。它可以在记事本等文本编辑器中显示，即文本文件都是可读的。

(2) 二进制文件。使用其它编码方式的文件即二进制文件，这种文件根据文件格式不同，需要相应的软件来处理，如：图像、音频、视频、可执行程序等。

9.2.2 文件的访问

文件访问，通常指对文件进行读、写操作。一般包括三个步骤：打开、读写、关闭。

(1) 打开文件

使用内置函数 `open` 进行文件打开。`open` 函数返回文件流操作的对象。简洁打开方式为：

文件对象变量 = `open(file, mode='r', encoding=None)`

参数 `file`：文件名，可以是绝对路径，也可以是相对路径。

参数 `mode`：访问文件的模式，默认的读取文本文件；参数的值为读/写+文件类型。`r` 表示读、`w` 表示写、`a` 表示追加，`+` 表示增加另外一种权限；文件类型，`t` 表示文本文件、`b` 表示二进制文件。如：“`wb+`”表示写读二进制文件，创建一个二进制文件并覆盖原来同名的二进制文件，同时支持从其中读取数据。

(2) 关闭文件

使用完文件必须关闭，及时释放对文件的占用。

文件对象变量.`close()`

(3) 表 9-1 为文本文件读写函数

表9-1 文本文件读写函数

方法/函数	描述	方法/函数	描述
<code>read(size=-1)</code>	读取指定数量的字符，返回字符串； <code>size</code> 默认值为-1，表示读取文件剩余的全部字符	<code>readline()</code>	读取一行
<code>readall()</code>	读取全部内容	<code>readlines()</code>	读取剩余多行内容，返回列表，列表中每个元素为一行字符串

write(str)	将字符串str，写入到文件中	writelines(seq)	将字符串序列以行的形式写入到文件中
tell()	返回文件读写指针当前位置	seek(offset, from)	移动文件指针，从from（0_起始 1_当前 2_末尾）偏移offset个字节

(4) csv 文件读写

csv 是逗号分隔符文本格式，常用于 Excel 和数据库的数据导入和导出。csv 文件特征有：

第一行标识数据列的名称；第二行开始每一行代表一条记录，存储具体的数值；每一条记录的各个数据之间一般用半角逗号（,）分隔；制表符（\t）、冒号（:）和分号（;）也是可替代半角逗号作为分隔符。

Python 中也提供了内置的 csv 模块实现 csv 文件的读写。用 csv 模块来处理，可准确、高效的对 csv 文件进行处理。表 9-2 中为 csv 模块读写 csv 格式文件的函数。

表9-2 csv模块针对csv文件读写函数

方法/函数	描述
csv.reader(csvfile, dialect='excel', **fmtparams)	csv文件读，返回一个可迭代对象，通过内置函数next()或者for...in结构可遍历csv的每行数据
csv.writer(csvfile, dialect='excel', **fmtparams)	csv文件写对象，该对象有write(row)和writerows(rows)方法，实现想csv文件中写一行和多行数据，row为一个序列，序列的每个对象为字符串。rows为row组成的序列。

9.3 实践与指导

实践 1 程序阅读

实践目的

- (1) 掌握 open 函数的使用
- (2) 深入文本文件的读操作
- (3) 掌握字符串函数的使用

实践要求

(1) data931.txt 文件与当前运行的程序代码文件在同一个文件夹中，其中的内容仅包含一行数据，内容为：

84 95 70 86 75 88 93 84 87 70 72 79 77 85 75 69 97 90 89 75

(2) 分析代码功能，并填表 9-3。

表9-3 代码分析

行号	代码	代码功能
#01	f = open("data931.txt")	
#02	s = f.read()	
#04	f.close()	
#06	s = s.replace(' ', ',')	

#07	numbers = eval(s)	
#08	avg = sum(numbers)/len(numbers)	
#09	print(f'{avg:.2f}')	

实践 2 求均值

实践目的

- (1) 掌握 csv 模块的使用
- (2) 掌握文件的打开与关闭

实践要求

(1) data932.txt 文件存放在 D 盘根目录，与当前运行的程序代码文件不在同一目录中，其中的内容包含两行数据，内容为：

84,95,70,86,75,88,93,84,87,70

72,79,77,85,75,69,97,90,89,75

- (2) 根据表 9-4 中的思路，编写程序，计算每行数据的均值。

表9-4 均值

行号	代码功能	代码
#01	## 导入csv模块	
#02	##定义变量f，用于保存open函数打开的文件流	
#04	## 调用csv模块的reader函数，对文件流f进行csv格式的读取，将 读取结果保存到 变量中	
#05	## 将#04行变量 转换为列表，并保存到变量ls中	
#06	## 关闭文件流	
#07	## 遍历列表ls	
#08	## 计算ls中每个元素(子列表)的均值，并输出。结果保留2位小数	

提示：#09 行的 ls 的元素(子列表)，是字符串类型，需要做进一步转换，才能计算均值。

实践 3 最高得分名单

实践目的

- (1) 掌握 csv 模块的使用
- (2) 掌握文件的打开与关闭

实践要求

(1) data933.txt 文件存放与当前运行的程序代码文件不在同一目录中，其中的内容包含多行数据，每行为一个学生的成绩，之间用英文逗号分隔，内容为：

BettyLewis,60
MarkWalker,62
MarkLee,55

(2) 阅读表 9-5 中的代码，输出最高分及最高分得分的学生人名。代码有两处错误，请找出并改正，有两处下划线需要补充完善。请为代码加上注释，改正并完善后，将代码输入到计算机，并用 IDLE 运行。

表9-5 最高得分者

行号	代码	注释
#01	import csv	
#02	f = open("data933.txt")	
#04	csv_file = csv.reader(f)	
#06	ls = list(csv_file)	
#07	f.closed()	
#08	scores = set()	
#09	for row in ls:	
#10	row[-1] = _____	
#11	scores.add(row[0])	
#12	print(f"最高分:{max(scores)}")	
#13	for row in ls:	
#14	_____	
#15	print(row[0])	

代码错误有两处，错误 1 的行号是_____，修改后的代码_____。
错误 2 的行号是_____，修改后的代码_____。
#10 行，空白处应填写的代码是_____。

读操作
的常规操作

145 11/23/2011

存放与当前运行
字母、数字、其
为：字母、数字

写思路,然后将

表9-6 字符统计

[illegible]

实践 5 村长选举

实践目的

- (1) 掌握文本文件读写方法
- (2) 掌握数据处理的一般方法

实践要求

(1) data935.txt 文件存放与当前运行的程序代码文件不在同一目录中，其中的内容为选举村长的投票，记录员将候选村长姓名保存到 data935.txt 中，每一行为一个候选人，请编写程序，统计村长候选人的票数，按照票数降序保存到 data935_result.txt 中。

data935.txt 中数据样式如下：

周洋
李娜
王芳

...

data935_result.txt 中数据样式如下：

吴娟 得票 59
黄磊 得票 58
李娜 得票 58
黄磊 得票 58
李娜 得票 58

...

(2) 在表 9-7 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表9-7 票数统计

算法思路	代码

实践 6 成绩统分析

实践目的

- (1) 掌握 csv 文件读写方法
- (2) 掌握数据处理的一般方法

实践要求

(1) data936.txt 文件存放与当前运行的程序代码文件不在同一目录中，文件编码为 utf-8。其中的内容包含 n 个学生的成绩，每个学生成绩分为平时、期中、期末成绩，请编程实现对所有学生成绩的总成绩进行计算，并按照分数段进行统计分析。

总评成绩 = 平时成绩 × 10% + 期中成绩 × 30% + 期末成绩 × 60%

data936.txt 的数据如下：

姓名,平时成绩,期中成绩,期末成绩,总评成绩
BettyLewis,91,60,77,
MarkWalker,98,62,78,
MarkLee,94,55,78,
...

处理后的成绩，保存到 data936_result.txt 文件中。

姓名,平时成绩,期中成绩,期末成绩,总评成绩
BettyLewis,91,60,77,73.3
...

MaryWhite,89,66,79,76.1

SusanLewis,95,57,74,71.0

同时将处理结果打印到屏幕上：

姓名	平时成绩	期中成绩	期末成绩	总评成绩
BettyLewis	91	60	77	73.3
MarkWalker	98	62	78	75.2
MarkLee 94	55	78	72.7	
BettyLopez	97	64	80	76.9
MaryWhite	89	66	79	76.1
SusanLewis	95	57	74	71.0

总人数：100

不及格 人数：0 占比：0.0

及格 人数：9 占比：0.09

中等 人数：61 占比：0.61

良好 人数：28 占比：0.28

优秀 人数：2 占比：0.02

(2) 在表 9-8 中，先写思路，然后将思路转换为代码。并通过 IDLE 的文件执行模式进行验证所写代码的准确性。

表9-8 成绩处理分析

算法思路	代码
------	----

第 10 章 中文分词和词云

10.1 学习目标与要求

- (1) 掌握中文文本分词处理的常用方法、关键词进行提取的方法
- (2) 能够用 jieba 模块进行中文分词和关键词提取
- (3) 掌握词云图的制作步骤及关键技术

10.2 知识要点

10.2.1 分词

分词就是将连续的文字序列按照一定的规范重新组合成语义独立词序列的过程。在英文的句子中，单词之间是以空格作为自然分界符的，因而分词相对容易。中文分词是文本挖掘的基础，对于输入的一段中文，成功的进行中文分词，可以达到电脑自动识别语句含义的效果。

10.2.2 jieba

jieba 是一个支持中文分词、高准确率、高效率的 Python 中文分词组件，它支持以下 3 种分词模式：精确模式、全模式和搜索引擎模式。jieba 库提供的主要函数如表 10-1 所示。

表10-1 jieba的函数

方法/函数	描述	方法/函数	描述
jieba.cut(s)	对s分词（精确模式） 返回可迭代对象	jieba.cut(s,cut_all=True)	对s分词（全模式） 返回可迭代对象
jieba.lcut(s)	对s分词（精确模式） 返回列表	jieba.lcut(s,cut_all=True)	对s分词（全模式） 返回列表
jieba.cut_for_search(s)	对s分词（搜索引擎模式），返回可迭代对象	jieba.lcut_for_search(s)	对s进行分词（搜索引擎模式），返回列表
jieba.add_word(w)	向分词词典中增加新词w	jieba.del_word(w)	从分词词典中删除词语w
jieba.load_userdict(filename)	载入使用自定义分词词典filename 注：自定义分词词典中每个单词占一行		

jieba 词典的维护，常用几种方法：
使用 jieba.add_word()方法添加自定义词语到分词词典中。
使用 jieba.del_word()方法从词典中删除指定词语。
使用 jieba.load_userdict('userdict.txt')方法加载自定义词典，自定义词典格式要和默认词典 dict.txt 一样，一个词占一行，每一行分三部分：词语、词频（可省略）、词性（可省略），用空格隔开，顺序不可颠倒。file_name 若为路径或二进制方式打开的文件，则文件必须为 UTF-8 编码。

10.2.3 关键词提取

关键词提取使用 jieba 中的 analyse 模块。其中，常用的关键词提取有两种算法。
第一种是 TF-IDF 算法，基于 TF-IDF 算法的关键词抽取方法如下：
jieba.analyse.extract_tags(sentence, topK=10, withWeight=False, allowPOS=())

sentence 为待提取的文本;
topK 为返回最大权重关键词的个数, 默认值为 20;
withWeight 表示是否返回权重, 是的话返回(word, weight)的 list, 默认为 False;
allowPOS 为筛选指定词性的词, 默认为空, 即不筛选 (n-名称 v-动词 ns-地点名词)
第二种是 TextRank 算法, 基于 TextRank 算法的关键词抽取方法如下:

```
jieba.analyse.textrank(sentence, topK=10, withWeight=False, allowPOS=())
```

10.2.4 词云图

词云通过以词语为基本单位, 然后根据词语的出现频率确定词语的大小, 将这些词放到图片里, 更加直观和艺术地展示文本。

1. wordcloud 库

wordcloud 是 Python 中优秀的词云展示第三方库。wordcloud 库的核心是 WordCloud 类, 所有的功能都封装在 WordCloud 类中。使用 wordcloud 库生成英文词云图, 一般按照以下步骤:

- 步骤 1: 实例化一个 WordCloud 对象, 例如 wc=WordCloud()。
- 步骤 2: 调用 wc.generate(text), 对文本 text 进行分词, 并生成词云图。
- 步骤 3: 调用 wc.to_file("wc_test.png"), 把生成的词云图输出到图像文件。

一般情况下, WordCloud 对象使用默认参数创建词云图。创建 WordCloud 实例对象时, 用户可以通过参数控制词云图的绘制。创建 WordCloud 对象的常用参数如表 10-2 所示。

表10-2 创建WordCloud对象的常用参数

参数	功能	参数	功能
width	生成图片宽度 默认400像素	height	生成图片高度 默认200像素
mask	指定词云形状 默认长方形	font_path	字体文件路径, 默认None 绘制中文词云图时须指定字体
max_words	词云显示的最大单词数 默认200	max_font_size	词云中最大的字体号
in_font_size	词云中最小的字体号 默认4号	font_step	词云中字体字号的步进间隔 默认为1
stop_words	指定词云的排除词列表 即不显示的单词列表	background_color	词云图的背景颜色 默认黑色

注: wordcloud 依赖于 pillow(PIL)和 numpy 库, 如果要预览图片, matplotlib 也是必须的。需要安装几个库:

```
pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install PIL -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install matplotlib -i https://pypi.tuna.tsinghua.edu.cn/simple
```

选取词云形状背景图提示:

- (1) 推荐使用 PNG 格式 (支持透明背景), 也支持 JPG/JPEG 格式, 但需要确保背景与主体对比明显;
- (2) 建议选择轮廓清晰、简洁明了的图片, 主体部分应为纯白色或透明 (词云将填充的区域), 背景部分应为纯黑色或其他深色 (词云不会出现的区域);
- (3) 简单的形状 (如心形、动物轮廓、地理形状等) 效果最佳。

2.中文词云图的制作

对于中文文本，词云图的生成一般按照以下步骤。

- 步骤 1：加载文本。
 - 步骤 2：使用 jieba 将文本进行分词处理。
 - 步骤 3：把分词结果（列表）以空格为分隔符拼接成本文。
 - 步骤 4：实例化一个 WordCloud 对象，如：wc=WordCloud()。中文需指定字体。
 - 步骤 5：调用 wc.generate(text)，生成词云图。
 - 步骤 6：调用 wc.to_file("wc.png")，把生成的词云图输出到图像文件。
- 也可以使用关键词来生成词云图，具体步骤如下：
- 步骤 1：加载文本。
 - 步骤 2：使用 jieba.analyse 将文本进行关键词提取，得到关键词列表。
 - 步骤 3：把关键词列表转换成字典。
 - 步骤 4：实例化一个 WordCloud 对象，如：wc=WordCloud()。中文需指定字体。
 - 步骤 5：调用 wc.generate_from_frequencies(keyDict)，生成词云图。
 - 步骤 6：调用 wc.to_file("wc.png")，把生成的词云图输出到图像文件。

10.3 实践与指导

实践 1 中文分词三种模式

实践目的

- (1) 掌握用 jieba 进行中文分词的常用方法，能够对词典进行维护
- (2) 掌握用 jieba 相应模块进行关键词进行提取的方法

实践要求

- (1) 安装 jieba 库；
- (2) 对一段中文文本进行分词，分别使用 3 种不同分词模式并输出分词结果；
- (3) 阅读代码，在表 10-3 中补充对代码的注释，并在开发环境中运行验证代码的准确性。

表10-3分词的3种模式

行号	代码	代码分析
#01	import jieba	## _____
#02	text = "自然语言处理是人工智能领域的重要方向， 中文分词是自然语言处理的基础步骤之一。"	
#03		
#04	words_precise = jieba.cut(text, cut_all=False)	## _____
#05	print("精确模式:", "/".join(words_precise))	## _____
#06	words_precise = jieba.lcut(text)	## _____
	print("精确模式:", words_precise)	## _____

#07	<code>words_full = jieba.cut(text, cut_all=True)</code>	## _____
	<code>print("全模式:", "/".join(words_full))</code>	
#08		
#09	<code>words_search = jieba.cut_for_search(text)</code>	## _____
#10	<code>print("搜索引擎模式:", "/".join(words_search))</code>	## _____
#11		

实践 2 词典的维护

实践目的

- (1) 熟练使用 jieba 分词的常用方法
- (2) 掌握分词词典常用的维护方法，进一步提高分词的准确性

实践要求

- (1) 对一段中文文本进行分词，使用默认词典分词，并输出分词结果；
- (2) 使用手动向词典中添加词语“工业互联网”，进行分词并输出结果；
- (3) 从词典中删除词语“改数”后，再次分词并输出结果；
- (4) 创建自定义用户词典 `user_dict.txt`，在词典中增加词语：“智改数转”、“工业互联网”、“智能制造”、“制造强国”（每行一个），使用用户词典进行分词并输出结果；
- (5) 在表 10-4 中补充完整的代码，并在开发环境中运行代码，比较几次分词的结果有何不同。

表10-4 词典的维护

行号	思路	代码
#01	## 导入 jieba库	_____
#02	##待处理的原始文本	<code>text = '工业互联网是智改数转新趋势下实现智能制造的关键'</code>
#03	# 默认分词测试	<code>default_words = jieba.lcut(text, cut_all=False)</code>
#04		<code>print("默认分词:", default_words)</code>
#05	#将“工业互联网”添加到词典	<code>jieba.add_word("工业互联网")</code>
#06	#使用精确模式将文本进行分词，返回分词列表	_____
#07		<code>print("添加新词后分词: ", cutlist)</code>

#08		
#09	#从词典中删除词语"改数"	
#10	#使用精确模式将文本进行分词，返回分词列表	
#11	#输出分词结果	print("删除词语后分词：", cutlist)
#12		
#13	# 添加自定义词典（在当前目录添加自定义词典文件 user_dict.txt）	
#14	#使用精确模式重新分词，返回分词列表	
#15	#输出分词结果	print("自定义词典分词:", custom_words)

实践 3 文件关键词提取

实践目的

- （1）能够用 jieba 模块进行中文分词
- （2）熟练掌握使用 jieba 进行关键词提取

实践要求

- （1）创建文件 npl.txt，从文件中读取文本内容，文件内容如下：

自然语言处理是人工智能和语言学领域的分支学科。此领域探讨如何处理及运用自然语言；自然语言处理包括多方面和步骤，基本有认知、理解、生成等部分。自然语言认知和理解是让电脑把输入的语言变成有意思的符号和关系，然后根据目的再处理。自然语言生成系统则是把计算机数据转化为自然语言。

- （2）分别使用 TF-IDF 和 TextRank 算法对文件内容进行关键词提取，输出排名前五的关键词及其权重。

- （3）根据表 10-5 中的思路提示，完成代码，并在开发环境中运行验证代码的准确性。

表10-5 文件关键词提取

行号	思路	代码
#01	## 导入 jieba库	
#02	## 用with方式打开文件'npl.txt'	
#03	## 读取文件内容	
#04		
#05	##打印原文内容，	print("原文摘要:")
#06	##只显示前100字符	
#07		

#08	print("\nTF-IDF关键词提取:")
#09	# 1. 基于TF-IDF算法的关键词提取
#10	##循环输出关键词及其权重
#11	## 权重保留4位小数, 输出格式如下: ## 处理: 0.4919
#12	
#13	print("\nTextRank关键词提取:")
#14	# 2. 基于TextRank算法的关键词提取
#15	##循环输出关键词及其权重
#15	## 权重保留4位小数, 输出格式如下: ## 处理: 0.4919
#16	

实践 4 英文小说词云图制作

实践目的

- (1) 掌握使用 wordcloud 库制作词云图的基本方法。
- (2) 熟练掌握定制词云图的绘制参数, 制作英文词云图。

实践要求

- (1) 安装第三方库 wordcloud、Numpy、PIL;
- (2) 从文件 Educated.txt 中读取英文小说《你当像鸟飞往你的山》的部分原文, 使用 wordcloud 库生成一个白色背景、飞鸟形状的词云图, 最多显示 100 个词语。文件内容如下:

I am only seven years old when I first understand what it is that my father fears.
We are standing on the deserted pasture near the southern slope of Buck's Peak. The wind is fierce, whipping my hair across my face and into my mouth. Mother is gesturing toward the peak. "That's where the school is," she says.
My father grunts. "I don't like it," he says. "The government's got no business telling me how to raise my kids."
This is the first I have heard of school. I know other kids go—Jentel down the road, the ones at church—but it has never occurred to me that I might go too.
The wind picks up, howling through the juniper trees. My father stares at the mountain for a long moment, then turns and walks back to the house. We follow him, our feet crunching on the frozen grass.
That was the beginning of my father's great fear of the government, and of the world beyond the mountain. It was a fear that would shape our lives, a fear that would keep us children trapped in the shadow of the peak, a fear that I would spend my youth trying to escape, and my adulthood trying to understand.

- (3) 根据注释提示完成表格 10-6 中的代码, 并在开发环境中调试、运行代码。

表10-6 英文小说词云图

行号	思路	代码
#01	## 导入 wordcloud模块	from wordcloud import WordCloud
#02	## 导入 numpy模块	_____
#03	## 导入 PIL中的Image模块	_____
#04		
#05	## 用with方式打开文件'Educated.txt'	_____
#06	## 读取文件内容	_____
#07		
#08	## 1. 加载并处理背景图片位np-array类型	_____
#09	## 2. 创建词云对象	wc = WordCloud(
#10	## 背景颜色	_____
#11	## 设置词云形状图片	_____
#12	## 设置最大词语数100	_____
#13		)
#14	## 3.生成词云	_____
#15	## 4.保存词云图到文件 'educated.png'	_____

实践 5 制作《给青年的十二封信》中文词云图

实践目的

- (1) 掌握使用 wordcloud 库制作中文词云图的方法
- (2) 能够熟练使用中文分词进行词云图的制作，定制词云图的绘制参数

实践要求

- (1) 读取文件“给青年的十二封信.txt”内容，去除全文标点符号，进行中文分词；
- (2) 分析分词结果，将一些未能识别的新名词、专有名词以词典的方式导入到分词库中；
- (3) 设置停用词：“的”、“很”、“我们”、“非常”等。
- (4) 用全文分词结果绘制词云图，图中设置最大显示词数为 100。
- (5) 设计程序思路，填到表 10-7 中，并在表 10-7 中将其转换为代码，并在开发环境

中运行、调试代码，验证代码的准确性。

表10-7 十二封信

行号	思路	代码
#01		
#02		
#03		
#04		
#05		
#06		
#07		
#08		
#09		
#10		
#11		
#12		
#13		
#14		
#15		
#16		

实践 6 制作《苏东坡传》关键词词云图

实践目的

- (1) 掌握使用 wordcloud 库制作中文词云图的方法
- (2) 熟练使用关键词制作词云图

实践要求

- (1) 读取文件“苏东坡传.txt”内容，去除全文标点符号，进行中文分词；
- (2) 选择一个关键词提取算法，输出前 10 个关键词及对应的权重；
- (3) 使用关键词绘制词云图（图形自选），并保存图片为‘苏东坡.png’。
- (4) 设计程序思路，填到表 10-8 中，并在表 10-8 中将其转换为代码，并在开发环境中运行、调试代码，验证代码的准确性。

表10-8 苏东坡传

行号	思路	代码
#01		
#02		
#03		
#04		
#05		
#06		
#07		
#08		
#09		
#10		
#11		
#12		
#13		
#14		
#15		
#16		

第 11 章 图形界面开发

11.1 学习目标与要求

- (1) 掌握图形界面开发的流程
- (2) 熟悉 tkinter 库的使用，tkinter 的几何布局，控件使用，事件管理等基本功能
- (3) 能够熟练使用 tkinter 进行应用程序开发

11.2 知识要点

11.2.1 Tkinter 库

Tkinter 是 Python 的标准 GUI 库。Python 使用 Tkinter 可以快速的创建 GUI 应用程序。Tkinter 是内置到 python 的安装包中、只要安装好 Python 之后就能 import Tkinter 库、而且 IDLE 也是用 Tkinter 编写而成、对于简单的图形界面 Tkinter 还是能应付自如。
注意：Python3.x 版本使用的库名为 tkinter,即首写字母 T 为小写，import tkinter。
可以用以下两种方式导入 tkinter 模块：

```
from tkinter import *  
import tkinter as tk
```

11.2.2 Tkinter 组件

Tkinter 提供各种控件，如按钮，标签和文本框，一个 GUI 应用程序中使用。这些控件通常被称为控件或者部件。目前有 15 种 Tkinter 的组件，如表 11-1 所示。

表11-1 Tkinter组件

控件	描述	控件	描述
Button	按钮控件 在程序中显示按钮	Canvas	画布控件 显示图形元素如线条或文本
Checkbutton	多选框控件 在程序中提供多项选择框	Entry	输入控件 用于显示简单的文本内容
Frame	框架控件 在屏幕上显示一个矩形区域，多用来作为容器	Label	标签控件 可以显示文本和位图
Listbox	列表框控件 用来显示一个字符串列表	Menubutton	菜单按钮控件 用于显示菜单项
Menu	菜单控件，显示菜单栏 下拉菜单和弹出菜单	Message	消息控件 用来显示多行文本
Radiobutton	单选按钮控件 显示一个单选的按钮状态	Scale	范围控件，显示一个数值刻度，为 输出限定范围的数字区间
Scrollbar	滚动条控件，内容超过显示 区域时使用	Text	文本控件 用于显示多行文本
Toplevel	容器控件，用来提供一个 单独的对话框	Spinbox	输入控件，与Entry类似，但是可以 指定输入范围值
PanedWindow	窗口布局管理插件	LabelFrame	简单的容器控件，用于复杂的窗口 布局
tkMessageBox	用于显示应用程序消息框		

标准属性也就是所有控件的共同属性，如大小，字体和颜色等，表 11-2 为 Tkinter 组件的标准属性。

表11-2 Tkinter组件标准属性

控件	描述	控件	描述
Dimension	控件大小	Color	控件颜色
Font	控件字体	Anchor	锚点
Relief	控件样式	Bitmap	位图
Cursor	光标		

11.2.3 布局管理器

Tkinter 提供了三种常用的布局管理器，分别是 pack()、grid() 以及 place()，如表 11-3 所示，上述三种方法适用于 Tkinter 中的所有控件。

表11-3 tkinter布局管理器

几何方法	描述
pack()	按照控件的添加顺序其进行排列，遗憾的是此方法灵活性较差；
grid()	以行和列（网格）形式对控件进行排列，此种方法使用起来较为灵活；
place()	可以指定组件大小以及摆放位置，三个方法中最为灵活的布局方法；

1. pack 布局

pack() 是一种较为简单的布局方法，在不使用任何参数的情况下，它会将控件以添加时的先后顺序，自上而下，一行一行的进行排列，并且默认居中显示。每个控件对象都有 pack()方法，调用控件的 pack()方法时，即通知 pack 布局管理器放置控件。pack() 方法的常用参数如表 11-4 所示。

表11-4 pack方法常用参数

属性	说明
anchor	组件在窗口中的对齐方式，有 9 个方位参数值，比如 "n"/"w"/"s"/"e"/"ne"，以及 "center" 等（这里的 e w s n 分别代表，东西南北）
expand	是否可扩展窗口，参数值为 True（扩展）或者 False（不扩展），默认为 False，若设置为 True，则控件的位置始终位于窗口的中央位置
fill	参数值为 X/Y/BOTH/NONE，表示允许控件在水平/垂直/同时在两个方向上进行拉伸，比如当 fill = X 时，控件会占满水平方向上的所有剩余的空间。
ipadx,ipady	需要与 fill 参数值共同使用，表示组件与内容和组件边框的距离（内边距），比如文本内容和组件边框的距离，单位为像素(p)，或者厘米(c)、英寸(i)
padx,pady	用于控制组件之间的上下、左右的距离（外边距），单位为像素(p)，或者厘米(c)、英寸(i)
side	组件放置在窗口的哪个位置上，参数值 'top','bottom','left','right'。注意，单词小写时需要使用字符串格式，若为大写单词则不必使用字符串格式

2. grid 布局

grid 布局是一种基于网格的布局管理方法，相当于把窗口看成了一张由行和列组成的表格。grid 占位表格的行和列都是从 0 开始编号的，控件定位时，使用 column 选项指定行编号，使用 row 选项指定列编号。grid 管理器管理的占位表格中，若行列上没有控件，则这些行列不可见，即没有高度和宽度。

grid() 函数的常用参数如表 11-5 所示。

表11-5 grid方法常用参数

属性	说明
column	控件位于表格中的第几列，窗体最左边的为起始列，默认为第 0 列
columnspan	控件实例所跨的列数，默认为 1 列，通过该参数可以合并一行中多个邻近单元格。
ipadx,ipady	用于控制内边距，在单元格内部，左右、上下方向上填充指定大小的空间。
padx,pady	用于控制外边距，在单元格外部，左右、上下方向上填充指定大小的空间。
row	控件位于表格中的第几行，窗体最上面为起始行，默认为第 0 行
rowspan	控件实例所跨的行数，默认为 1 行，通过该参数可以合并一列中多个邻近单元格。
sticky	该属性用来设置控件位于单元格那个方位上，参数值和 anchor 相同，若不设置该参数则控件在单元格内居中

注意，在一个程序中不能同时使用 pack() 和 grid() 方法，这两个方法只能二选一，否则程序会运行错误。

3. place 布局

place 布局管理器是使用绝对坐标来排列控件。采用 place() 方法进行布局管理要更加精细化。place() 方法的一般调用格式如下：

控件对象.place(坐标[,其他选项...])

坐标有两种方式：方式一(绝对坐标)：x=值 1,y=值 2；方式二(相对坐标)：relx=值 1, rely=值 2

place 布局管理器的常用属性如表 11-6 所示。

表11-6 place方法常用参数

属性	说明
anchor	定义控件在窗体内的方位，参数值N/NE/E/SE/S/SW/W/NW 或 CENTER，默认值是 NW
bordermode	定义控件的坐标是否要考虑边界的宽度，参数值为 OUTSIDE（排除边界）或 INSIDE（包含边界），默认值 INSIDE。
x、y	定义控件在根窗体中水平和垂直方向上的起始绝对位置
relx、rely	1. 定义控件相对于根窗口（或其他控件）在水平和垂直方向上的相对位置（即位移比例），取值范围再 0.0~1.0 之间 2. 可设置 in_ 参数项，相对于某个其他控件的位置
height、width	控件自身的高度和宽度（单位为像素）

relheight、relwidth	控件高度和宽度相对于根窗体高度和宽度的比例，取值也在 0.0~1.0 之间
--------------------	---------------------------------------

relx 和 rely 参数指定的是控件相对于父组件的位置，而 relwidth 和 relheight 参数则是指定控件相对于父组件的尺寸大小。

11.2.4 事件处理

Tkinter 中主要有三种事件：鼠标事件、键盘事件和窗体事件。

1. 事件描述

事件描述符是以字符串的形式表示的，tkinter 中经常使用的事件类型描述格式为：
<modifier-type-detail>，其中：

- modifier：事件修饰符，如 Alt、Shift 组合键。
 - type：事件类型，如按键（Key）、鼠标(Button/Enter/Leave/Release)等。
 - detail：事件细节，如：鼠标左键（1）、鼠标中键（2）、鼠标右键（3）。
- 常用事件描述符表示如表 11-7 所示。

表11-7常用事件描述符

描述	说明	描述	说明
<Button-1> <ButtonPress-1> <1>	1按下鼠标左键，2右键，3中键	<ButtonRelease-1>	左键释放
<B1-Motion>	按住鼠标左键移动	<Double-Button-1>	双击左键
<Enter>	鼠标指针进入某一组件区域	<Leave>	鼠标指针离开某一组件区域
<KeyPress>	按下任意键	<KeyPress-A>	按下键盘A键
<Alt-KeyPress-a>	同时按下alt和a，alt可用ctrl和shif代替	<Double-KeyPress-a>	快速按两下a
<Control-V>	CTRL 和V键同时按下，V可以换成其他键		

2. 事件绑定

（1）创建组件对象时绑定

创建组件对象实例时，通过其命名参数 command 指定事件处理函数。command=函数，点击控件的时候 Tkinter 将会自动调用被绑定的函数。

按钮 Button、单选按钮 Radiobutton、复选框 Checkbutton 等都有 command 属性。

#事件处理函数

```
def callback():
    showinfo("Python command","人生苦短、我用 Python")
    Bu1=Button(root, text="设置事件调用命令",command=callback)
    Bu1.pack()
```

（2）使用 bind()方法进行事件绑定

Tkinter 可以使用 bind()方法在三个级别上将事件处理程序绑定到事件，实例绑定、类绑定、应用程序绑定。

步骤 1：在 Python 程序中创建 GUI 控件，一般把需要事件处理的控件存放在变量中。

步骤 2：创建自定义的处理函数，该函数必须包含一个 event 参数。自定义的处理函数

格式如下：

def 函数名(参数 event):

函数体

步骤 3：调用 bind()方法建立控件和处理函数间的绑定。

步骤 4：当调用窗口的 mainloop()函数激活窗体及其控件后，tkinter 将监控用户的动作，并生成 event 对象。对建立了事件处理绑定的控件，一旦发生了指定事件后，调用处理函数，执行其中代码。

11.3 实践与指导

实践 1 猜数字游戏

实践目的

- (1) 掌握图形界面开发流程
- (2) 掌握 tkinter 库的使用，熟练控件的创建、布局管理器、事件的绑定

实践要求

(1) 使用 Tkinter 开发猜数字游戏，运行效果如图所示。游戏中电脑随机生成 1024 以内数字，玩家去猜，如果猜得数字过大过小都会提示，程序要统计玩家猜的次数。图 11-1、11-2 为猜数字游戏运行的截图。



图 11-1 猜数字游戏 1



图 11-2 猜数字游戏 2

- (2) 阅读下面程序，填写表 11-8，然后在开发环境中运行代码。

表11-8 猜数字游戏

行号	代码	分析
#01	import tkinter as tk	##
#02	import random	
#03		
#04	number = random.randint(0,1024)	##
#05	running = True	##
#06	num = nminn = 0	##
#07	nmaxn = 1024	##

#08		
#09	def eBtnClose(event):	
#10	root.destroy()	## _____
#11		
#12	def eBtnGuess(event):	## _____
#13	global nmaxn, nminn , num, running	## _____
#14	if running:	
#15	val_a = int(entry_a.get())	
#16	if val_a == number:	##获取用户输入值
#17	labelqval("恭喜答对了！")	
#18	num+=1	
#19	running = False	
#20	numGuess()	## _____
#21	elif val_a < number:	
#22	if val_a > nminn:	
#23	nminn = val_a	
#24	num+=1	##config方法的作用:
#25	label_tip_min.config(label_tip_min,text=nminn)	_____
#26	labelqval("小了哦")	
#27	else:	
#28	if val_a < nmaxn:	
#29	nmaxn = val_a	
#30	num+=1	
#31	label_tip_max.config(label_tip_max,text=nmaxn)	
#32	labelqval("大了哦")	## _____
#33	else:	
#34	labelqval('你已经答对啦...')	
#35		
#36	def numGuess():	##函数功能: _____
#37	if num == 1:	
#38	labelqval('wow! 一次答对!')	
#39	elif num < 10:	
#40	labelqval('=十次以内就答对了, 牛! 尝试次数:	
#41	'+str(num))	
#42	elif num < 50:	
#43	labelqval('还行哦尝试次数: '+str(num))	
#44	else:	
#45	labelqval('好吧...您都试了超过50次了...尝试次数:	
#46	'+str(num))	
#47		
#48	def labelqval(vText):	
#49	label_val_q.config(label_val_q,text=vText)	
#50		
#51	root = tk.Tk(className="猜数字游戏")	## _____
#52	root.geometry("400x90+200+200")	
#53		
#54	line_a_tip = tk.Frame(root)	
#55	label_tip_max = tk.Label(line_a_tip,text=nmaxn)	## _____
	label_tip_min = tk.Label(line_a_tip,text=nminn)	
	label_tip_max.pack(side = "top",fill = "x")	
	label_tip_min.pack(side = "bottom",fill = "x")	
	line_a_tip.pack(side = "left",fill = "y")	
	line_question = tk.Frame(root)	
	label_val_q = tk.Label(line_question,width="80")	
	label_val_q.pack(side = "left")	

#56	line_question.pack(side = "top",fill = "x")	
#57	line_input = tk.Frame(root)	## _____
#58	entry_a = tk.Entry(line_input,width="40")	## 创建输入框控件
#59	btnGuess = tk.Button(line_input,text="猜")	## _____
#60	entry_a.pack(side = "left")	## _____
#61	entry_a.bind('<Return>',eBtnGuess)	## _____
#62	btnGuess.bind('<Button-1>',eBtnGuess)	## _____
#63	btnGuess.pack(side = "left")	## _____
#63	line_input.pack(side = "top",fill = "x")	
#65	line_btn = tk.Frame(root)	
#66	btnClose = tk.Button(line_btn,text="关闭")	
#67	btnClose.bind('<Button-1>',eBtnClose)	## btnClose.bind方法的作用是:
#68	btnClose.pack(side="left")	## _____
#69	line_btn.pack(side = "top")	
#70	labelqval("请输入0到1024之间任意整数: ")	
#71	entry_a.focus_set()	
#72	print(number)	
#73	root.mainloop()	

实践 2 背单词游戏

实践目的

- (1) 掌握图形界面开发流程
- (2) 掌握 tkinter 库的使用，熟练控件的创建、布局管理器、事件的绑定
- (3) 能够将控制台程序转换成 GUI 程序，处理用户的输入输出和事件的触发

实践要求

- (1) 使用 csv 文件（EnglishWords.csv，一列英文一列中文词义）存储单词，作为单词库，随机抽取一个单词作为目标单词。
- (2) 将目标单词中文输出到窗口，请用户回答正确的英文单词。用户在窗口文本框中输入自己的答案，点击按钮提交答案。
- (3) 将用户输入的单词与目标单词进行比较，根据猜测的结果，在窗口给出相应的反馈。用户对该单词猜测结束后，可以选择是否进行下一局游戏。如
- (4) 根据表 11-9 中相关提示思路，完成代码，并在开发环境中运行、调试代码。

表11-9 背单词练习

算法思路	代码
#01 ## 导入需要的模块	
#02	
#03	
#04 ## 定义单词库准备函数 getWordList ## 打开文件EnglishWords.csv ## 读取单词的中英文2列，结果以列表返回所有单词 的中英文二元组[(english,中文),()...]	

#20	## 定义随机抽取单词函数 getRandomWord ## 调用getWordList, 从单词库随机获取一个单词元组作为目标单词, 并保存作为答案 ## 获取其中文含义, 并返回
#028	## 定义用户答案判断函数 checkAnswer ## 获取用户输入 ## 与答案进行比对, 生成对或错不同的提示信息, 并计数 ## 将提示信息更新至用户窗口的标签内容, 告知答题结果
#40	## 定义游戏结束函数 gameOver ## 统计用户一共背的单词数, 其中正确、错误的数量, 输出到窗口
#48	## 主程序部分 ##创建窗口对象, 并作设置 ## 添加需要的控件, 如文本标签、文本框、按钮等 ## 文本标签用来显示提示信息 ## 文本框供用户输入答案 ## “提交”按钮, 设置属性, 事件绑定-checkAnswer函数 ## “下一个”按钮, 抽取新单词, 绑定事件-getRandomWord函数 ## “结束练习”按钮, 绑定事件gameOver函数

实践 3 单位转换器

实践目的

- (1) 掌握图形界面开发流程
- (2) 掌握 tkinter 控件的创建、布局管理器、事件的绑定
- (3) 能够熟练处理 GUI 应用程序用户的输入输出和事件的触发

实践要求

- (1) 使用 tkinter 库开发图形界面程序, 实现常用单位的转换, 效果如图所示。
- (2) 用户选择要转换的单位类型 (如: 长度、温度), 在“从”下拉菜单中选择原单位, 在“到”下拉菜单中选择目标单位, 在原单位输入框中输入数值, 程序自动 (或点击按钮) 计算出结果并显示。图 11-3 为单位换算器的运行图。

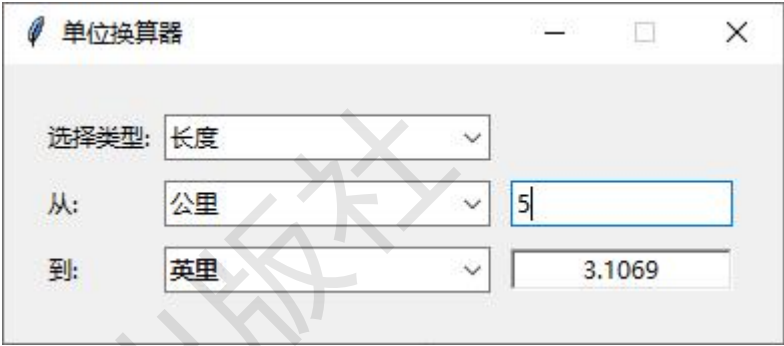


图 11-3 单位换算器

(3) 可以通过预先准备单位转换的字典，设置转换规则：

```
CONVERSIONS = {
    "长度": {
        "米 -> 英尺": lambda m: m * 3.28084,
        "英尺 -> 米": lambda ft: ft / 3.28084,
        "公里 -> 英里": lambda km: km * 0.621371,
        "英里 -> 公里": lambda mi: mi / 0.621371,
    },
    "温度": {
        "摄氏度 -> 华氏度": lambda c: (c * 9 / 5) + 32,
        "华氏度 -> 摄氏度": lambda f: (f - 32) * 5 / 9,
    }
}
```

(4) 按照 GUI 程序开发流程，从创建窗口、添加控件、事件函数的编写及绑定几个步骤设计程序的思路，并完成代码，填入到表 11-10 中。

表11-10 单位换算器

算法思路	代码
## 导入需要的模块	

[illegible]

实践 4 用户登录界面

实践目的

- (1) 掌握图形界面开发流程
- (2) 掌握 tkinter 控件的创建、布局管理器、事件的绑定
- (3) 能够熟练处理 GUI 应用程序用户的输入输出和事件的触发

实践要求

- (1) 使用 Tkinter 实现用户登录界面，如图 11-4 所示。



图 11-4 登录

(2) 建立一个文本文件 `users.txt`，其中每一行存储一个用户的名字和密码，二者之间使用冒号分割，例如“`admin:123456`”。

(3) 用户输入名字和密码后，单击“登录”按钮，根据文件 `users.txt` 中存储的信息判断用户输入是否正确。如果不正确就提示“用户名或者密码不正确”，如果正确就提示“登录成功”。请将界面中的文字全部改为中文。

(4) 按照 GUI 程序开发流程，从创建窗口、添加控件、事件函数的编写及绑定几个步骤设计程序的思路，并完成代码，填入表 11-11 中。

表11-11 登录界面

算法思路	代码
## 导入需要的模块	

[illegible]

第 12 章 网络爬虫

12.1 学习目标与要求

- (1) 了解常见的爬虫
- (2) 了解爬虫相关的伦理
- (3) 掌握 selenium 爬虫的使用方法

12.2 知识要点

12.2.1 爬虫介绍

网络爬虫是一种按照一定规则自动地抓取万维网信息的程序或者脚本的计算机程序。爬虫与人工抓取信息区别在于：人工缓慢、抓取信息少，爬虫是大量、快速获取信息。爬虫应用场景非常广泛，如：搜索引擎中的网页抓取、数据抓取、舆情监测等领域。

12.2.2 爬虫伦理

《中华人民共和国刑法》第二百八十五条 非法侵入计算机信息系统罪：违反国家规定，侵入前款规定以外的计算机信息系统或者采用其他技术手段，获取该计算机信息系统中存储、处理或者传输的数据，或者对该计算机信息系统实施非法控制，情节严重的，处三年以下有期徒刑或者拘役，并处或者单处罚金；情节特别严重的，处三年以上七年以下有期徒刑，并处罚金。

因爬虫而被判刑的案件：头条爬虫官司：<https://dwz.cn/qBBHc2fq>，酷米客公司官司：<https://dwz.cn/9IpAOHGB360> 和百度爬虫官司：<https://dwz.cn/RGRa3HJc>，在使用爬虫时需遵守法律法规，守住底线，不做违反国家法律法规的事情。

12.2.3 常见的爬虫

Python 中使用的常见爬虫有：scrapy、requests、selenium 等。表 12-1 为常见的爬虫比较。

表12-1 常见的爬虫

名称	简介	优点	缺点	适用场景
scrapy	功能强大的高级爬虫框架，基于Twisted异步网络框架开发	1. 高效处理大量数据 2. 可扩展性高 3. 支持分布式爬取 4. 自动反爬虫功能	1. 学习曲线较陡 2. 配置复杂	大规模数据爬取、复杂网站结构、需要高度定制化的爬虫项目
requests	简洁且功能丰富的HTTP库，用于发送HTTP请求	1. 简洁直观 2. 功能丰富 3. 易于集成	1. 不支持异步操作 2. 对复杂爬虫任务支持有限	简单的网页数据抓取、HTTP请求处理、静态网页爬取
selenium	一个自动化测试工具，可以模拟浏览器操作	1. 模拟真实浏览器行为 2. 处理动态页面 3. 灵活性高	1. 效率较低 2. 资源消耗大	需要模拟用户操作的网站、动态加载的网页、登录表单提交

12.2.4 selenium

Selenium 是目前主流 Web 浏览器自动化方案。它可以像人一样在浏览器里操作 Web

界面，如：点击界面按钮、在文本框中输入文字等基本操作，进而实现如：抢票、表单自动录入等功能操作。同时还能从 Web 界面获取信息，如：获取票务、招聘网站职位信息，财经网站股票价格信息。

selenium 通过调用浏览器驱动程序提供的接口实现对浏览器的自动化操作，接收浏览器驱动程序返回的结果，对结果进行处理，获取需要的数据。图 12-1 为 selenium 自动化测试工具的操作流程。

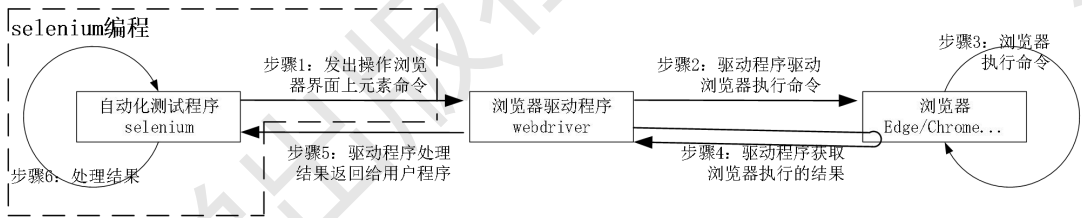


图 12-1 selenium 交互流程

图 12-1 中，实现 Web 自动化操作流程中，仅需完成 selenium 编程部分，即图中的步骤 1、6 部分。

(1) 环境搭建

安装 selenium 库： `pip install selenium -i https://pypi.tuna.tsinghua.edu.cn/simple`

安装浏览器： selenium 支持 Chrome, Edge, Firefox, Safari 等浏览器，安装其中一款浏览器即可。

安装浏览器驱动，分为自动安装和手动安装。手动安装需要去浏览器对应的驱动下载网站下载与浏览器版本对应或相近的驱动。高版本的 selenium 提供了自动下载浏览器驱动的功能。在运行程序时，会自动检查本地 %PATH% 环境变量中是否包含浏览器版本对应的驱动程序，未包含则自动下载。如：在程序中使用 Edge 浏览器，自动下载驱动的代码为：

```
from selenium import webdriver #导入 selenium 库中的 webdriver
wd = webdriver.Edge() #指定使用 Edge，selenium 自动下载 Edge 对应版本的驱动程序
```

(2) 编写程序

selenium 编程部分可分为几步来实现，下面以豆瓣电影 250 排行榜为例，分步骤实现对数据的获取。

步骤 1：导入库

```
import time #程序中延迟使用
from selenium import webdriver #浏览器驱动库
from selenium.webdriver.common.by import By #定位页面元素
```

步骤 2：创建浏览器对象，设置全局等待时间

```
#创建 Edge 对象，保存到 wd 变量中。本语句可自动安装浏览器版本对应的驱动程序。
#驱动程序执行完，会自动打开 Edge 浏览器
wd = webdriver.Edge()
#在定位网页元素时，selenium 会间隔 0.5s 执行 find_element 或 find_elements 函数
#当定位元素时间超过 10s，selenium 会报错、抛出异常
wd.implicitly_wait(10)
```

步骤 3：打开网址

```
weburl = "https://movie.douban.com/top250"
wd.get(weburl)
```

步骤 4：定位元素，获取元素里的具体数据，下面采用 CSS 选择器方式进行定位元素。

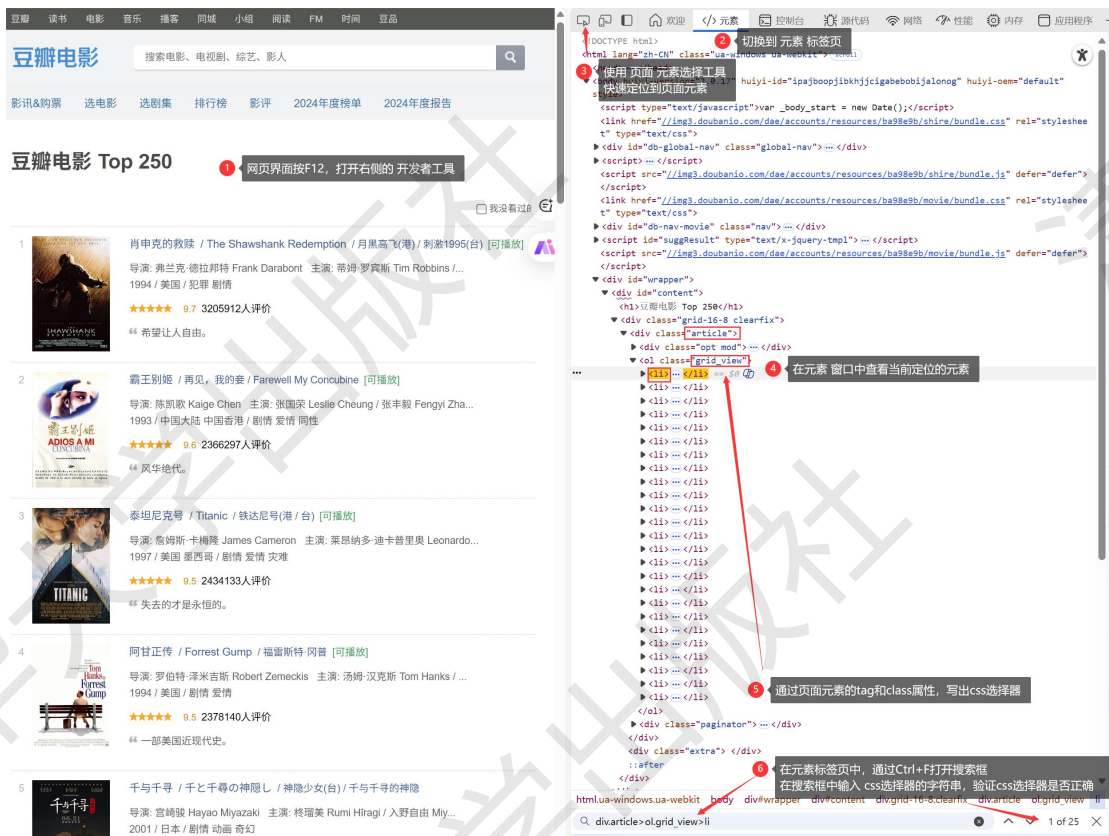


图 12-2 浏览器开发者工具辅助定位元素

定位 25 部电影在页面上的位置

```
elements = wd.find_elements(By.CSS_SELECTOR, "div.article>ol.grid_view>li")
for element in elements: # 遍历每一部电影
    order = element.find_element(By.CSS_SELECTOR, "div.pic>em").text
    name = element.find_element(By.CSS_SELECTOR, "div.info>div.hd>a[href]").text
    info = element.find_element(By.CSS_SELECTOR, "div.info>div.bd>p").text
    tele = element.find_elements(By.CSS_SELECTOR, "div.info>div.bd>div>span")
    score = tele[1].text
    personcount = tele[-1].text[:-3]
    print(order, name, info, score, personcount) #打印输出
```

步骤 5：退出浏览器驱动

```
wd.quit()
```

12.3 实践与指导

实践 1 爬取豆瓣 Top250 电影数据

实践目的

- (1) 掌握 selenium 爬虫的使用
- (2) 理解 time.sleep 函数在数据抓取中的作用
- (3) 掌握 selenium 定位元素的方法

实践要求

(1) 豆瓣 Top250 数据的页面，每个页面展示 25 条数据，爬取 250 条数据需要循环 10 次，通过点击分页按钮，查看 url，可分析出 url 的格式为：

https://movie.douban.com/top250?start=0
https://movie.douban.com/top250?start=25
https://movie.douban.com/top250?start=50
...

图 12-3 为 url 分析，通过改变 url 中 start 参数的数值可获取接下来的 25 条数据。



图 12-3 url 分析

(2) 分析代码功能，并填表 12-2。

表12-2 代码分析

行号	代码	代码功能
#01	import time	
#02	from selenium import webdriver from	
#03	selenium.webdriver.common.by import By	
#04	wd = webdriver.Edge()	
#05	wd.implicitly_wait(10)	
#06	movies = list()	
#07	weburl = "https://movie.douban.com/top250"	
#08	for i in range(10):	

#09	wd.get(weburl + "?start=" + str(i*25))	
#10	time.sleep(1)	# 等待1s，等页面加载完毕，再去定位元素 # 也可有效降低网站的防爬
#11	elements = wd.find_elements(By.CSS_SELECTOR, "div.article>ol.grid_view>li")	
#12	for element in elements:	
#13	order = element.find_element(By.CSS_SELECTOR, "div.pic>em").text	
#14	name = element.find_element(By.CSS_SELECTOR, "div.info>div.hd>a[href]").text	
#15	tele = element.find_elements(By.CSS_SELECTOR, "div.info>div.bd>div>span")	#获取电影评分、评分人数等信息
#16		#获取电影评分
#17	personcount = tele[-1].text[:-3]	
#18	movies.append([order, name, info, score, personcount])	
#19	wd.quit()	

实践 2 爬取苏州空气质量数据

实践目的

- (1) 掌握 selenium 库的使用
- (2) 掌握 CSS_SELECTOR 选择器定位元素及子元素的方法
- (3) 掌握文件基本操作方法

实践要求

(1) 空气质量 AQI 数据可通过 www.tianqihoubao.com 网站获取，获取苏州地区 AQI 数据的网址为：

2020 年 1 月份数据网址：<https://www.tianqihoubao.com/aqi/suzhou-202001.html>

2021 年 12 月份数据网址：<https://www.tianqihoubao.com/aqi/suzhou-202112.html>

页面每次列出查询月份的 AQI 指数相关数据，请分析网页 url，以及页面数据规律，采用 selenium 操作网页，获取 2014 年 1 月到 2025 年 8 月的数据。

- (2) 将获取到的数据保存到文件中，以便后面数据分析时使用。
- (3) 根据表 12-3 中的思路，编写程序。

表12-3 苏州AQI数据获取

行号	代码功能	代码
#01	## 导入time库	
#02	## 导入selenium库的webdriver模块	

#03	## 导入selenium库中的By类	
#04	## 创建浏览器操控对象	
#05	## 设置全局等待实践为10秒	
#06	## 定义变量year、month，初始化为2014、1	
#07	## 定义空列表aqi_suzhou变量，用于存储苏州aqi数据	
#08	## 向aqi_suzhou列表中添加第一个元素，元素类型为列表。该元素主要是爬取数据的标题	
#09	## 循环年份，从2014年到2025年	
#10	## 根据年份，对结束月份endmonth进行赋值 如果年份为2025，endmonth值为9，否则值为13	
#11	## 子循环，遍历月份	
#12	## 根据url变化规律，构造weburl	
#13	## 用selenium打开weburl	
#14	## 睡眠1s，等待网页请求响应回来	
#15	## 查找AQI数据的所有tr	trs = wd.find_elements(By.CSS_SELECTOR, "#content table>tbody>tr")
#16	## 首个tr除外，遍历所有tr	
#17	## 定义aqi_day空列表，用于存储每天的AQI数据	
#18	## 查找当前tr元素下面的所有td	tds = tr.find_elements(By.TAG_NAME, "td")
#19	## 遍历所有的td元素	
#20	## 将td标签中的文本添加到aqi_day列表中	
#21	## 将aqi_day以子元素形式，添加到aqi_suzhou列表中	
#22	## 将aqi_suzhou列表中数据保存到aqi_suzhou.txt文件中	

实践 3 爬取成绩数据

实践目的

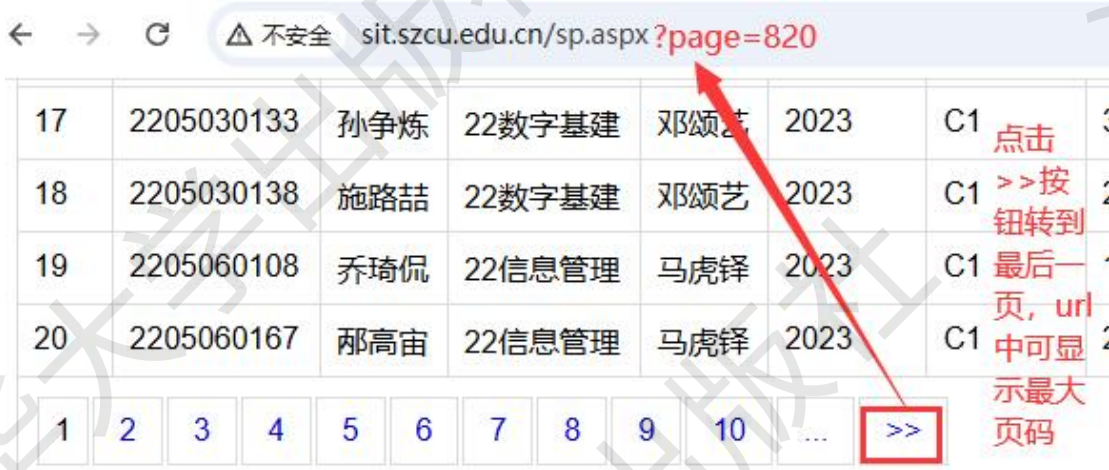
- (1) 掌握 selenium 库的使用
- (2) 掌握 CSS_SELECTOR、Tag 等选择器定位元素及子元素的方法
- (3) 掌握文件基本操作方法

实践要求

(1) 学生成绩数据:

成绩网址为 (仅限校内使用): `http://sit.szcu.edu.cn/sp.aspx`

通过分页导航的按钮 “>>”, 切换到最后一页, 在地址栏中, 可查看到最大页码的数值为 820, url 的格式为: `http://sit.szcu.edu.cn/sp.aspx?page=820`; 可通过循环方式改变 page 参数的值来获取全部数据。



(2) 将获取到的数据保存到文件 `score.csv` 中, 以便后面数据分析时使用。

(3) 在表 12-4 中先写思路, 然后编写程序。

表12-4 成绩数据获取

行号	代码功能	代码
#01	## 导入time库	
#02	## 导入selenium库的webdriver模块	
#03	## 导入selenium库中的By类	
#04	## 创建浏览器操控对象	
#05	## 设置全局等待实践为10秒	
#06		
#07		
#08		
#09		
#10		
#11		
#12		
#13		

#14	
#15	
#16	
#17	
#18	
#19	
#20	
#21	
#22	
#23	
#24	
#25	
#26	

第 13 章 数据分析与可视化

13.1 学习目标与要求

- (1) 了解数据可视化流程
- (2) 掌握 numpy 库的使用
- (3) 掌握 pandas 库的使用
- (4) 掌握 matplotlib 库的使用

13.2 知识要点

13.2.1 数据分析与可视化介绍

数据分析是指用适当的统计分析方法，对收集的数据进行处理与分析，提取出一些有价值的信息形成结论，从而对数据加以详细研究和概括总结的过程。

数据挖掘是指对大量的、有噪声的和随机的实际应用数据，通过应用聚类、分类、回归和关联规则等技术，得到一些未知的、有价值的信息，挖掘潜在价值的过程。数据挖掘过程主要有：

1. 获取数据：通过爬虫采集、或下载一些统计网站发布的数据
2. 数据探索：了解数据特征和关系，以便更好地理解、使用数据。
3. 数据预处理：数据清洗、数据集成、数据变换、数据规约。
4. 挖掘建模：分类、聚类、关联、预测。
5. 模型评价与发布。

数据分析与数据挖掘密不可分，数据挖掘通常是数据分析的提升。在数据挖掘的数据探索和预处理阶段通常使用的就是数据分析技术。

数据可视化是数据分析的关键技术之一，它是以图形化方式准确、清晰、有效地传达数据中所包含的信息。

13.2.2 数据分析

数据可视化是数据分析的关键技术之一，它是以图形化方式准确、清晰、有效地传达数据中所包含的信息。

(1) Numpy

NumPy (Numerical Python)是 Python 中科学计算的基础包，提供多维数组对象、各种派生对象以及用于对数组进行快速操作的各种方法，包括数学、逻辑、形状操作、排序、选择、I/O、离散傅里叶变换、基本线性代数、统计运算、随机模拟等。表 13-1 为 numpy 的常用操作。

表13-1 numpy常用操作

功能	语法形式	示例	说明
创建数组	<code>np.array(object, dtype, ndmin)</code>	<code>arr = np.array([1, 2, 3, 4, 5], dtype=np.int32)</code>	ndmin：最小维数，一般不设置
生成矩阵，随机数填充	<code>arr = np.random.randn(M, N)</code>	<code>arr = np.random.randn(3, 5)</code>	

生成全0矩阵	<code>arr = np.zeros(shape=(M, N), dtype=datatype)</code>	<code>arr = np.zeros(shape=(3,5), dtype=np.int16)</code>	默认dtype为float64
生成全1矩阵	<code>arr = np.ones(shape=(M, N), dtype=datatype)</code>	<code>arr = np.ones (shape=(3,5), dtype=np.int16)</code>	默认dtype为float64
生成N×N单位矩阵	<code>arr = np.eye(N, dtype=datatype)</code>	<code>arr = np.eye(10, dtype=np.int16)</code>	默认dtype为float64

矩阵的常见操作有：加法、减法、乘法、哈达玛乘积等操作，表 13-2 为矩阵常见的运算。

表13-2 numpy矩阵常见操作

功能	语法形式	示例	说明
矩阵加法	<code>A+B</code>	<code>A, B = np.eye(3), np.random.randn(3,3)</code> <code>C = A+B</code> <code>D = A+3</code>	$C_{ij} = A_{ij} + B_{ij}$ $D_{ij} = A_{ij} + 3$
矩阵减法	<code>A-B</code>	<code>C = A-B</code> <code>D = A-3</code>	$C_{ij} = A_{ij} - B_{ij}$ $D_{ij} = A_{ij} - 3$
矩阵乘法	<code>A@B</code> 或者 <code>np.matmul(A, B)</code>	<code>C = A@B</code>	A矩阵 $M \times P$ B矩阵 $P \times N$ C矩阵为 $M \times N$ $C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}$
矩阵哈达玛乘积	<code>A*B</code> 或者 <code>np.multiply(A, B)</code>	<code>C = A*B</code>	A、B矩阵尺寸一致 $C_{ij} = A_{ij} * B_{ij}$

(2) Pandas

Pandas 基于 NumPy 的数据分析模块，提供大量标准模型和高效操作大型数据集的工具。Pandas 提供三种数据类型，分别是序列 Series、数据帧 DataFrame、多级索引 MultiIndex。序列 Series 为一维数据结构。数据帧 DataFrame 为二维表格型数据结构，由有序的列组成，每列可以是不同的数据类型，既有行索引也有列索引，可看成由多个序列 Series 组成的字典。表 13-3 为 DataFrame 常见的操作。

表13-3 pandas的DataFrame常用操作

功能	语法形式	示例	说明
创建对象	<code>df = pd. DataFrame(data, columns, index)</code>	<code>data = { 'Name':['Tom', 'Jack', 'Steve'], 'Age':[28,34,29]}</code> <code>df = pd.DataFrame(data, columns=['Age','Name'], index=[1,2,3,4])</code>	参数 columns 省略为 0,1,2... 参数 index 省略为 0,1,2..
访问行	<code>df[start:end:step]</code> 或者 <code>df.loc[start:end:step]</code> 或者 <code>df.iloc[start:end:step]</code>	<code>df[1:2]</code> #访问第1行数据 <code>df.loc[1:2]</code> #显式索引访问 <code>df.iloc[0:1]</code> #隐式索引访问	切片 每个df都存在一个从0开始依次增大的隐式行索引
访问列	<code>df['列名']</code> 或者 <code>df[['列1名', '列2名', ...]]</code> 或者 <code>df.列名</code> 或者 <code>df[:, 1:3]</code>	<code>df['Name']</code> #返回Series对象 <code>df[['Name', 'Age']]</code> # 返回 DataFrame对象 <code>df.Name</code> #返回Series对象	

访问单个元素	df.at[行索引, 列标签] df.iat[行索引, 列索引]	df.at(1, "Name") df.iat[1, 1]	iat函数使用的行、列索引均为隐式索引
增加一行	pd.concat([df1, pd.DataFrame(x)])	df = pd.concat([df, pd.DataFrame([df.loc[52], index=[len(df)])])	pd.concat() 返回新的 DataFrame 对象
增加一列	df[列名] = data	df['Gender'] = ['M', 'M', 'F']	
删除一行	df.drop(index[, inplace=True])	df.drop(3, inplace=True)	inplace 默认值为 False
删除一列	df.drop(列名, axis=1[, inplace=True])	df.drop("Age", axis=1, inplace=True)	
数据填充	df[列名].fillna(value) 或者 df.fillna(value[, inplace=False])	df['Age'] = df['Age'].fillna(value=22) df.fillna(value=22, inplace=True)	
数据统计	df.count() 或者 df[列名].count()	df.count() df["Age"].count()	按列统计非空数据数 按指定列统计非空数据数
csv 文件读	df.read_csv(file)	df.read_csv("data/scores.csv")	
csv 文件写	df.to_csv(file)	df.to_csv("data/scores.csv")	

下面以豆瓣 Top250 数据的处理为例。

步骤 1: 将 13.1 小节的实践 1 最终获取的 movies 列表转换为 DataFrame 对象。

```
import pandas as pd
df = pd.DataFrame(movies, columns=["排名", "电影名称", "内容", "评分", "评分人数"])
```

步骤 2: 数据特征查看

```
# 查看前几行数据
df.head()
# 查看后几行数据
df.tail()
# 数据描述
df.describe()
# 查看数据量
df.shape
```

步骤 3: 非空查询

```
# 查询空值
df.isna().sum()
```

步骤 4: 将“内容”列拆分为“导演主演”、“其他”两列

①查询“内容”列中，不包含“...\n”字符串的

```
# 1. 查找“内容”列不包括“...\n”字符串的数据, contains 函数默认使用正则表达式
# 在正则表达式中 . 表示任意一个字符
df[~df["内容"].str.contains("...\n", regex=False)]
```

②将“内容”列中的“...\n”替换为“\n”

```
df["内容"] = df["内容"].str.replace("...\n", "\n")
```

③将“内容”列以“\n”字符进行拆分为“导演主演”、“其他”两列

```
df[["导演主演", "其他"]] = df["内容"].str.split("\n", expand=True)
```

④删除“内容”列

```
df.drop("内容", axis=1, inplace=True)
```

步骤 5: 将“导演主演”列拆分为“导演”、“主演”两列

①将“导演主演”列以“主演:”字符进行拆分为“导演”、“主演”两列

```
df[["导演","主演"]] = df["导演主演"].str.split("主演:", expand=True)
```

②删除“导演主演”列

```
df.drop("导演主演", axis=1, inplace=True)
```

③将“导演”列中的“导演:”去除

```
df["导演主演"] = df["导演主演"].str.replace("导演:", "")
```

步骤6: 将“其他”列拆分为“年份”、“地点”、“类别”三列

①查看“其他”列中“/”个数的数量不等于2的数据

```
df[df["其他"].str.count("/") != 2]
```

查询结果为: index 为: 52、106、185, 原因时这三部电影有多个名称, 上映年份不一样, 豆瓣将其归并为一部电影进行处理, 处理方法有两种: 一种直接删除, 另一种将其拆分为多部电影, 这里采用后者。

②为①中的数据插入几行

```
df = pd.concat([df, pd.DataFrame([df.loc[52], index=[len(df)])])
df = pd.concat([df, pd.DataFrame([df.loc[52], index=[len(df)])])
df = pd.concat([df, pd.DataFrame([df.loc[106], index=[len(df)])])
df = pd.concat([df, pd.DataFrame([df.loc[185], index=[len(df)])])
```

③对新插入及原数据的“电影名称”、“其他”两列数据进行处理

```
df.at[52, "电影名称"] = '大闹天宫 / The Monkey King'
df.at[52, "其他"] = '1961 / 中国大陆 / 剧情 动画 奇幻 古装'
df.at[250, "电影名称"] = '大闹天宫 上集 / The Monkey King'
df.at[250, "其他"] = '1964 / 中国大陆 / 剧情 动画 奇幻 古装'
df.at[251, "电影名称"] = '大闹天宫 下集 / The Monkey King'
df.at[251, "其他"] = '1978 / 中国大陆 / 剧情 动画 奇幻 古装'
df.at[106, "电影名称"] = '天书奇谭 / The Legend of Sealed Book'
df.at[106, "其他"] = '1983 / 中国大陆 / 动画 奇幻'
df.at[252, "电影名称"] = '天书奇谭 4K 纪念版 / The Legend of Sealed Book'
df.at[252, "其他"] = '2021 / 中国大陆 / 动画 奇幻'
df.at[185, "电影名称"] = '高山下的花环 / Wreaths at the Foot of the Mountain'
df.at[185, "其他"] = '1983 / 中国大陆 / 动画 奇幻'
df.at[253, "电影名称"] = '卫国军魂(港) / Wreaths at the Foot of the Mountain'
df.at[253, "其他"] = '2021 / 中国大陆 / 动画 奇幻'
```

④将“其他”列拆分为“年份”、“地点”、“类别”三列

```
df[["年份","地点","类别"]] = df["其他"].str.split("/", expand=True)
```

步骤7: 更改列数据类型

①查看各列数据类型

```
df.dtypes
```

②去除“年份”列首尾空格

```
df["年份"] = df["年份"].str.strip()
```

③查看“年份”列, 长度超过4的数据

```
df[df["年份"].str.len()>4]
```

查询结果为: index 为: 113 的“年份”值为“1982(中国大陆)”

④修改③中数据

```
df.at[113, "年份"] = "1984"
```

⑤更改“排名”、“评分”、“评分人数”、“年份”列的数据类型

```
df["排名"] = df["排名"].astype("int16")
df["评分"] = df["评分"].astype("float64")
df["评分人数"] = df["评分人数"].astype("int64")
df["年份"] = df["年份"].astype("int16")
```

(3) matplotlib

数据可视化是分析数据的重要环节，通过图像能更加了解数据的含义。Matplotlib 是 Python 中一个常用的、用于绘图的工具，可以绘制折线图、直方图、饼图、散点图等。其应用最广的是 matplotlib.pyplot 模块。表 13-4 为 matplotlib.pyplot 模块常用方法。

表13-4 matplotlib.pyplot常用方法

方法	功能	方法	功能
plot()	折线图	bar()	柱状图
hist()	直方图	barh()	条形图
pie()	饼图	scatter()	散点图
boxplot()	箱型图	stackplot()	堆叠图
show()	显示	title()	设置标题
xlabel()	设置x轴标题	ylabel()	设置y轴标题
legend()	添加图例		

下面以豆瓣 Top250 数据的处理为例。

针对本小节（2）Pandas 中数据处理结果进行绘制图表。

步骤 1：导入库

```
import matplotlib.pyplot as plt
from matplotlib import rcParams
# 设置字体 微软雅黑
rcParams['font.sans-serif'] = 'Microsoft YaHei'
```

步骤 2：对数据按照“评分”进行分组，按照“排名”进行计数

```
score_rank = df.groupby(["评分"])[["排名"].count()
```

步骤 3：创建图表、设置大小

```
plt.figure(figsize=(10,5))
```

步骤 4：绘制柱形图

```
plt.bar(score_rank.index, score_rank, label="豆瓣评分数量分布")
# 显示图例
plt.legend()
# x/y 轴标题
plt.xlabel("分数")
plt.ylabel("电影数量")
```

步骤 5：绘制折线图

```
plt.plot(score_rank.index, score_rank, linestyle='-', color='red', marker='o')
```

步骤 6：自动优化排版、显示图表，图 13-1 为豆瓣评分分布图

```
plt.tight_layout()
plt.show()
```

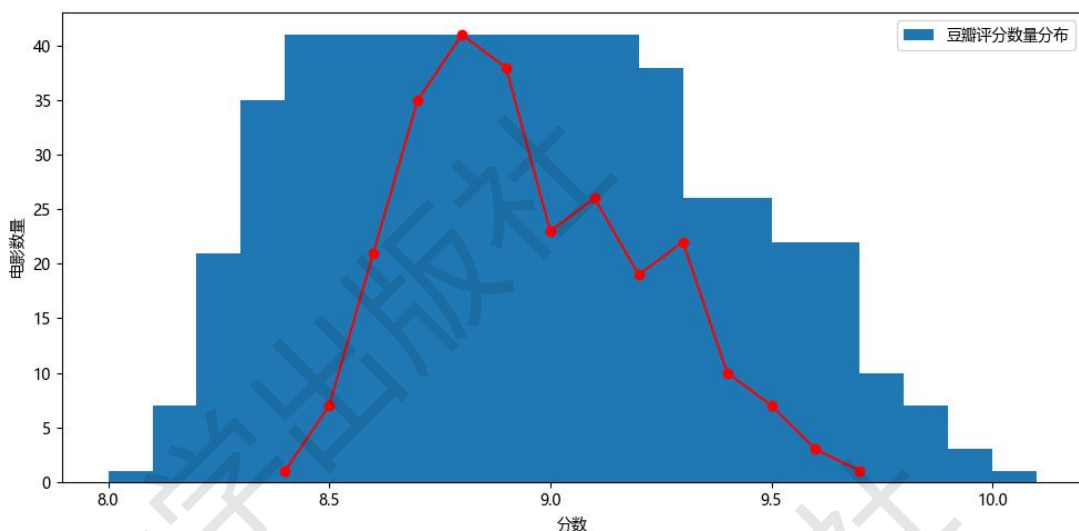


图 13-1 豆瓣评分分布图

13.3 实践与指导

实践 1 苏州空气质量数据分析

实践目的

- (1) pandas 的 dataframe 常见的操作
- (2) matplotlib 图表的绘制

实践要求

(1) 对 12.3 小节中的实践 2，爬取下来的数据进行进一步处理，包括：数据清洗、数据可视化。

(2) 分析代码功能，结合思路，完善代码。填表 13-5，下表的行号仅作参考，不作为具体的行号。

表13-5 苏州空气质量分析

行号	代码	代码功能
#01	import pandas as pd	导入pandas库，为其起别名pd
#02	import matplotlib.pyplot as plt	导入matplotlib库的pyplot模块，为其起别名plt
#03	from matplotlib import rcParams	导入rcParams
#04	df = pd.DataFrame(_____)	将 aqi_suzhou 列表转换为DataFrame对象df 标题为aqi_suzhou[0]
#05	datatypes = { '日期': 'datetime64[ns]', 'AQI指数': 'int64', '质量等级': 'category', '当天AQI排名': 'int64', 'PM2.5': 'int64', 'PM10': 'int64', 'No2': 'int64', 'So2': 'int64', 'Co': 'float64', 'O3': 'int64'} df = df.astype(_____)	更新数据类型

	df.head(3)	查看数据前3行
#06	_____	查看数据后3行
	_____	查看数据形状
	_____	查看数据描述
#07	year_aqi_q = df.groupby(df["日期"].dt.to_period('Q')) [_____]._____	按年份、季度 求AQI指数平均值
#08	rcParams[_____] = _____	设置绘图的字体系为 微软雅黑
#09		创建图表，设置图表大小为 10, 5
#10	plt.plot(year_aqi_q.index[0::4].strftime("%YQ%q").str[:4], year_aqi_q[0::4], label="1季度AQI指数")	绘制1季度AQI指数的折线图，x轴为年份，y轴为AQI指数值
#11		绘制2季度AQI指数的折线图，x轴为年份，y轴为AQI指数值
#12		绘制3季度AQI指数的折线图，x轴为年份，y轴为AQI指数值
#13		绘制4季度AQI指数的折线图，x轴为年份，y轴为AQI指数值
#14		显示图例
#15	plt.title("苏州空气质量AQI指数变化图")	添加标题
#16		自动优化布局
#17		显示图表

图 13-2 为上述代码生成的图表，从中可看出每年的 3 季度空气质量最好，自 2014 年以来，空气质量逐年得到好转。

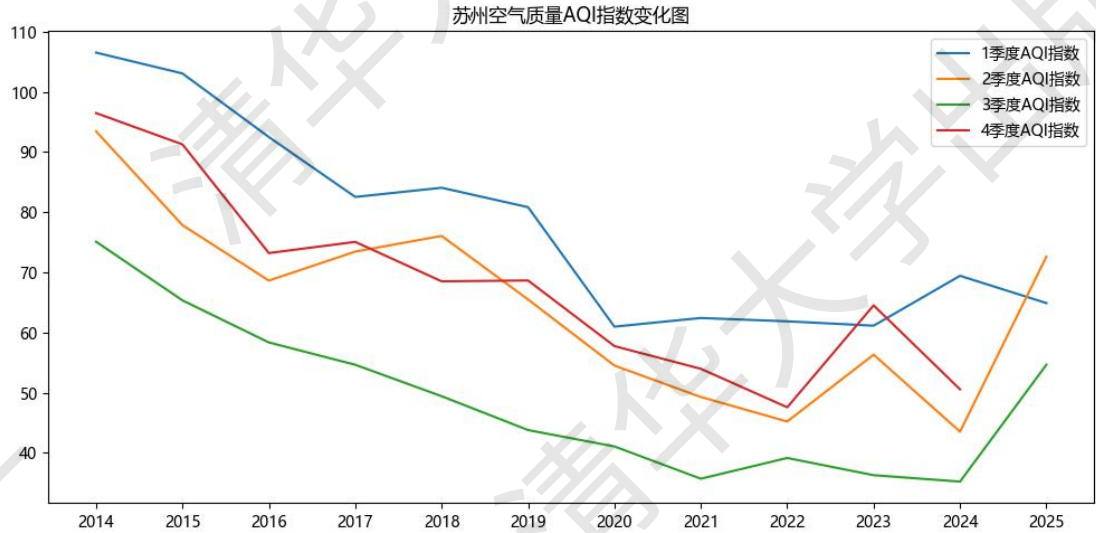


图 13-2 苏州市空气质量指数变化图

实践 2 成绩数据分析——阶段性成绩对比柱状图

实践目的

- (1) pandas 的 dataframe 常见的操作
- (2) matplotlib 图表的绘制

实践要求

(1) 对 12.3 小节中的实践 3，爬取下来的数据进行进一步处理，包括：数据清洗、数据可视化。

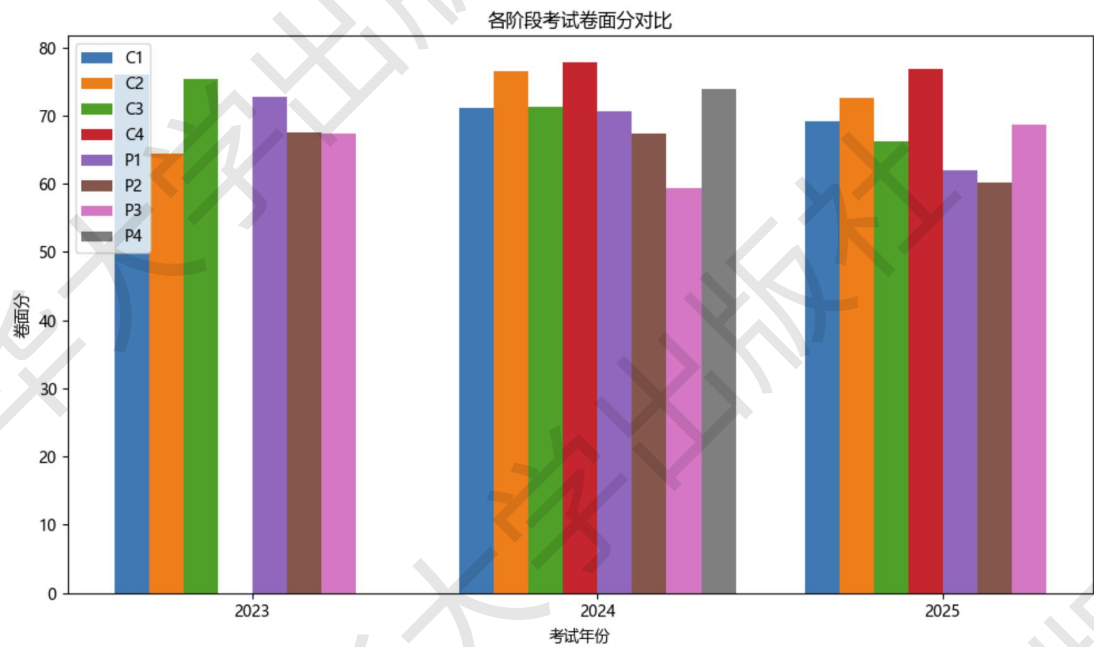


图 13-3 各阶段成绩对比图

(2) 分析代码功能，根据图 13-3，结合思路，将表 13-6 补充完整。下表的行号仅作参考，不作为具体的行号。

(3) 验证表 13-6 中的代码，是否能够绘制出图 13-3。

表13-6 学生成绩分析

行号	代码功能	代码
#01	导入库	
#02	加载数据	score_df = pd.read_csv("data/score.csv", index_col=0)
#03	查看数据相关信息	
#04	查看是否有空置	
#05	查看数据类型，分别设置“考试名称”、“开考时间”、“交卷时间”数据类型为：category、datetime64[ns]	

#06	根据考试名称、考试年份进行分组，对卷面分进行求均值	
#07	重塑数据	<code>df1 = score_year_mean.reset_index(name='卷面分') .pivot(index='考试年份', columns='考试名称', values='卷面分')</code>
#08	设置绘制图表的字体为 Microsoft YaHei	
#09	设置创建图表的大小为 10,6	
#10	每组柱形图的数量	<code>n_exams = len(df1.columns)</code>
#11	每个柱形的宽度	<code>bar_width = 0.8 / n_exams</code>
#12	获取年份	<code>years = df1.index.astype(int)</code>
#13	每个年份的位置	<code>import numpy as np x = np.arange(len(years))</code>
#14	绘制每个考试名称的柱形	<code>for i, exam in enumerate(df1.columns): plt.bar(x + i * bar_width, df1[exam], width=bar_width, label=exam)</code>
#15	设置x轴刻度位置为每组柱形的中心	<code>plt.xticks(x + bar_width * (n_exams - 1) / 2, years)</code>
#16	设置图表标题和轴标签	
#17	显示图表	

实践 3 成绩数据分析——时长和卷面分关系的散点图绘制

实践目的

- (1) pandas 的 dataframe 常见的操作
- (2) matplotlib 图表的绘制

实践要求

(1) 对 12.3 小节中的实践 3，爬取下来的数据进行进一步处理，包括：数据清洗、数据可视化。

(2) 分析代码功能，根据图 13-4，结合思路，将表 13-7 补充完整。表 13-7 的行号仅作参考，不作为具体的行号。

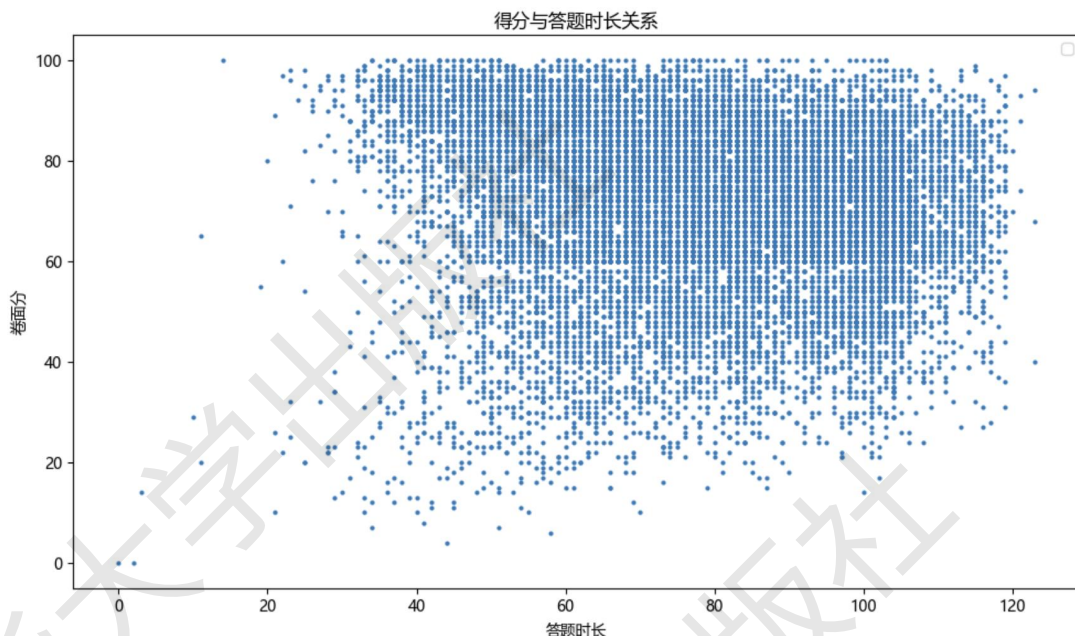


图 13-4 得分与答题时长关系散点图

(3) 验证表 13-7 中的代码，是否能够绘制出图 13-4。

表13-7 学生成绩分析

行号	代码功能	代码
#01	导入库	
#02	加载数据	<code>score_df = pd.read_csv("data/score.csv", index_col=0)</code>
#03	查看数据相关信息	
#04	查看是否有空置	
#05	查看数据类型，分别设置“考试名称”、“开考时间”、“交卷时间”数据类型为: <code>category</code> 、 <code>datetime64[ns]</code>	
#06	根据 开考时间、交卷时间，计算 答卷时长，作为新的一列“考试时长”插入到 <code>score_df</code>	
#07	更新C1、C3 考试时长，将其调整与C2、C4时长一致 C1、C3 为90分钟 C2、C4为120分钟	<code>c_condition = ((score_df['考试名称'].isin(['C1', 'C3'])))</code> <code>score_df.loc[c_condition, "考试时长"] *= 4/3</code>
#10	更新2023、2024年的P1、P3 考试时长，将其调整与P2、P4时长一致 P1、P3 为90分钟 P2、P4为120分钟	
#11	根据 考试时长 列，增加 考试分钟 一列，该列为 考试时长列 向下取整，类型为 <code>int64</code>	<code>score_df["考试分钟"] = np.floor(score_df["考试时长"]</code> <code>.astype('timedelta64[s']).astype(int) // 60)</code>
#12		

#13	
#14	
#15	
#16	
#17	
#18	
#19 设置图表标题和轴标签	
#20 显示图表	

实践 4 证券指数数据分析

实践目的

- (1) pandas 的 dataframe 常见的操作
- (2) matplotlib 图表的绘制
- (3) 了解证券交易数据获取方式

实践要求

- (1) 通过 akshare 获取上证综合指数、深圳综合指数，并合并成一个 dataframe。

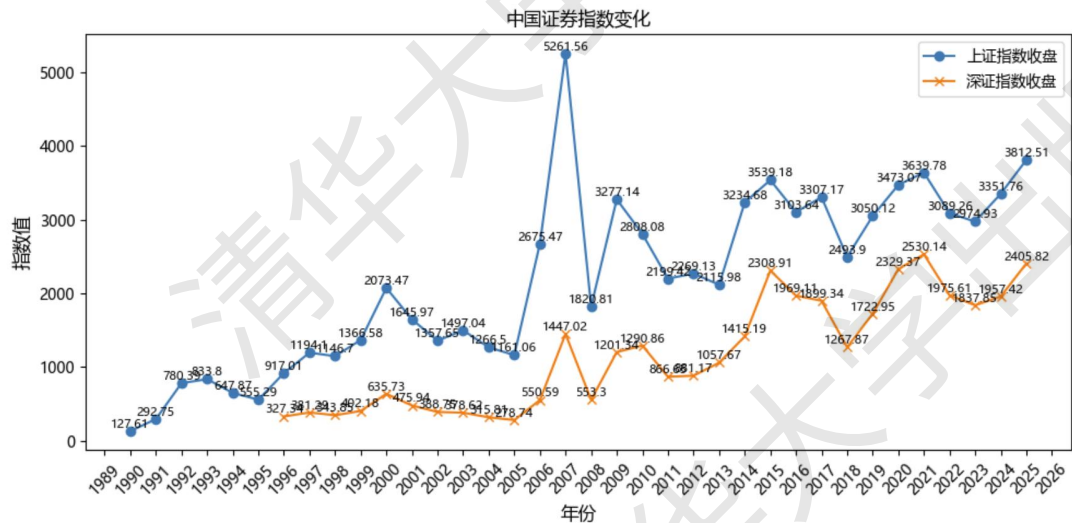


图 13-5 证券指数变化

- (2) 根据图 13-5，结合表 13-8 中的思路。

表13-8 证券指数分析粗略思路

步骤	实现的功能	功能分解
1	数据获取	安装akshare: https://akshare.akfamily.xyz/installation.html 官网查询指数获取方法: https://akshare.akfamily.xyz/data/index/index.html

	分别获取上证指数(sh000001)和深圳指数(sz399106) 对两个dataframe对象添加一列：“交易所” 合并两个dataframe对象为df
2 数据清洗	查看df的数据情况
3 数据可视化	以年份为横坐标，以指数值为纵坐标，分别绘制上证指数、深圳指数折线图例、横坐标数值倾斜45°，显示数据点上的数值、数据点分别用圆点和叉号表示

（3）根据表 13-8，细化实现的步骤，将其填入到表 13-9 中，表中的行号仅作参考，不作为具体的行号。根据构思，将其转为代码。

（4）将代码输入到文件中，执行并验证是否正确。注意：本例中的数据获取最大的日期为 2025 年 9 月 5 日。

表13-9 证券指数分析精细思路及代码

行号	功能	代码
#01	导入库	
#02	通过akshare的stock_zh_index_daily_em接口获取上证指数数据，保存到sh_zs_df中	
#03	通过akshare的stock_zh_index_daily_em接口获取深圳指数数据，保存到sz_zs_df中	
#04	分别sh_zs_df、sz_zs_df添加“交易所”列，分别赋值为“SH”、“SZ” 设置该列数据类型为：category	
#05	合并sh_zs_df、sz_zs_df为df	
#06	设置“date”列数据类型为datetime64[ns]	
#07	查看各列是否有空值	
#08	查看数据描述、前5行、后5行、df形状	
#09	通过“date”列，新增一列“年份”	
#10	按“交易所”、“年份”升序，“date”降序对df进行排序，排序后保存到df_sorted	df_sorted = df.sort_values(['交易所', '年份', 'date'], ascending=[True, True, False])
#11	对df_sorted去重，每个年份仅保留一行（即当年交易的最后一天），去重后存储到result_df中	result = df_sorted.drop_duplicates(['交易所', '年份']).reset_index(drop=True)
#12	绘图：	
#13		
#14		
#15		
#16		

#17	
#18	
#19	
#20	
#21	
#22	
#23	
#24	

第 14 章 综合案例

本章将介绍苏州市二手房线上数据处理过程，处理过程包括：数据获取、数据清洗、数据可视化等操作。

14.1 学习目标与要求

- (1) 掌握 selenium 进行网络数据的抓取
- (2) 掌握 pandas 数据清洗、数据处理方法
- (3) 掌握 matplotlib 库用于数据图形化的方法
- (4) 本案例为苏州二手房线上数据处理的过程，涉及到数据的获取、数据清洗、可视化操作等。

本案例完成后，图 14-1 为本案例完成后的效果图。

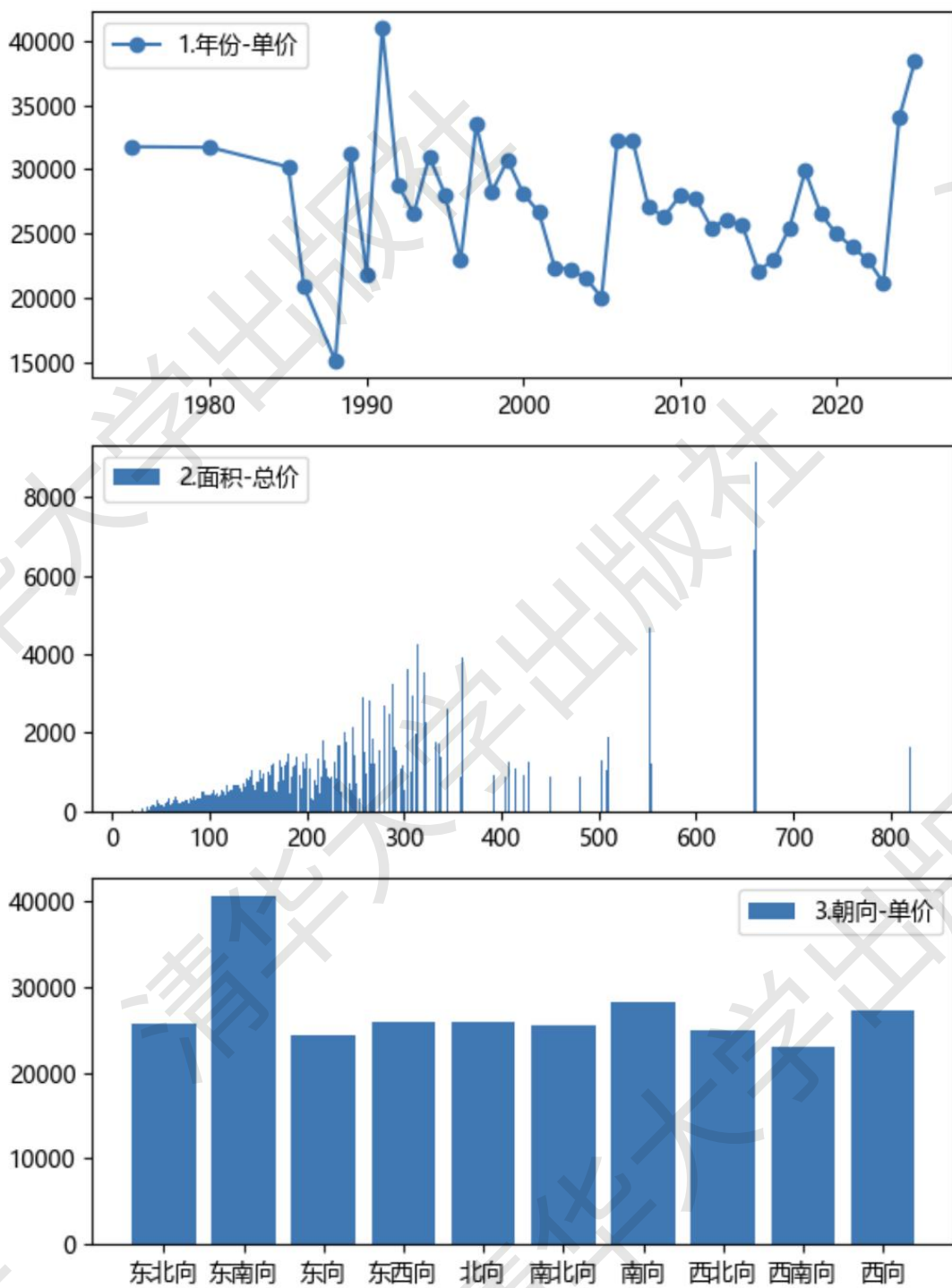


图 14-1 苏州市二手房线上数据分析图

14.2 知识点

14.2.1 数据获取

(1) 环境搭建

安装 selenium 库: `pip install selenium -i https://pypi.tuna.tsinghua.edu.cn/simple`
-i 参数用于指定安装源。

(2) 页面分析

本案例通过房天下获取二手房数据。

步骤 1: 打开网站。打开苏州二手房交易网站 <https://suzhou.esf.fang.com>。

步骤 2: url 分析。图 14-2 为二手房页面元素的分析, 从中可看出 url 的变化规律以及页面元素的定位。点击页面底端的分页按钮“2”、“3”、...、“下一页”、“尾页”等按钮, 查看 url 的变化规律, 结合“共 100 页”, 可分析出苏州二手房线上数据的 url 为: <https://suzhou.esf.fang.com/house/i3{i}>, 其中{i}的值从 1 变化到 100。



图 14-2 二手房页面元素分析

(3) 数据爬取

步骤 1: 对爬取的数据部分进行分析。打开 <https://suzhou.esf.fang.com/house/i31> 页面。按 F12 打开“开发者工具箱”, 对页面元素选中后, 查看元素页面的信息。按下 Ctrl+F 快捷键, 打开搜索框, 根据对元素的分析, 在搜索框中输入“`div.main945.floatl > div.shop_list.shop_list_4 > dl`”, 查看能否搜索出 60 个元素, 通过切换元素次序, 来查看页面上的元素选择情况。图 14-2 为二手房页面元素分析的过程。

步骤 2: 爬取数据。

导入库

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
创建浏览器对象并设置全局等待时长
wd = webdriver.Edge()
wd.implicitly_wait(10)
爬取的数据存储
houses_list = list()
```

```
houses_list.append(
    ["标题", "户型", "面积", "楼层", "朝向", "年份", "中介", "小区", "地址", "卖点", "总价", "单价"])
循环爬取 100 页数据，并处理
```

```
for i in range(1, 101):
    weblink = f"https://suzhou.esf.fang.com/house/i3{i}"
    wd.get(weblink)
    dls = wd.find_elements(By.CSS_SELECTOR, "div.main945.floatl > div.shop_list.shop_list_4 > dl")
    time.sleep(1)
    for dl in dls:
        house = list() #每个房子信息
        # [0]为房屋基本信息 [1]为房屋价格
        dd0, dd1 = dl.find_elements(By.TAG_NAME, "dd")
        # 获取基本信息
        # 标题
        house.append( dd0.find_element(By.TAG_NAME, "h4").text )
        # [0]房型、面积、楼层、方向、建造年份 [1]小区名、地址 [2]卖点
        p0, p1, p2 = dd0.find_elements(By.TAG_NAME, "p")
        house.extend(p0.text.split("|"))
        house.extend(p1.text.split("\n"))
        house.append(p2.text)
        # 获取价格
        house.extend(dd1.text.split("\n"))
        houses_list.append(house)
```

退出

```
wd.quit()
```

14.2.2 数据清洗

(1) 环境搭建

安装 numpy 库: `pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple`

安装 pandas 库: `pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple`

(2) 数据转换

步骤 1: 导入 pandas 库。

```
import pandas as pd
```

步骤 2: 将 houses_list 列表转换为 DataFrame 格式, houses_list[0] 为标题行

```
df = pd.DataFrame(houses_list[1:], columns=houses_list[0])
```

(3) 数据清洗

步骤 1: 粗洗。去除各列数据首尾空格、将数据中包含换行符去掉。

```
for col_name in df.columns:
    df[col_name] = df[col_name].str.strip()
    df[col_name] = df[col_name].str.replace("\n", "")
```

步骤 2: 查看数据信息

查看数据形状

```
df.shape
```

查看数据前几行、后几行

```
df.head(3)
```

```
df.tail(3)
```

查看数据类型

```
df.dtypes
```

步骤 2: 处理空值

查看空值

```
df.isna().sum()
```

分析空值产生原因。显示空值数据，并分析。

```
condition = (df["卖点"].isna()) | (df["单价"].isna())  
df[condition]
```

从 NaN 查看结果来看，空值因数据收集产生列数据偏差导致。

更新 单价为 NaN 的值为 总价、总价为卖点、卖点为地址、地址为小区、小区为中介、中介为年份，年份为“-”。也可以将这部分数据直接删除掉。

待更新列的名称

```
update_col = ["单价", "总价", "卖点", "地址", "小区", "中介", "年份"]
```

条件

```
condition = df["单价"].isna()
```

更新

```
for i in range(len(update_col)-1):
```

```
    df.loc[condition, update_col[i]] = df.loc[condition, update_col[i+1]]
```

```
df.loc[condition, "年份"] = "-"
```

处理单价、总价

将单价中的单位去除

```
df["单价"] = df["单价"].str.replace("元/m²", "")
```

更改数据类型

```
df["单价"] = df["单价"].astype("int64")
```

将总价中 单位 去除

```
df["总价"] = df["总价"].str.replace("万", "")
```

更改数据类型

```
df["总价"] = df["总价"].astype("float64")
```

处理卖点

填充卖点为空的数据(主要是 卖点)

```
df.fillna("-")
```

处理年份

筛选条件

```
condition = df["年份"].str.contains("年")
```

只保留 包含“年”的行

```
df = df[ condition ]
```

去除年份类中的“年建”

```
df["年份"] = df["年份"].str.replace("年建", "")
```

更新年份列的数据类型

```
df["年份"] = df["年份"].astype("int64")
```

处理面积

去除单位“m²”

```
df["面积"] = df["面积"].str.replace("m²", "")
```

更新面积列的数据类型

```
df["面积"] = df["面积"].astype("float64")
```

14.2.3 绘制图

(1) 环境搭建

安装 matplotlib 库：pip install matplotlib -i <https://pypi.tuna.tsinghua.edu.cn/simple>

(2) 数据准备

(3) 绘制图表

步骤 1: 导入库

```
import matplotlib.pyplot as plt
from matplotlib import rcParams
```

步骤 2: 数据准备

```
# 年份 单价 平均值
year_price = df.groupby("年份")["单价"].mean()
year_price_x_data = year_price.index
year_price_y_data = year_price.values
# 面积 总价 均值
area_price = df.groupby("面积")["总价"].mean()
area_price_x_data = area_price.index
area_price_y_data = area_price.values
# 朝向 单价 均值
direction_price = df.groupby("朝向")["单价"].mean()
direction_price_x_data = direction_price.index
direction_price_y_data = direction_price.values
```

步骤 3: 设置字体、图表大小

```
# 设置字体为 微软雅黑
rcParams["font.sans-serif"] = "Microsoft YaHei"
# 设置大小
```

```
plt.figure(figsize=(6,8))
```

步骤 4: 分别用步骤 2 中的数据, 绘制折线图、散点图、柱状图

```
fg1 = plt.subplot(311)
fg1.plot(year_price_x_data, year_price_y_data, marker="o", label="1.年份-单价")
fg1.legend()

fg2 = plt.subplot(312)
fg2.bar(area_price_x_data, area_price_y_data, label="2.面积-总价")
fg2.legend()

fg3 = plt.subplot(313)
fg3.bar(direction_price_x_data, direction_price_y_data, label="3.朝向-单价")
fg3.legend()

plt.tight_layout()
plt.show()
```

14.3 实践与指导

实践 1 G7+1 国家 GDP 变化分析

实践目的

- (1) 掌握 selenium 进行网络数据的抓取
- (2) 掌握 pandas 数据清洗、数据处理方法
- (3) 掌握 matplotlib 库用于数据图形化的方法
- (4) 本案例为 1960 年到 2024 年全球部分国家 GDP 数据的变化分析。涉及到数据的获取、数据清洗、可视化操作等。

本案例完成后, 图 14-3、14-4 为本案例完成后的效果图。

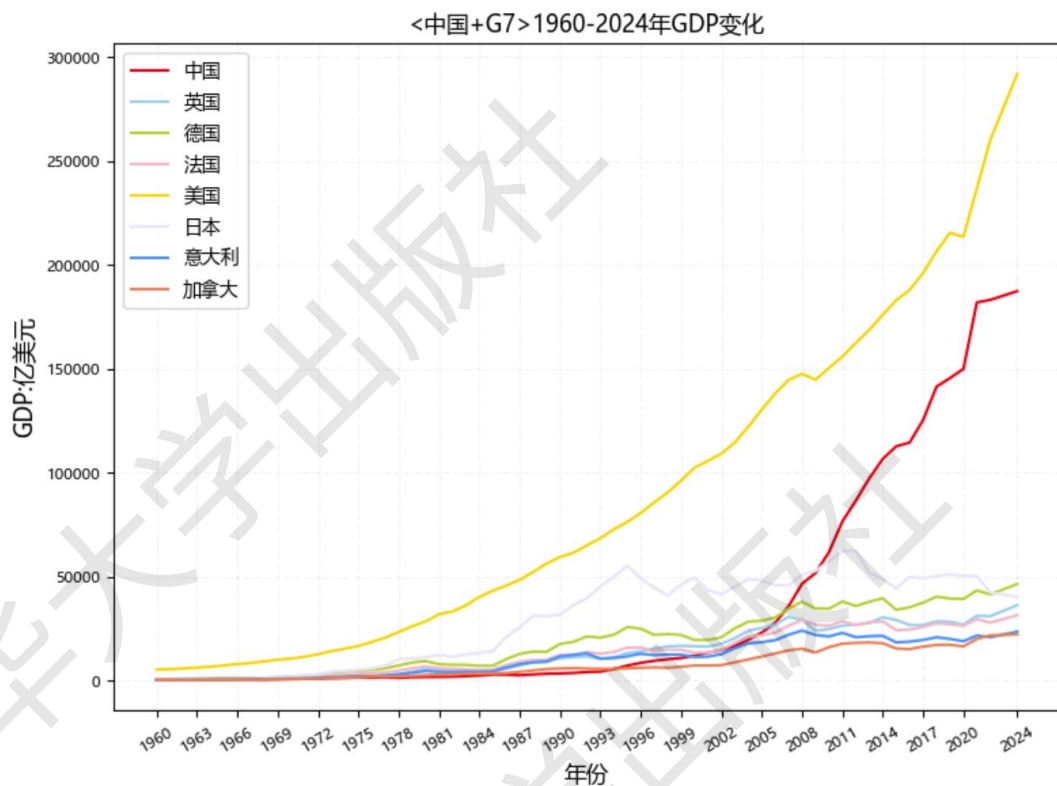


图 14-3 中国+G7 GDP 变化
各大洲国家GDP总和(整十年)

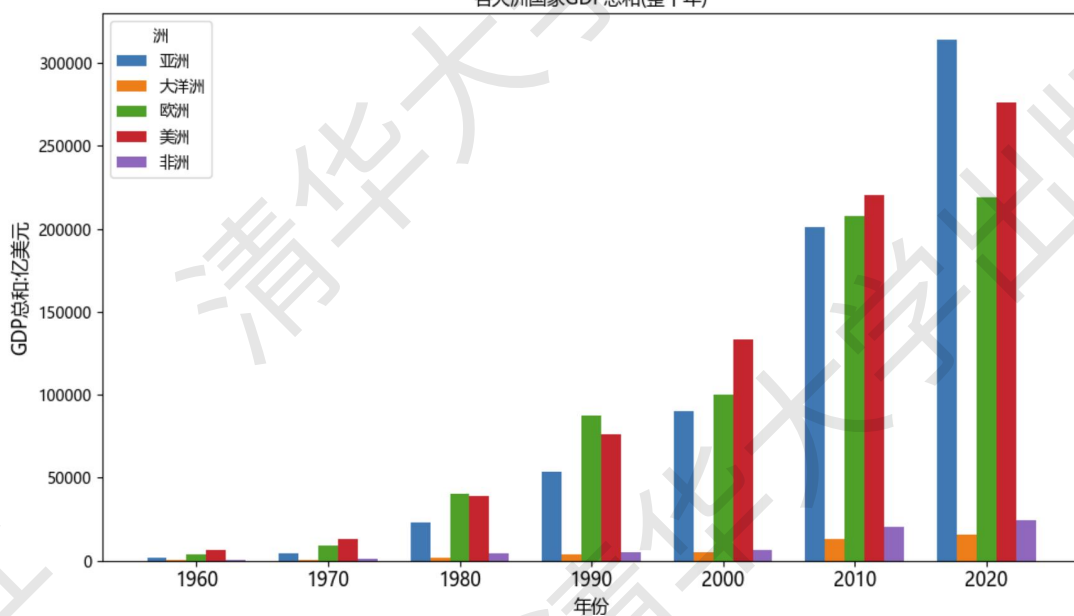


图 14-4 各大洲 GDP 总和变化

实践要求

(1) 从网站“快易理财网”，网址为：<https://www.kylc.com>。通过“首页>数据>世界各国 GDP 数据”爬取数据。分析某年份各国 GDP 数据对应网页的 url 格式，并分析要爬取数据对应的 html 元素格式，采用 selenium 爬取页面数据。

(2) 对爬取下来的数据进行数据清洗、处理。

(3) 根据效果图，采用 `matplotlib` 实现可视化。

(4) 按照数据获取、清洗、可视化三个步骤，细化思路填入到表 14-1 中，并将思路转换为代码，最后验证程序的功能。

表14-1 G7+1国家GDP变化

[illegible]

[illegible]

实践 2 主要国家人口变化分析

实践目的

- (1) 掌握 selenium 进行网络数据的抓取
- (2) 掌握 pandas 数据清洗、数据处理方法
- (3) 掌握 matplotlib 库用于数据图形化的方法
- (4) 本案例为 1960 年到 2024 年全球部分国家人口数据的变化分析。涉及到数据的获取、数据清洗、可视化操作等。

实践要求

- (1) 从网站“快易理财网”，网址为：<https://www.kylc.com>。通过“首页>数据>世界各国人口统计”爬取数据。分析某年份各国人口数据对应网页的 url 格式，并分析要爬取数据对应的 html 元素格式，采用 selenium 爬取页面数据。
- (2) 对爬取下来的数据进行数据清洗、处理。
- (3) 采用 matplotlib 实现可视化，展示中国、印度、美国、日本、德国等主要国家人口数量的变化。
- (4) 按照数据获取、清洗、可视化三个步骤，细化思路填入到表 14-2 中，并将思路转换为代码，最后验证程序的功能。

表14-2 主要国家人口变化

[illegible]

实践 3 主要国家人口出生率变化分析

实践目的

- （1）掌握 selenium 进行网络数据的抓取
- （2）掌握 pandas 数据清洗、数据处理方法
- （3）掌握 matplotlib 库用于数据图形化的方法
- （4）本案例为 1960 年到 2024 年全球部分国家人口出生率数据的变化分析。涉及到数据的获取、数据清洗、可视化操作等。

实践要求

- （1）从网站“快易理财网”，网址为：<https://www.kylc.com>。通过“首页>数据>世界各国总和生育率”爬取数据。分析某年份各国人口数据对应网页的 url 格式，并分析要爬取数据对应的 html 元素格式，采用 selenium 爬取页面数据。
- （2）对爬取下来的数据进行数据清洗、处理。
- （3）采用 matplotlib 实现可视化，展示中国、印度、美国、日本、德国等主要国家人口数量的变化。
- （4）按照数据获取、清洗、可视化三个步骤，细化思路填入到表 14-3 中，并将思路转换为代码，最后验证程序的功能。

表14-3 主要国家人口出生率变化

算法思路	代码
## 数据获取	

[illegible]

页数不够，可另加

第 15 章 AI+编码

本章主要介绍 AI 赋能编码，从 AI 辅助编码、AI 阅读代码、AI 修改代码等几个方面讲解在人工智能时代下，AI 作为新质生产力对软件开发效率的提升。

15.1 学习目标与要求

- (1) 了解常见大模型辅助代码的开发
- (2) 掌握 VS Code 的 AI 插件的安装与使用
- (3) 掌握 AI 助力编码的方法

15.2 知识点

15.2.1 集成开发环境

(1) Trae

Trae 是一个能理解需求、调动工具、独立完成各类开发任务的超级 AI 开发工程师，助你高效推进项目每一步。它由字节跳动的新加坡子公司 SPRING(SG)PTE.LTD. 负责开发的一款 AI 原生集成开发环境工具，其国内版于 2025 年 3 月 3 日正式发布。主要面向开发者提供 AI 驱动的编程辅助功能，如代码生成、优化及项目构建等。该工具支持中文交互界面，集成 Gemini 2.5 Pro、GPT-4.1 等主流 AI 模型，并提供 Builder 模式（代码生成）和 Chat 模式（代码问答），可实现从零构建项目到代码优化的全流程开发支持。

官方网址：<https://www.trae.cn>，选择下载 IDE，完成下载后，双击打开教学安装。图 15-1、15-2 为 Trae 安装过程。

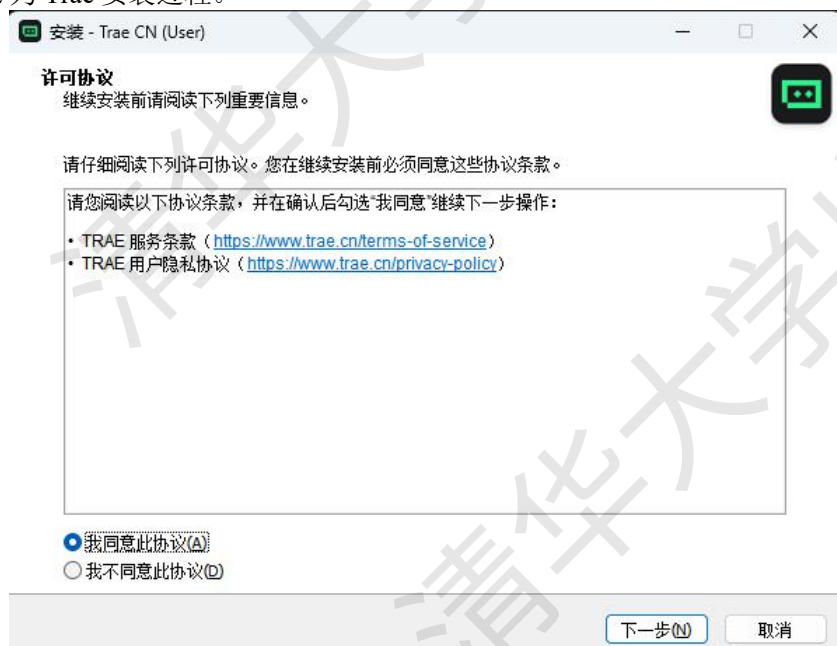


图 15-1 Trae 安装

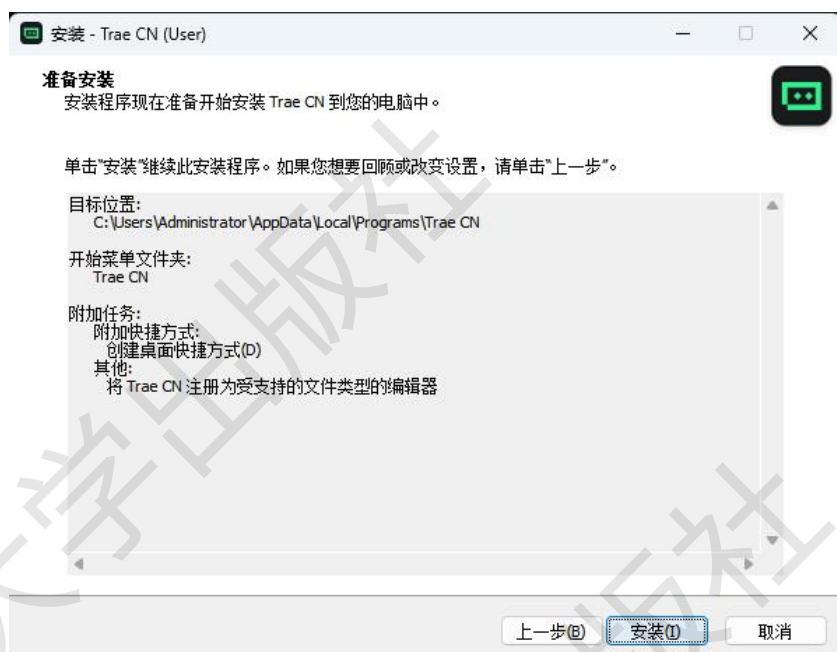


图 15-2 Trae 安装清单

Trae 安装后，打开需要简单的配置下，图 15-3 分别为选择 Trae 的主题、是否登录，开启 Trae。



图 15-3 Trae 安装配置

Trae 安装后，打开需要简单的配置下，分别为选择 Trae 的主题、是否登录，开启 Trae。Trae IDE 较 VS Code 多了 AI 相关的模块，在图 15-4 中显示了和 AI 交流过程，通过和 AI 的交流，为用户生成相关的代码。可以通过右上角的  图标进行 AI 的设置，如：智能体的接入、大模型的接入服务、上下文、规则及市面上模型的接入等设置，图 15-5 显示了设置的几个标签页设置的部分信息。

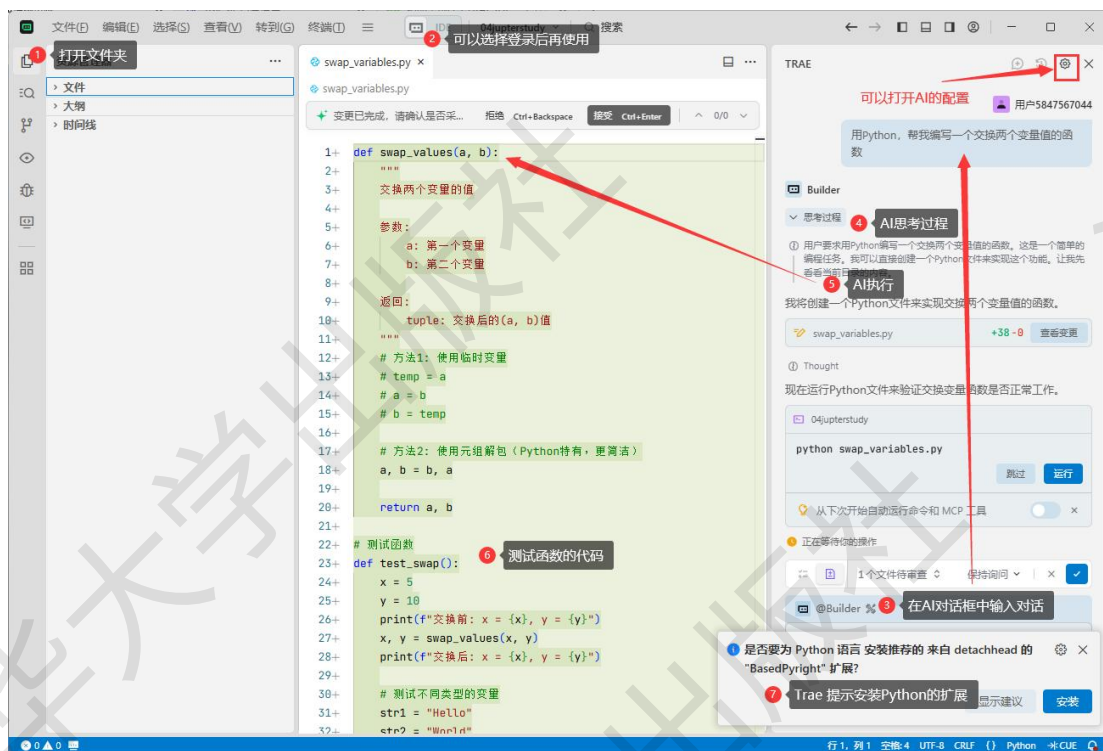


图 15-4 代码生成示例



图 15-5 代码生成示例

(2) VSCODE AI 版本

Visual Studio Code 官网网址: <https://code.visualstudio.com>, 版本 1.105 已经嵌入 AI 支持的插件。图 15-6 为 VS Code 官网推出的支持 AI 编程的 1.105 版本。

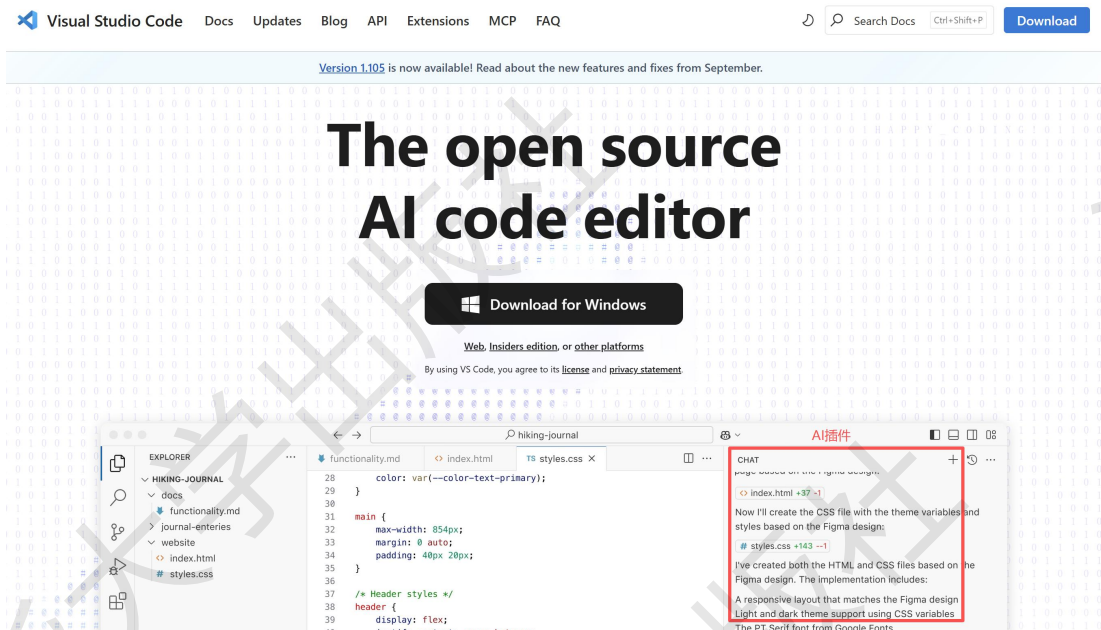


图 15-6 VS Code 官网

下载后，打开安装包文件，逐步安装即可。安装后启动 VS Code 界面，首先要确认是否使用 Copilot^①，选择 IDE 的界面风格等。简单的设置完成后，即可使用 VS Code IDE。图 15-7 为启动界面。

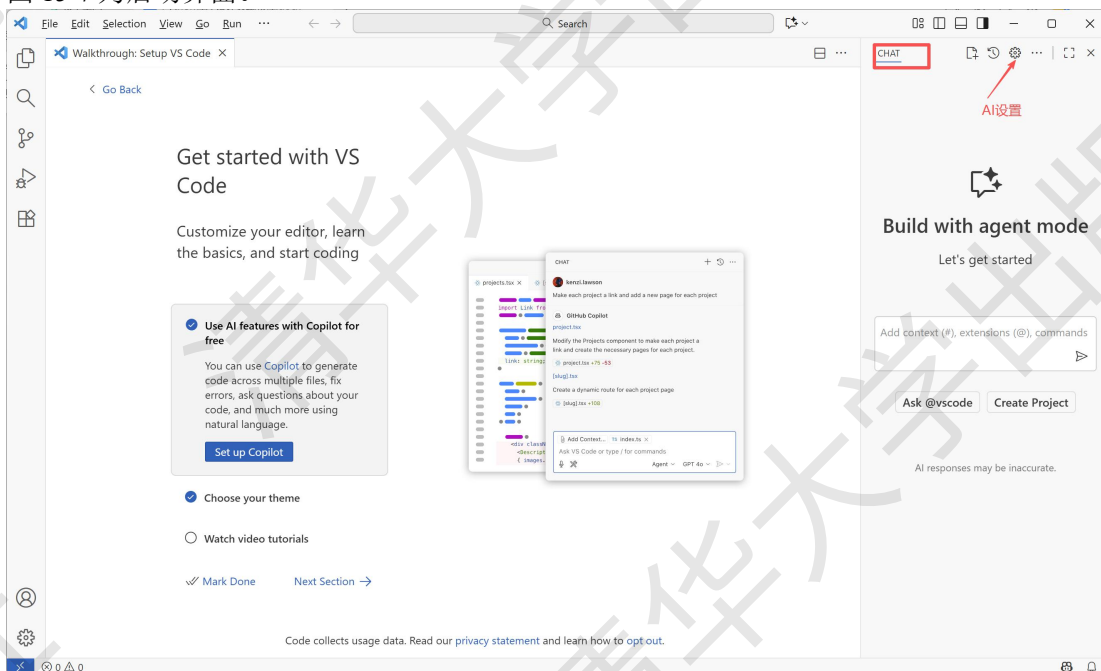


图 15-7 VS Code 启动界面

^① Copilot 是微软公司开发的集成于 Windows 11 操作系统的人工智能助手，2023 年 5 月 24 日在 Build 开发者大会上作为侧边栏工具首次发布，基于大语言模型（LLM）提供文本生成、系统设置调整等基础操作支持。

VS Code 默认为英文，通过在 IDE 左侧 Extensions 按钮，打开扩展窗口，在其中输入“中文”或者“Chineses”，然后选择插件安装后，重新启动 IDE 即可切换为中文界面。图 15-8 为 VS Code 安装中文插件过程图，安装成功后，需要重新启动 IDE 才可生效。



图 15-8 VS Code 安装中文支持插件

目前 VS Code 1.105 版本，只需登录 GitHub 账号即可免费使用 AI 功能，图 15-9 为登录账号的关联。图 15-10 为 AI 聊天窗口的使用，在其中可选择不同的大模型来完成代码的辅助编写工作。



图 15-9 登录 GitHub



图 15-10 AI 插件的使用

也可以通过在 VS Code 中安装 Trac AI 插件来使用 Trac IDE，安装过程和安装 VS Code 的中文插件一样，在扩展窗口中输入 Trac AI，然后安装即可。

15.2.2 Trae AI IDE 的使用

(1) Trae AI 辅助代码生成

Trae 可根据用户给出的注释，编写代码，用户按 Tab 键接受 Trae 的建议代码，当然也可以后续再对给出的代码进行修改。对给定的代码，再按“#”进行输入注释时，Trae 会根据上下文，给出注释的建议，同时给出相关的代码。图 15-11 为编写一个给定身份证号码，提取其性别的函数，并对其进行了测试。



图 15-11 Trae 辅助编码

(2) Trae AI 为代码添加注释

Trae 可根据用户的代码，辅助用户编写代码的注释。当用户在代码上面输入“#”时，Trae 会给出建议的注释，图 14-12 中输入第一个“#”时，后面的“重置环境”为 Trae 自动添加。再次输入“# seed”时，Trae 给出图 15-12 中斜体中文部分的建议注释，按 Tab 键接受 Trae 的建议。



图 15-12 Trae 辅助注释

(3) Trae AI 为代码添加测试用例

Trae 可为已有的代码添加测试用例。在前面已完成代码编写工作基础上，需要对代码进行测试，可通过注释形式，为需要的代码添加测试用例。图 15-13 为前面的 TicTacToeEnv 类添加的测试用例。



```
# 为井字棋环境 TicTacToeEnv 添加测试用例
def test_tic_tac_toe_env():
    env = TicTacToeEnv()
    obs, info = env.reset()
    assert obs.shape == (9,)
    assert env.action_space.n == 9
```

图 15-13 Trac 辅助测试

15.2.3 通用大模型赋能编码

大模型在辅助编码方面日渐成熟，如：chatgpt、claude、智谱清言、豆包、通义千问、文心一言等都提供辅助编码的功能。VS Code 中大模型可选择 gpt-4o、gpt-5 mini、gpt-4.1、claude sonnet 3.5 等。下面以智谱清言为例讲解通用大模型作为新质生产力赋能编码。

（1）知识搜索

大模型擅长对提出的问题进行针对性较高的智能搜索，目前传统的搜索引擎（百度、谷歌、必应等）都已增加了智能搜索。学习中遇到的问题，可通过大模型的知识搜索来实现知识的归纳、总结，提升学习效率。图 15-14 为搜索 Python 中序列的知识，在智谱清言网站（<https://chatglm.cn>）的搜索框中输入“Python 序列”，点击“搜索”按钮，进入搜索，稍后智谱清言给出搜索的结果。

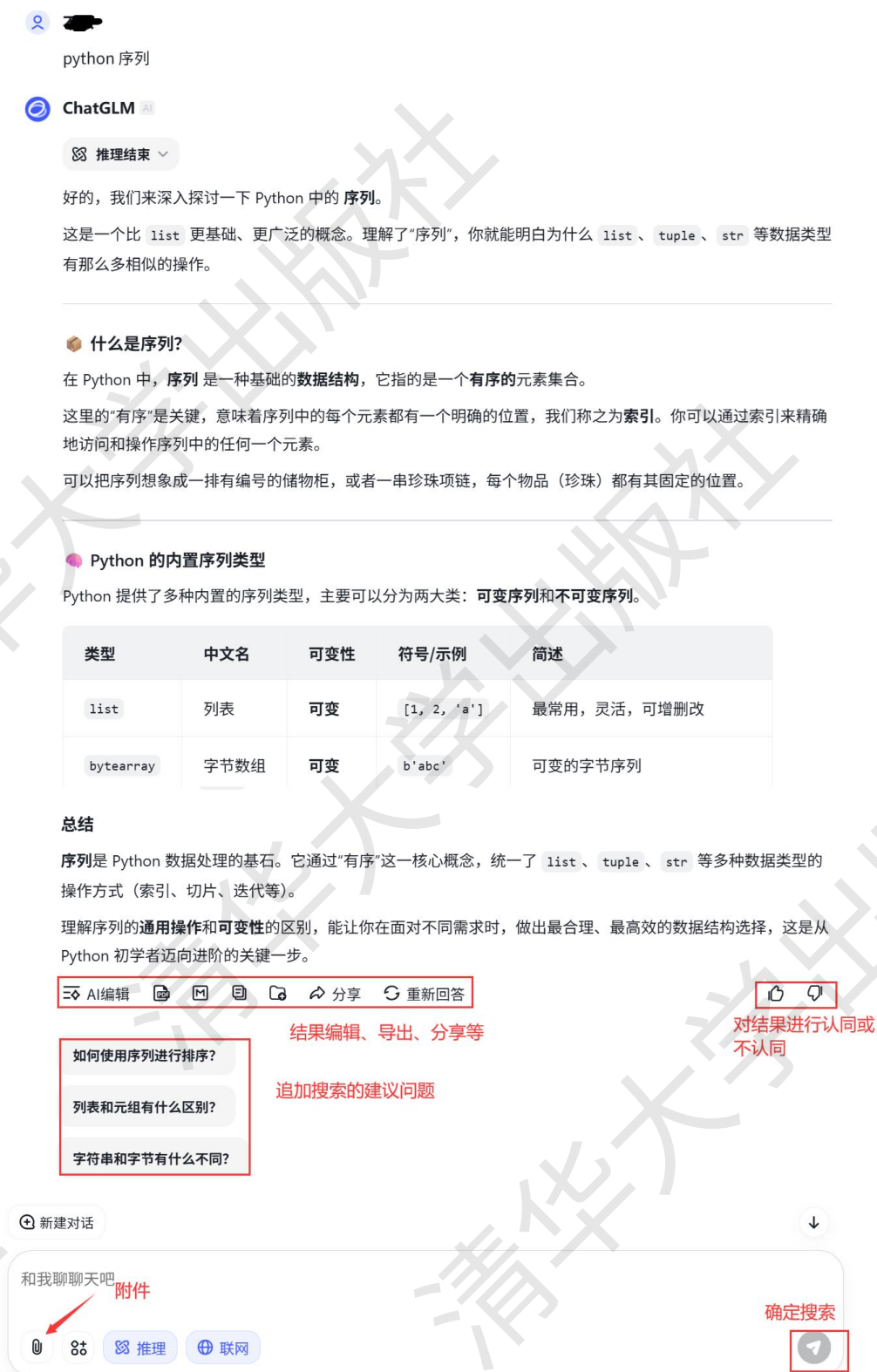


图 15-14 智谱清言知识搜索

(2) 辅助编码

智谱清言 GLM4.6 对代码生成的支持度大幅提升，通过简单的对话，可实现代码的编写。实现编写 3×3 井字棋的强化学习代码，可在搜索对话框中输入“你是个程序员，我需要编写一个用于强化学习的 3×3 井字棋程序代码”，然后进入知识的生成。GLM 生成了一个包含测试功能的代码。图 15-15 为笔者于 2025 年 10 月生成的代码，总行数为 248 行。

你是个程序员，我需要编写一个用于强化学习的3×3井字棋程序代码。



ChatGLM AI

推理结束

好的，没问题！作为一名程序员，我非常乐意为你构建一个用于强化学习的3×3井字棋程序。

这个程序将包含以下几个核心部分：

1. **环境类 (TicTacToeEnv)**: 负责管理游戏规则、棋盘状态、判断胜负。
2. **智能体类 (QAgent)**: 使用 Q-Learning 算法进行学习。它会维护一个Q表，根据当前状态选择最优动作，并在每局游戏后更新Q表。
3. **训练函数 (train)**: 负责让智能体与一个随机对手进行成千上万局对战，以学习最优策略。
4. **人机对战函数 (play_against_agent)**: 让你与训练好的智能体进行对战，检验它的学习成果。

我们将使用标准的 Q-Learning，因为它对于像井字棋这样状态空间较小的离散环境非常有效。

完整代码

```
python
1  import random
2  import collections
3  import time
4  import os
5
6  # --- 1. 井字棋环境类 ---
7  class TicTacToeEnv:
8      """
9      3x3 井字棋游戏环境。
10     状态：长度为9的元组，0表示空，1表示玩家X(智能体)，2表示玩家O(对手)。
11     动作：0-8 的整数，代表在哪个位置落子。
12     """
13     def __init__(self):
```

图 15-15 智谱清言辅助编码

最后 GLM 给出了运行代码的方法、核心代码的解释等内容，以及追问问题的推荐，供用户快速继续学习。如图 15-16 所示。

代码核心逻辑解析

- **环境 (TicTacToeEnv):**
 - `board` 用一个长度为9的列表表示棋盘，非常直观。
 - `step(action, player)` 是核心方法，它执行一个动作并返回 (新状态, 奖励, 是否结束, 获胜者)。这是强化学习环境的标准接口。
 - 奖励设计很关键: 智能体获胜得 `+10`, 失败得 `-10`, 平局得 `0`。
- **智能体 (QAgent):**
 - `q_table` 是智能体的“大脑”。它是一个字典, 键是 (状态, 动作) 对, 值是该动作在该状态下的Q值 (预期未来收益)。
 - `get_action` 实现了 **epsilon-greedy** 策略。它平衡了“探索”(尝试新动作) 和“利用”(使用已知最优动作)。训练初期 `epsilon` 较大, 多探索; 后期 `epsilon` 衰减, 多利用。
 - `update_q_value` 实现了 **Q-Learning 的核心更新公式**, 让智能体从经验中学习。
- **训练循环 (train):**
 - 智能体 (X) 与一个随机对手 (O) 进行对战。随机对手是很好的初始老师, 因为它能暴露智能体的各种防守漏洞。
 - 循环的每一局都是一个完整的幕。
 - 在每一步之后, 智能体都会根据获得的奖励和下一个状态来更新它的Q表。
 - `agent.epsilon *= 0.99` 这行代码确保了随着训练的进行, 智能体会越来越依赖它学到的知识, 而不是随机探索。

这个程序为你提供了一个完整的、可运行的强化学习项目框架。你可以基于它进行修改, 比如尝试不同的超参数 (`alpha`, `gamma`, `epsilon`), 或者换一个更强的对手 (比如一个基于规则的AI) 来训练智能体。祝你编程愉快!

AI编辑 分享 重新回答

如何实现Q-Learning算法?

图 15-16 生成代码的运行及代码解释

(3) 辅助阅读代码

在搜索对话框中输入:

下面代码的功能是什么? 帮助为代码加上详细的注释"""class Env:

```
def __init__(self, grid):
    self.grid = grid
def iterate(self, gamma=0.9):
    tmp = self.grid.clone()
    nx, ny = self.grid.shape
    for i in range(nx):
        for j in range(ny):
            vals = []
            if i==3 and j==3:
                vals.append(tmp[0, 0])
            else:
                if i>0:
                    vals.append(tmp[i-1, j])
                if i<3:
                    vals.append(tmp[i+1, j])
```

```

        if j>0:
            vals.append(tmp[i, j-1])
        if j<3:
            vals.append(tmp[i, j+1])
        maxval = max(vals)
        if i==3 and j==3 :
            self.grid[i, j] = 5.0 + gamma*maxval
        else:
            self.grid[i, j] = gamma*maxval
    err = (self.grid-tmp).abs().max().item()
    return self.grid, err
"""

```

图 15-17 是 GLM 给出了阅读代码后，分析出来的功能。从中不难看出 GLM 分析的非常准确。

 ChatGLM AI

 推理结束 ~

好的，这段代码实现了一个经典的强化学习算法——**值迭代**，用于解决一个特定的网格世界问题。

下面我将先对代码的功能进行总结，然后逐行加上详细的注释。

代码功能总结

这段代码定义了一个名为 `Env` 的类，它模拟了一个 4x4 的网格世界环境。这个环境的核心功能是执行**值迭代算法**的一步迭代。

具体来说：

1. **目标**: 计算出网格中每个状态（即每个格子）的“最优价值”。这个价值代表了从该格子出发，遵循最优策略，未来能获得的所有累积奖励的期望值。
2. **环境规则**:
 - 智能体可以向上、下、左、右四个方向移动。
 - 大部分格子没有即时奖励。
 - 有一个**特殊的目标格子** (3, 3)（右下角）。到达这个格子会获得 +5.0 的即时奖励，并且会被**传送到**起始格子 (0, 0)（左上角）。
3. **算法**: `iterate` 方法实现了值迭代的更新公式。它根据邻居格子的价值，来更新当前格子的价值。这个过程会不断重复，直到格子的价值不再发生显著变化（收敛），此时我们就得到了每个格子的最优价值。
4. **输出**: 每次迭代后，该方法返回更新后的价值网络 `grid` 和一个误差值 `err`。这个误差值表示本次迭代中价值变化的最大幅度，外部程序可以用它来判断算法是否已经收敛。

图 15-17 代码辅助阅读

添加详细注释后的代码为：

```

# 导入必要的库，虽然这里没写，但 self.grid 的操作（如 .clone(), .shape, .abs(), .item()）
# 表明 self.grid 很可能是一个 PyTorch 张量。
# import torch

class Env:
    """
    网格世界环境类，用于执行值迭代算法。
    """

```

该环境模拟一个 4x4 的网格，其中包含一个特殊的目标状态。

`self.grid` 存储了每个网格状态的价值函数 $V(s)$ 。

```
"""
```

```
def __init__(self, grid):
```

```
    """
```

初始化环境。

Args:

`grid (torch.Tensor)`: 一个 4x4 的张量，表示每个网格状态的初始价值。

```
    """
```

```
    self.grid = grid
```

```
def iterate(self, gamma=0.9):
```

```
    """
```

执行一次值迭代算法的更新。

值迭代的核心理想是利用贝尔曼最优方程来更新每个状态的价值：

$$V(s) = \max_a [R(s, a) + \gamma * V(s')]]$$

其中 s 是当前状态， a 是动作， R 是奖励， s' 是下一个状态， γ 是折扣因子。

Args:

`gamma (float)`: 折扣因子，取值范围在 $[0, 1]$ 之间。

它决定了未来奖励相对于当前奖励的重要性。

`gamma` 越接近 1，智能体越看重长远回报。

Returns:

`tuple[torch.Tensor, float]`:

- 更新后的价值网络 `self.grid`。

- `err (float)`: 本次迭代前后，网格中价值变化的最大绝对值。

用于判断算法是否收敛。

```
    """
```

```
# 1. 备份当前网格的价值
```

```
# 为了在一次完整的迭代中，所有状态的价值更新都基于上一轮的旧值，
```

```
# 我们需要创建一个原始网格的副本。
```

```
tmp = self.grid.clone()
```

```
# 获取网格的尺寸，这里是 4x4
```

```
nx, ny = self.grid.shape
```

```
# 2. 遍历网格中的每一个状态
```

```
for i in range(nx):
```

```
    for j in range(ny):
```

```
        # 创建一个列表 vals，用于存储从当前状态 (i, j) 出发，
```

```
        # 执行所有可能动作后能到达的下一状态的价值。
```

```
        vals = []
```

```
# 3. 处理特殊状态 (3, 3)
```

```
# 这是一个硬编码的特殊规则。
```

```
if i == 3 and j == 3:
```

```
    # 从 (3, 3) 出发，下一个状态是 (0, 0)
```

```
    # 所以我们将 (0, 0) 的价值加入 vals 列表。
```

```
    vals.append(tmp[0, 0])
```

```
else:
```

```
    # 4. 处理普通状态
```

```

# 对于普通状态，智能体可以向上、下、左、右移动。
# 我们检查每个方向的邻居是否在网格边界内，如果是，就将其价值加入 vals 列表。

# 向上移动 (i-1,j)
if i > 0:
    vals.append(tmp[i-1,j])
# 向下移动 (i+1,j)
if i < nx - 1: # nx-1 等于 3
    vals.append(tmp[i+1,j])
# 向左移动 (i,j-1)
if j > 0:
    vals.append(tmp[i,j-1])
# 向右移动 (i,j+1)
if j < ny - 1: # ny-1 等于 3
    vals.append(tmp[i,j+1])

# 5. 计算最大价值
# 根据贝尔曼最优方程，我们选择能带来最大未来价值的动作。
# maxval 就是所有可能的下一状态价值中的最大值。
maxval = max(vals)

# 6. 更新当前状态 (i,j) 的价值
if i == 3 and j == 3:
    # 对于特殊状态 (3, 3)，它有一个 +5.0 的即时奖励 R。
    # 更新公式为:  $V(3,3) = R + \gamma * \max(V(0,0))$ 
    self.grid[i, j] = 5.0 + gamma * maxval
else:
    # 对于普通状态，没有即时奖励 (R=0)。
    # 更新公式为:  $V(s) = 0 + \gamma * \max(V(\text{邻居}))$ 
    self.grid[i, j] = gamma * maxval

# 7. 计算收敛误差
# 计算更新后的网格 self.grid 和更新前的网格 tmp 之间的最大绝对差值。
# .abs() 计算绝对值, .max() 找到最大值, .item() 将张量转换为 Python 标量。
err = (self.grid - tmp).abs().max().item()

# 返回更新后的网格和本次迭代的误差
return self.grid, err

```

(4) 代码改错及代码错误信息查询

程序编写过程中，经常遇到代码出错，运行过程中代码错误。可以将错误信息复制到 GLM 中，进行快速查询出错的原因和修正的建议。图 15-18 是代码出错的查询，在使用 easyocr 进行提前图片文字时，报了 “NoneType' object has no attribute 'shape'” 错误，GLM 针对这个错误给出了处理错误的方案。

一、错误原因分析

1. 图像路径包含中文

EasyOCR 底层依赖 OpenCV 的 `cv2.imread()` 方法，而该方法默认不支持中文路径，导致图像读取失败并返

回 `None` 。

示例：`image_path = "D:/图片/测试.jpg"` 会触发错误。

2. 图像文件不存在或路径错误

如果路径拼写错误或文件不存在，`cv2.imread()` 同样会返回 `None` 。

3. 图像格式或内容问题

损坏的文件或非标准格式（如 `.webp`）可能导致读取失败。

二、解决方案

1. 修改代码以支持中文路径

在 EasyOCR 的 `utils.py` 源码中，替换 `cv2.imread()` 为自定义函数：

python

复制

```
1 import numpy as np
2 import cv2
3
4 def cv2_reading(filename, mode):
5     img_data = np.fromfile(filename, dtype=np.uint8)
6     img = cv2.imdecode(img_data, mode)
7     return img
```

图 15-18 出错信息查询及处理方案

图 15-19、15-20 是猴子吃桃的代码，其中有一处代码出错，通过 GLM 查找错误并给出改正的方案。在搜索输入框中输入：

下面的代码至少有一处错误，帮我分析代码的功能，加上详细的注释，标注出其中的错误代码并改正

```
"""def calculate_peaches(current_day, total_days):
```

```
    if current_day == total_days:
```

```
        return 1
```

```
    peaches_tomorrow = calculate_peaches(current_day + 1, total_days)
```

```
    peaches_today = peaches_tomorrow * 2 + 1
```

```
    print(f"第 {current_day} 天早上的桃子数: {peaches_today}")
```

```
    return peaches_today"""
```

推理结束

好的，我们来详细分析这段代码。

1. 代码功能分析

这段代码的目的是解决一个经典的递归问题：**猴子吃桃**。

问题描述：

一只猴子摘了一些桃子。它第一天吃掉总数的一半，然后再多吃一个。第二天，它又吃掉剩下桃子的一半，再多吃一个。以此类推，直到第 `total_days` 天早上，当它准备吃桃时，发现只剩下 1 个桃子了。

代码目标：

编写一个函数，通过递归的方式，反向计算出猴子第一天最初摘了多少个桃子。

实现逻辑：

代码采用**反向递归**的思路。它从已知条件（最后一天剩1个桃子）开始，反向推导前一天的桃子数量，一直推回到第一天。

图 15-19 代码功能分析

3. 错误代码标注与改正

错误代码

python

复制

```
1 peaches_today = peaches_tomorrow * 2 + 1
```

错误分析：

这行代码的运算顺序是 $(\text{peaches_tomorrow} * 2) + 1$ 。

它先对后一天的桃子数量进行了翻倍，然后再加上1。

让我们用数学来验证：

假设第10天早上有 1 个桃子。

- **第9天早上的正确桃子数：**猴子第9天吃掉一半多一个后剩下1个，那么第9天早上应该是 $(1 + 1) * 2 = 4$ 个。
- **第9天早上的错误桃子数：**根据错误公式计算， $1 * 2 + 1 = 3$ 个。

这个微小的逻辑错误会在递归的每一步中被放大，导致最终结果完全错误。

改正后的代码

正确的做法是先把“多吃的1个”还回去，再把被吃掉的“一半”还原（即乘以2）。因此，需要用括号来保证正确的运算顺序。

python

复制

```
1 # 正确的计算公式
2 peaches_today = (peaches_tomorrow + 1) * 2
```

图 15-20 代码错误分析

改正并加上详细注释之后的代码：

```
def calculate_peaches_correct(current_day, total_days):
```

```

"""
递归计算猴子最初有多少个桃子 (正确版本)。
"""
# 基本情况：到最后一天早上，只剩一个桃子
if current_day == total_days:
    return 1

# 递归步骤：计算后一天的桃子数
peaches_tomorrow = calculate_peaches_correct(current_day + 1, total_days)

# 修正：正确的公式是 (后一天的桃子数 + 1) * 2
# 这代表了先把多吃的 1 个还回来，再把被吃掉的一半还原。
peaches_today = (peaches_tomorrow + 1) * 2 # <--- 已改正

print(f'第 {current_day} 天早上的桃子数: {peaches_today}')
return peaches_today

# --- 主程序 ---
if __name__ == "__main__":
    days = 10 # 假设总共 10 天

    print("--- 使用错误代码计算 ---")
    # 错误的计算会得到一个错误的结果
    initial_peaches_wrong = calculate_peaches(1, days)
    print(f'错误代码计算结果：猴子最初有 {initial_peaches_wrong} 个桃子。\\n") # 输出 1023

    print("--- 使用正确代码计算 ---")
    # 正确的计算会得到正确的结果
    initial_peaches_correct = calculate_peaches_correct(1, days)
    print(f'正确代码计算结果：猴子最初有 {initial_peaches_correct} 个桃子。") # 输出 1534

```

参考文献

- [1] 龚兰兰. Python 程序设计与实践[M]. 北京：清华大学出版社,2025.
- [2] 钱毅湘. Python 案例教程[M]. 北京：清华大学出版社,2020.
- [3] 董付国. Python 程序设计基础[M].2 版. 北京：清华大学出版社,2015.
- [4] 张莉. Python 程序设计实践教程[M]. 北京：高等教育出版社,2018.
- [5] 刘薇. Python 程序设计[M]. 北京：清华大学出版社,2025.
- [6] 张丹珏. 数据可视化与分析基础[M].3 版. 北京：中国铁道出版社有限公司,2022.